

property. In both of the examples above, 60% of the memory accesses in the global memory region `[ptr..ptr+num_bytes)` have the persisting property and 40% of the memory accesses have the streaming property. Which specific memory accesses are classified as persisting (the `hitProp`) is random with a probability of approximately `hitRatio`; the probability distribution depends upon the hardware architecture and the memory extent.

For example, if the L2 set-aside cache size is 16KB and the `num_bytes` in the `accessPolicyWindow` is 32KB:

- ▶ With a `hitRatio` of 0.5, the hardware will select, at random, 16KB of the 32KB window to be designated as persisting and cached in the set-aside L2 cache area.
- ▶ With a `hitRatio` of 1.0, the hardware will attempt to cache the whole 32KB window in the set-aside L2 cache area. Since the set-aside area is smaller than the window, cache lines will be evicted to keep the most recently used 16KB of the 32KB data in the set-aside portion of the L2 cache.

The `hitRatio` can therefore be used to avoid thrashing of cache lines and overall reduce the amount of data moved into and out of the L2 cache.

A `hitRatio` value below 1.0 can be used to manually control the amount of data different `accessPolicyWindows` from concurrent CUDA streams can cache in L2. For example, let the L2 set-aside cache size be 16KB; two concurrent kernels in two different CUDA streams, each with a 16KB `accessPolicyWindow`, and both with `hitRatio` value 1.0, might evict each others' cache lines when competing for the shared L2 resource. However, if both `accessPolicyWindows` have a `hitRatio` value of 0.5, they will be less likely to evict their own or each others' persisting cache lines.

4.13.3. L2 Access Properties

Three types of access properties are defined for different global memory data accesses:

1. `cudaAccessPropertyStreaming`: Memory accesses that occur with the streaming property are less likely to persist in the L2 cache because these accesses are preferentially evicted.
2. `cudaAccessPropertyPersisting`: Memory accesses that occur with the persisting property are more likely to persist in the L2 cache because these accesses are preferentially retained in the set-aside portion of L2 cache.
3. `cudaAccessPropertyNormal`: This access property forcibly resets previously applied persisting access property to a normal status. Memory accesses with the persisting property from previous CUDA kernels may be retained in L2 cache long after their intended use. This persistence-after-use reduces the amount of L2 cache available to subsequent kernels that do not use the persisting property. Resetting an access property window with the `cudaAccessPropertyNormal` property removes the persisting (preferential retention) status of the prior access, as if the prior access had been without an access property.

4.13.4. L2 Persistence Example

The following example shows how to set-aside L2 cache for persistent accesses, use the set-aside L2 cache in CUDA kernels via CUDA Stream and then reset the L2 cache.

```
cudaStream_t stream;
cudaStreamCreate(&stream);
↪ // Create CUDA stream
```

(continues on next page)

(continued from previous page)

```

cudaDeviceProp prop;
→          // CUDA device properties variable
cudaGetDeviceProperties( &prop, device_id);
→          // Query GPU properties
size_t size = min( int(prop.l2CacheSize * 0.75) , prop.
→persistingL2CacheMaxSize );
cudaDeviceSetLimit( cudaLimitPersistingL2CacheSize, size);
→          // set-aside 3/4 of L2 cache for persisting accesses or the
→max allowed

size_t window_size = min(prop.accessPolicyMaxWindowSize, num_bytes);
→          // Select minimum of user defined num_bytes and max window
→size.

cudaStreamAttrValue stream_attribute;
→          // Stream level attributes data structure
stream_attribute.accessPolicyWindow.base_ptr = reinterpret_cast<void*>
→(data1);          // Global Memory data pointer
stream_attribute.accessPolicyWindow.num_bytes = window_size;
→          // Number of bytes for persistence access
stream_attribute.accessPolicyWindow.hitRatio = 0.6;
→          // Hint for cache hit ratio
stream_attribute.accessPolicyWindow.hitProp = cudaAccessPropertyPersisting;
→          // Persistence Property
stream_attribute.accessPolicyWindow.missProp = cudaAccessPropertyStreaming;
→          // Type of access property on cache miss

cudaStreamSetAttribute(stream, cudaStreamAttributeAccessPolicyWindow, &stream_
→attribute); // Set the attributes to a CUDA Stream

for(int i = 0; i < 10; i++) {
    cuda_kernelA<<<grid_size,block_size,0,stream>>>(data1);
→          // This data1 is used by a kernel multiple times
}
→          // [data1 + num_bytes) benefits from L2 persistence
cuda_kernelB<<<grid_size,block_size,0,stream>>>(data1);
→          // A different kernel in the same stream can also benefit
→
→          // from the persistence of data1

stream_attribute.accessPolicyWindow.num_bytes = 0;
→          // Setting the window size to 0 disable it
cudaStreamSetAttribute(stream, cudaStreamAttributeAccessPolicyWindow, &stream_
→attribute); // Overwrite the access policy attribute to a CUDA Stream
cudaCtxResetPersistingL2Cache();
→          // Remove any persistent lines in L2

cuda_kernelC<<<grid_size,block_size,0,stream>>>(data2);
→          // data2 can now benefit from full L2 in normal mode

```

4.13.5. Reset L2 Access to Normal

A persisting L2 cache line from a previous CUDA kernel may persist in L2 long after it has been used. Hence, a reset to normal for L2 cache is important for streaming or normal memory accesses to utilize the L2 cache with normal priority. There are three ways a persisting access can be reset to normal status.

1. Reset a previous persisting memory region with the access property, `cudaAccessPropertyNormal`.
2. Reset all persisting L2 cache lines to normal by calling `cudaCtxResetPersistingL2Cache()`.
3. **Eventually** untouched lines are automatically reset to normal. Reliance on automatic reset is strongly discouraged because of the undetermined length of time required for automatic reset to occur.

4.13.6. Manage Utilization of L2 set-aside cache

Multiple CUDA kernels executing concurrently in different CUDA streams may have a different access policy window assigned to their streams. However, the L2 set-aside cache portion is shared among all these concurrent CUDA kernels. As a result, the net utilization of this set-aside cache portion is the sum of all the concurrent kernels' individual use. The benefits of designating memory accesses as persisting diminish as the volume of persisting accesses exceeds the set-aside L2 cache capacity.

To manage utilization of the set-aside L2 cache portion, an application must consider the following:

- Size of L2 set-aside cache.
- CUDA kernels that may concurrently execute.
- The access policy window for all the CUDA kernels that may concurrently execute.
- When and how L2 reset is required to allow normal or streaming accesses to utilize the previously set-aside L2 cache with equal priority.

4.13.7. Query L2 cache Properties

Properties related to L2 cache are a part of `cudaDeviceProp` struct and can be queried using CUDA runtime API `cudaGetDeviceProperties`

CUDA Device Properties include:

- `l2CacheSize`: The amount of available L2 cache on the GPU.
- `persistingL2CacheMaxSize`: The maximum amount of L2 cache that can be set-aside for persisting memory accesses.
- `accessPolicyMaxWindowSize`: The maximum size of the access policy window.

4.13.8. Control L2 Cache Set-Aside Size for Persisting Memory Access

The L2 set-aside cache size for persisting memory accesses is queried using CUDA runtime API `cudaDeviceGetLimit` and set using CUDA runtime API `cudaDeviceSetLimit` as a `cudaLimit`. The maximum value for setting this limit is `cudaDeviceProp::persistingL2CacheMaxSize`.

```
enum cudaLimit {
    /* other fields not shown */
    cudaLimitPersistingL2CacheSize
};
```

4.14. Memory Synchronization Domains

4.14.1. Memory Fence Interference

Some CUDA applications may see degraded performance due to memory fence/flush operations waiting on more transactions than those necessitated by the CUDA memory consistency model.

```
__managed__ int x = 0;
__device__ cuda::atomic
    <int, cuda::thread_
    <scope_device> a(0);
__managed__ cuda::atomic
    <int, cuda::thread_
    <scope_system> b(0);
```

Thread 1 (SM)

```
x = 1;
a = 1;
```

Thread 2 (SM)

```
while (a != 1) ;
assert(x == 1);
b = 1;
```

Thread 3 (CPU)

```
while (b != 1) ;
assert(x == 1);
```

Consider the example above. The CUDA memory consistency model guarantees that the asserted condition will be true, so the write to `x` from thread 1 must be visible to thread 3, before the write to `b` from thread 2.

The memory ordering provided by the release and acquire of `a` is only sufficient to make `x` visible to thread 2, not thread 3, as it is a device-scope operation. The system-scope ordering provided by release and acquire of `b`, therefore, needs to ensure not only writes issued from thread 2 itself are visible to thread 3, but also writes from other threads that are visible to thread 2. This is known as *cumulativity*. As the GPU cannot know at the time of execution which writes have been guaranteed at the source level to be visible and which are visible only by chance timing, it must cast a conservatively wide net for in-flight memory operations.

This sometimes leads to interference: because the GPU is waiting on memory operations it is not required to at the source level, the fence/flush may take longer than necessary.

Note that fences may occur explicitly as intrinsics or atomics in code, like in the example, or implicitly to implement *synchronizes-with* relationships at task boundaries.

A common example is when a kernel is performing computation in local GPU memory, and a parallel kernel (e.g. from NCCL) is performing communications with a peer. Upon completion, the local kernel will implicitly flush its writes to satisfy any *synchronizes-with* relationships to downstream work. This may unnecessarily wait, fully or partially, on slower nvlink or PCIe writes from the communication kernel.

4.14.2. Isolating Traffic with Domains

Beginning with compute capability 9.0 (Hopper architecture) GPUs and CUDA 12.0, the memory synchronization domains feature provides a way to alleviate such interference. In exchange for explicit assistance from code, the GPU can reduce the net cast by a fence operation. Each kernel launch is given a domain ID. Writes and fences are tagged with the ID, and a fence will only order writes matching the fence's domain. In the concurrent compute vs communication example, the communication kernels can be placed in a different domain.

When using domains, code must abide by the rule that **ordering or synchronization between distinct domains on the same GPU requires system-scope fencing**. Within a domain, device-scope fencing remains sufficient. This is necessary for cumulativity as one kernel's writes will not be encompassed by a fence issued from a kernel in another domain. In essence, cumulativity is satisfied by ensuring that cross-domain traffic is flushed to the system scope ahead of time.

Note that this modifies the definition of `thread_scope_device`. However, because kernels will default to domain 0 as described below, backward compatibility is maintained.

4.14.3. Using Domains in CUDA

Domains are accessible via the new launch attributes `cudaLaunchAttributeMemSyncDomain` and `cudaLaunchAttributeMemSyncDomainMap`. The former selects between logical domains `cudaLaunchMemSyncDomainDefault` and `cudaLaunchMemSyncDomainRemote`, and the latter provides a mapping from logical to physical domains. The remote domain is intended for kernels performing remote memory access in order to isolate their memory traffic from local kernels. Note, however, the selection of a particular domain does not affect what memory access a kernel may legally perform.

The domain count can be queried via device attribute `cudaDevAttrMemSyncDomainCount`. Devices of compute capability 9.0 (Hopper) have 4 domains. To facilitate portable code, domains functionality can be used on all devices and CUDA will report a count of 1 on devices prior to compute capability 9.0.

Having logical domains eases application composition. An individual kernel launch at a low level in the stack, such as from NCCL, can select a semantic logical domain without concern for the surrounding application architecture. Higher levels can steer logical domains using the mapping. The default value for the logical domain if it is not set is the default domain, and the default mapping is to map the default domain to 0 and the remote domain to 1 (on GPUs with more than 1 domain). Specific libraries may tag launches with the remote domain in CUDA 12.0 and later; for example, NCCL 2.16 will do so. Together, this provides a beneficial use pattern for common applications out of the box, with no code changes needed in other components, frameworks, or at application level. An alternative use pattern, for example in an application using NVSHMEM or with no clear separation of kernel types, could be to partition parallel streams. Stream A may map both logical domains to physical domain 0, stream B to 1, and so on.

```
// Example of launching a kernel with the remote logical domain
cudaLaunchAttribute domainAttr;
domainAttr.id = cudaLaunchAttrMemSyncDomain;
domainAttr.val = cudaLaunchMemSyncDomainRemote;
cudaLaunchConfig_t config;
// Fill out other config fields
config.attrs = &domainAttr;
config.numAttrs = 1;
cudaLaunchKernelEx(&config, myKernel, kernelArg1, kernelArg2...);
```

```
// Example of setting a mapping for a stream
// (This mapping is the default for streams starting on compute capability 9.0
// →(Hopper) or later if not
// explicitly set, and provided for illustration)
cudaLaunchAttributeValue mapAttr;
mapAttr.memSyncDomainMap.default_ = 0;
mapAttr.memSyncDomainMap.remote = 1;
cudaStreamSetAttribute(stream, cudaLaunchAttributeMemSyncDomainMap, &mapAttr);
```

```
// Example of mapping different streams to different physical domains, ignoring
// logical domain settings
cudaLaunchAttributeValue mapAttr;
mapAttr.memSyncDomainMap.default_ = 0;
mapAttr.memSyncDomainMap.remote = 0;
cudaStreamSetAttribute(streamA, cudaLaunchAttributeMemSyncDomainMap, &
→mapAttr);
mapAttr.memSyncDomainMap.default_ = 1;
mapAttr.memSyncDomainMap.remote = 1;
cudaStreamSetAttribute(streamB, cudaLaunchAttributeMemSyncDomainMap, &
→mapAttr);
```

As with other launch attributes, these are exposed uniformly on CUDA streams, individual launches using `cudaLaunchKernelEx`, and kernel nodes in CUDA graphs. A typical use would set the mapping at stream level and the logical domain at launch level (or bracketing a section of stream use) as described above.

Both attributes are copied to graph nodes during stream capture. Graphs take both attributes from the node itself, essentially an indirect way of specifying a physical domain. Domain-related attributes set on the stream a graph is launched into are not used in execution of the graph.

4.15. Interprocess Communication

Communication between multiple GPUs managed by different host processes is supported through the use of interprocess communication (IPC) APIs and IPC-shareable memory buffers, by creating process-portable handles that are subsequently used to obtain process-local device pointers to the device memory on peer GPUs.

Any device memory pointer or event handle created by a host thread can be directly referenced by any other thread within the same process. However, device pointers or event handles are not valid outside the process that created them, and therefore cannot be directly referenced by threads belonging to a different process. To access device memory and CUDA events across processes, an application must use CUDA Interprocess Communication (IPC) or Virtual Memory Management APIs to create process-portable handles that can be shared with other processes using standard host operating system IPC mechanisms, e.g., interprocess shared memory or files. Once the process-portable handles have been exchanged between processes, process-local device pointers must be obtained from the handles using CUDA IPC or VMM APIs. Process-local device pointers can then be used just as they would within a single process.

The same kind of portable-handle approach used for IPC within a single-node and single operating system instance is also used for peer-to-peer communication among the GPUs in multi-node NVLink-connected clusters. In the multi-node case, communicating GPUs are managed by processes running within independent operating system instances on each cluster node, requiring additional abstraction

above the level of operating system instances. Multi-node peer communication is achieved by creating and exchanging so-called “fabric” handles between multi-node GPU peers, and by then obtaining process-local device pointers within the participating processes and operating system instances corresponding to the multi-node ranks.

See below (single-node CUDA IPC) and [ref::virtual-memory-management](#) for the specific APIs used to establish and exchange process-portable and node and operating system instance-portable handles that are used to obtain process-local device pointers for GPU communication.

Note

There are individual advantages and limitations associated with the use of the CUDA IPC APIs and Virtual Memory Management (VMM) APIs when used for IPC.

The CUDA IPC API is only currently supported on Linux platforms.

The CUDA Virtual Memory Management APIs permit per-allocation control over peer accessibility and sharing at memory allocation time, but require the use of the CUDA Driver API.

4.15.1. IPC using the Legacy Interprocess Communication API

To share device memory pointers and events across processes, an application must use the CUDA Interprocess Communication API, which is described in detail in the reference manual. The IPC API permits an application to get the IPC handle for a given device memory pointer using `cudaIpcGetMemHandle()`. A CUDA IPC handle can be passed to another process using standard host operating system IPC mechanisms, e.g., interprocess shared memory or files. `cudaIpcOpenMemHandle()` uses the IPC handle to retrieve a valid device pointer that can be used within the other process. Event handles can be shared using similar entry points.

An example of using the IPC API is where a single primary process generates a batch of input data, making the data available to multiple secondary processes without requiring regeneration or copying.

Note

The IPC API is only supported on Linux.

Note that the IPC API is not supported for `cudaMallocManaged` allocations.

Applications using CUDA IPC to communicate with each other should be compiled, linked, and run with the same CUDA driver and runtime.

Allocations made by `cudaMalloc()` may be sub-allocated from a larger block of memory for performance reasons. In such case, CUDA IPC APIs will share the entire underlying memory block which may cause other sub-allocations to be shared, which can potentially lead to information disclosure between processes. To prevent this behavior, it is recommended to only share allocations with a 2MiB aligned size.

Only the IPC events-sharing APIs are supported on L4T and embedded Linux Tegra devices with compute capability 7.x and higher. The IPC memory-sharing APIs are not supported on Tegra platforms.

4.15.2. IPC using the Virtual Memory Management API

The CUDA Virtual Memory Management API allows the creation of IPC-shareable memory allocations, and it supports multiple operating systems by virtue of operating-system specific IPC handle data structures.

4.16. Virtual Memory Management

In the CUDA programming model, memory allocation calls (such as `cudaMalloc()`) return a memory address in GPU memory. The address can be used with any CUDA API or inside a device kernel. Developers can enable peer device access to that memory allocations by using `cudaEnablePeerAccess`. By doing so, kernels on different devices can access the same data. However, all past and future user allocations are also mapped to the target peer device. This can lead to users unintentionally paying a runtime cost for mapping all `cudaMalloc` allocations to peer devices. In most situations, applications communicate by sharing only a few allocations with another device. It is usually not necessary to map all allocations to all devices. In addition, extending this approach to multi-node settings becomes inherently difficult.

CUDA provides a *virtual memory management* (VMM) API to give developers explicit, low-level control over this process.

Virtual memory allocation, a complex process managed by the operating system and the Memory Management Unit (MMU), works in two key stages. First, the OS reserves a contiguous range of virtual addresses for a program without assigning any physical memory. Then, when the program attempts to use that memory for the first time, the OS commits the virtual addresses, assigning physical storage to the virtual pages as needed.

CUDA's VMM API brings a similar concept to GPU memory management by allowing developers to explicitly reserve a virtual address range and then later map it to physical GPU memory. With VMM, applications can specifically choose certain allocations to be accessible by other devices.

The VMM API lets complex applications manage memory more efficiently across multiple GPUs (and CPU cores). By enabling manual control over memory reservation, mapping, and access permissions, the VMM API enables advanced techniques like fine-grained data sharing, zero-copy transfers, and custom memory allocators. The CUDA VMM API exposes fine-grained control to the user for managing the GPU memory in applications.

Developers can benefit from the VMM API in several key ways:

- ▶ Fine-grained control over virtual and physical memory management, allowing allocation and mapping of non-contiguous physical memory chunks to contiguous virtual address spaces. This helps reduce GPU memory fragmentation and improve memory utilization, especially for large workloads like deep neural network training.
- ▶ Efficient memory allocation and deallocation by separating the reservation of virtual address space from the physical memory allocation. Developers can reserve large virtual memory regions and map physical memory on demand without costly memory copies or reallocations, leading to performance improvements in dynamic data structures and variable-sized memory allocations.
- ▶ The ability to grow GPU memory allocations dynamically without needing to copy and reallocate all data, similar to how `realloc` or `std::vector` works in CPU memory management. This supports more flexible and efficient GPU memory use patterns.
- ▶ Enhancements to developer productivity and application performance by providing low-level APIs that allow building sophisticated memory allocators and cache management systems, such as

dynamically managing key-value caches in large language models, improving throughput and latency.

- The CUDA VMM API is highly valuable in distributed multi-GPU settings as it enables efficient memory sharing and access across multiple GPUs. By decoupling virtual addresses from physical memory, the API allows developers to create a unified virtual address space where data can be dynamically mapped to different GPUs. This optimizes memory usage and reduces data transfer overhead. For instance, NVIDIA's libraries like NCCL and NVSHMEM actively use VMM.

In summary, the CUDA VMM API gives developers advanced tools for fine-tuned, efficient, flexible, and scalable GPU memory management beyond traditional malloc-like abstractions, which is important for high-performance and large-memory applications

Note

The suite of APIs described in this section require a system that supports UVA. See The [Virtual Memory Management APIs](#).

4.16.1. Preliminaries

4.16.1.1 Definitions

Fabric Memory: Fabric memory refers to memory that is accessible over a high-speed interconnect fabric such as NVIDIA's NVLink and NVSwitch. This fabric provides a memory coherence and high-bandwidth communication layer between multiple GPUs or nodes, enabling them to share memory efficiently as if the memory is attached to a unified fabric rather than isolated on individual devices.

CUDA 12.4 and later have a VMM allocation handle type `CU_MEM_HANDLE_TYPE_FABRIC`. On supported platforms and provided the NVIDIA IMEX daemon is running, this allocation handle type enables sharing allocations not only intra-node with any communication mechanism, e.g. MPI, but also inter-node. This allows GPUs in a multi-node NVLink system to map the memory of all other GPUs that are part of the same NVLink fabric even if they are in different nodes.

Memory Handles: In VMM, handles are opaque identifiers that represent physical memory allocations. These handles are central to managing memory in the low-level CUDA VMM API. They enable flexible control over physical memory objects that can be mapped into virtual address spaces. A handle uniquely identifies a physical memory allocation. Handles serve as an abstract reference to memory resources without exposing direct pointers. Handles allow operations like exporting and importing memory across processes or devices, facilitating memory sharing and virtualization.

IMEX Channels: The name IMEX stands for *internode memory exchange* and is part of NVIDIA's solution for GPU-to-GPU communication across different nodes. IMEX channels are a GPU driver feature that provides user-based memory isolation in multi-user or multi-node environments within an IMEX domain. IMEX channels serve as a security and isolation mechanism.

IMEX channels are directly related to the fabric handle and has to be enabled in multi-node GPU communication. When a GPU allocates memory and wants to make it accessible to a GPU on a different node, it first needs to export a handle to that memory. The IMEX channel is used during this export process to generate a secure fabric handle that can only be imported by a remote process with the correct channel access.

Unicast Memory Access: Unicast memory access in the context of VMM API refers to the controlled, direct mapping and access of physical memory to a unique virtual address range by a specific device or process. Instead of broadcasting access to multiple devices, unicast memory access means that a particular GPU device is granted explicit read/write permissions to a reserved virtual address range that maps to a physical memory allocation.

Multicast Memory Access: Multicast memory access in the context of the VMM API refers to the capability for a single physical memory allocation or region to be mapped simultaneously to multiple devices' virtual address spaces using a multicast mechanism. This allows data to be efficiently shared in a one-to-many fashion across multiple GPUs, reducing redundant data transfers and improving communication efficiency. NVIDIA's CUDA VMM API supports creating a multicast object that binds together physical memory allocations from multiple devices.

4.16.1.2 Query for Support

Applications should query if features are supported before attempting to use them, as their availability can vary depending on the GPU architecture, driver version, and specific software libraries being used. The following sections detail how to programmatically check for the necessary support.

VMM Support Before attempting to use VMM APIs, applications must ensure that the devices they want to use support CUDA virtual memory management. The following code sample shows querying for VMM support:

```
int deviceSupportsVmm;
CUresult result = cuDeviceGetAttribute(&deviceSupportsVmm, CU_DEVICE_
↳ATTRIBUTE_VIRTUAL_MEMORY_MANAGEMENT_SUPPORTED, device);
if (deviceSupportsVmm != 0) {
    // `device` supports Virtual Memory Management
}
```

Fabric Memory Support: Before attempting to use fabric memory, applications must ensure that the devices they want to use support fabric memory. The following code sample shows querying for fabric memory support:

```
int deviceSupportsFabricMem;
CUresult result = cuDeviceGetAttribute(&deviceSupportsFabricMem, CU_DEVICE_
↳ATTRIBUTE_HANDLE_TYPE_FABRIC_SUPPORTED, device);
if (deviceSupportsFabricMem != 0) {
    // `device` supports Fabric Memory
}
```

Aside from using `CU_MEM_HANDLE_TYPE_FABRIC` as the handle type and not requiring OS native mechanisms for inter-process communication to exchange shareable handles, there is no difference in using fabric memory compared to other allocation handle types.

IMEX Channels Support Within an IMEX domain, IMEX channels enable secure memory sharing in multi-user environments. The NVIDIA driver implements this by creating a character device, `nvidia-caps-imex-channels`. To use fabric handle-based sharing, users should verify two things:

- First, applications must verify that this device exists under `/proc/devices`:

```
# cat /proc/devices | grep nvidia
195 nvidia
195 nvidiactl
234 nvidia-caps-imex-channels
509 nvidia-nvswitch
```

The `nvidia-caps-imex-channels` device should have a major number (e.g., `234`).

- Second, for two CUDA processes (an exporter and an importer) to share memory, they must both have access to the same IMEX channel file. These files, such as `/dev/nvidia-caps-imex-channels/channel0`, are nodes that represent individual IMEX channels. System administrators

must create these files, for example, using the `mknod()` command.

```
# mknod /dev/nvidia-caps-imex-channels/channelN c <major_number> 0
```

This command creates `channelN` using the major number obtained from `/proc/devices`.

Note

By default, the driver can create `channel0` if the `NVreg_CreatImexChannel0` module parameter is specified.

Multicast Object Support: Before attempting to use multicast objects, applications must ensure that the devices they want to use support them. The following code sample shows querying for multicast object support:

```
int deviceSupportsMultiCast;
CUresult result = cuDeviceGetAttribute(&deviceSupportsMultiCast, CU_DEVICE_
    ↪ATTRIBUTE_MULTICAST_SUPPORTED, device);
if (deviceSupportsMultiCast != 0) {
    // `device` supports Multicast Objects
}
```

4.16.2. API Overview

The VMM API provides developers with granular control over virtual memory management. VMM, being a very low-level API, requires use of the *CUDA Driver API* directly. This versatile API can be used in both single-node and multi-node environments.

To use VMM effectively, developers must have a solid grasp of a few key concepts in memory management:

- ▶ Knowledge of the operating system's virtual memory fundamentals, including how it handles pages and address spaces
- ▶ An understanding of memory hierarchy and hardware characteristics is necessary
- ▶ Familiarity with inter-process communication (IPC) methods, such as sockets or message passing,
- ▶ A basic knowledge of security for memory access rights

The VMM API workflow involves a sequence of steps for memory management, with a key focus on sharing memory between different devices or processes. Initially, a developer must allocate physical memory on the source device. To facilitate sharing, the VMM API utilizes handles to convey necessary information to the target device or process. The user must export a handle for sharing, which can be either an OS-specific handle or a fabric-specific handle. OS-specific handles are limited to inter-process communication on a single node, while fabric-specific handles offer greater versatility and can be used in both single-node and multi-node environments. It's important to note that using fabric-specific handles requires the enablement of IMEX channels.

Once the handle is exported, it must be shared with the receiving process or processes using an inter-process communication protocol, with the choice of method left to the developer. The receiving process then uses the VMM API to import the handle. After the handle has been successfully exported, shared, and imported, both the source and target processes must reserve virtual address space where

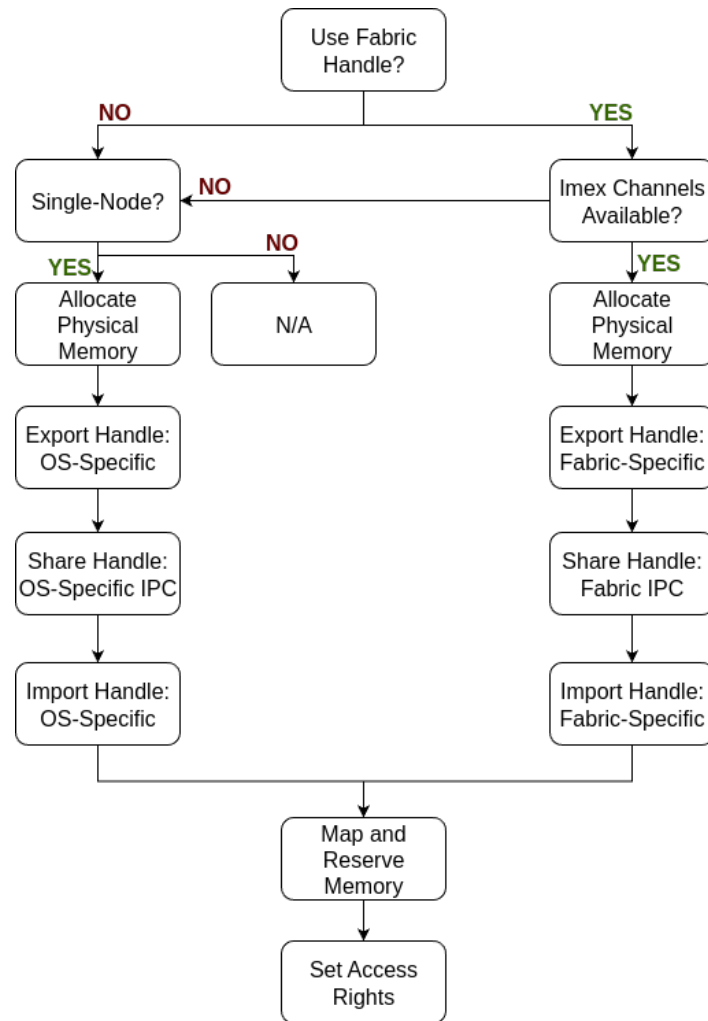


Figure 55: VMM Usage Overview. This diagram outlines the series of steps required for VMM utilization. The process begins by evaluating the environmental setup. Based on this assessment, the user must make a critical initial decision: whether to utilize fabric memory handles or OS-specific handles. A distinct series of subsequent steps must be taken based on the initial handle choice. However, the final memory management operations—specifically mapping, reserving, and setting access rights of the allocated memory—are identical, regardless of the type of handle that was selected.

the allocated physical memory will be mapped. The final step is to set the memory access rights for each device, ensuring proper permissions are established. This entire process, including both handle approaches, is further detailed in the accompanying figure.

4.16.3. Unicast Memory Sharing

Sharing GPU memory can happen on one machine with multiple GPUs or across a network of machines. The process follows these steps:

- **Allocate and Export:** A CUDA program on one GPU allocates memory and gets a sharable handle for it.
- **Share and Import:** The handle is then sent to other programs on the node using IPC, MPI, or NCCL etc. In the receiving GPUs, the CUDA driver imports the handle and creates the necessary memory objects.
- **Reserve and Map:** The driver creates a mapping from the program's Virtual Address (VA) to the GPU's Physical Address (PA) to its network Fabric Address (FA).
- **Access Rights:** Setting access rights for the allocation.
- **Releasing the Memory:** Freeing all allocations when program ends its execution.

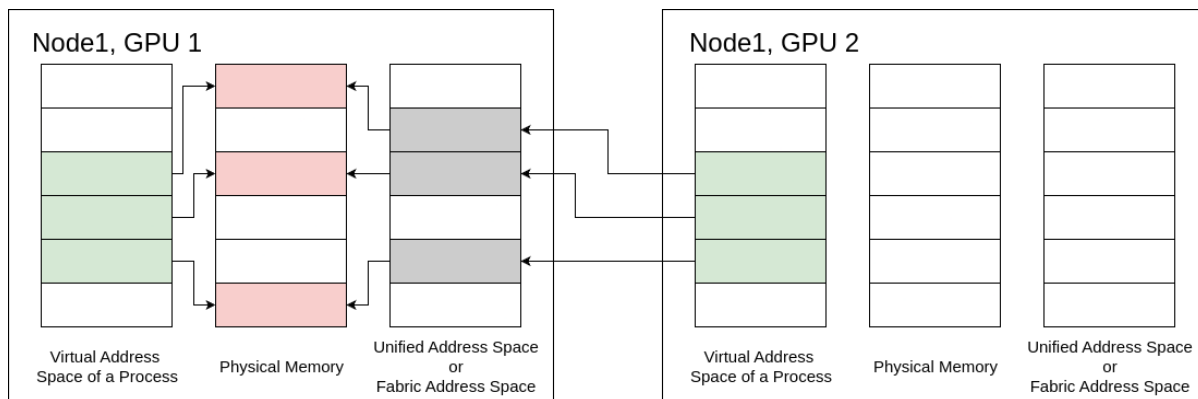


Figure 56: Unicast Memory Sharing Example

4.16.3.1 Allocate and Export

Allocating Physical Memory The first step in memory allocation using virtual memory management APIs is to create a physical memory chunk that will provide a backing for the allocation. In order to allocate physical memory, applications must use the `cuMemCreate` API. The allocation created by this function does not have any device or host mappings. The function argument `CUMemGenericAllocationHandle` describes the properties of the memory to allocate such as the location of the allocation, if the allocation is going to be shared to another process (or graphics APIs), or the physical attributes of the memory to be allocated. Users must ensure the requested allocation's size is aligned to appropriate granularity. Information regarding an allocation's granularity requirements can be queried using `cuMemGetAllocationGranularity`.

OS-Specific Handle (Linux)

```

CUmemGenericAllocationHandle allocatePhysicalMemory(int device, size_t size) {
    CUmemAllocationHandleType handleType = CU_MEM_HANDLE_TYPE_POSIX_FILE_
↪DESCRIPTOR;
    CUmemAllocationProp prop = {};
    prop.type = CU_MEM_ALLOCATION_TYPE_PINNED;
    prop.location.type = CU_MEM_LOCATION_TYPE_DEVICE;
    prop.location.id = device;
    prop.requestedHandleType = handleType;

    size_t granularity = 0;
    cuMemGetAllocationGranularity(&granularity, &prop, CU_MEM_ALLOC_
↪GRANULARITY_MINIMUM);

    // Ensure size matches granularity requirements for the allocation
    size_t padded_size = ROUND_UP(size, granularity);

    // Allocate physical memory
    CUmemGenericAllocationHandle allocHandle;
    cuMemCreate(&allocHandle, padded_size, &prop, 0);

    return allocHandle;
}

```

Fabric Handle

```

CUmemGenericAllocationHandle allocatePhysicalMemory(int device, size_t size) {
    CUmemAllocationHandleType handleType = CU_MEM_HANDLE_TYPE_FABRIC;
    CUmemAllocationProp prop = {};
    prop.type = CU_MEM_ALLOCATION_TYPE_PINNED;
    prop.location.type = CU_MEM_LOCATION_TYPE_DEVICE;
    prop.location.id = device;
    prop.requestedHandleType = handleType;

    size_t granularity = 0;
    cuMemGetAllocationGranularity(&granularity, &prop, CU_MEM_ALLOC_
↪GRANULARITY_MINIMUM);

    // Ensure size matches granularity requirements for the allocation
    size_t padded_size = ROUND_UP(size, granularity);

    // Allocate physical memory
    CUmemGenericAllocationHandle allocHandle;
    cuMemCreate(&allocHandle, padded_size, &prop, 0);

    return allocHandle;
}

```

Note

The memory allocated by `cuMemCreate` is referenced by the `CUmemGenericAllocationHandle` it returns. Note that this reference is not a pointer and its memory is not accessible yet.

Note

Properties of the allocation handle can be queried using `cuMemGetAllocationPropertiesFromHandle`.

Exporting Memory Handle The CUDA virtual memory management API expose a new mechanism for interprocess communication using handles to exchange necessary information about the allocation and physical address space. One can export handles for OS-specific IPC or fabric-specific IPC. OS-specific IPC handles can only be used on a single-node setup. Fabric-specific handles can be used on a single or multi node setups.

OS-Specific Handle (Linux)

```
CUmemAllocationHandleType handleType = CU_MEM_HANDLE_TYPE_POSIX_FILE_
↪DESCRIPTOR;
CUmemGenericAllocationHandle handle = allocatePhysicalMemory(0, 1<<21);
int fd;
cuMemExportToShareableHandle(&fd, handle, handleType, 0);
```

Fabric Handle

```
CUmemAllocationHandleType handleType = CU_MEM_HANDLE_TYPE_FABRIC;
CUmemGenericAllocationHandle handle = allocatePhysicalMemory(0, 1<<21);
CUmemFabricHandle fh;
cuMemExportToShareableHandle(&fh, handle, handleType, 0);
```

Note

OS-specific handles require all processes to be part of the same OS.

Note

Fabric-specific handles require IMEX channels to be enabled by sysadmin.

The `memMapIpcDrv` sample can be used as an example for using IPC with VMM allocations.

4.16.3.2 Share and Import

Sharing Memory Handle Once the handle is exported, it must be shared with the receiving process or processes using an inter-process communication protocol. The developer is free to use any method for sharing the handle. The specific IPC method used depends on the application's design and environment. Common methods include OS-specific inter-process sockets and distributed message passing. Using OS-specific IPC offers high-performance transfer, but is limited to processes on the same machine and not portable. Fabric-specific IPC is simpler and more portable. However, fabric-specific IPC

requires system-level support. The chosen method must securely and reliably transfer the handle data to the target process so it can be used to import the memory and establish a valid mapping. The flexibility in choosing the IPC method allows the VMM API to be integrated into a wide range of system architectures, from single-node applications to distributed, multi-node setups. In the following code snippets, we'll provide examples for sharing and receiving handles using both socket programming and MPI.

Send: OS-Specific IPC (Linux)

```
int ipcSendShareableHandle(int socket, int fd, pid_t process) {
    struct msghdr msg;
    struct iovec iov[1];

    union {
        struct cmsghdr cm;
        char* control;
    } control_un;

    size_t sizeof_control = CMSG_SPACE(sizeof(int)) * sizeof(char);
    control_un.control = (char*) malloc(sizeof_control);

    struct cmsghdr *cmptr;
    ssize_t readResult;
    struct sockaddr_un cliaddr;
    socklen_t len = sizeof(cliaddr);

    // Construct client address to send this SHareable handle to
    memset(&cliaddr, 0, sizeof(cliaddr));
    cliaddr.sun_family = AF_UNIX;
    char temp[20];
    sprintf(temp, "%s%u", "/tmp/", process);
    strcpy(cliaddr.sun_path, temp);
    len = sizeof(cliaddr);

    // Send corresponding shareable handle to the client
    int sendfd = fd;

    msg.msg_control = control_un.control;
    msg.msg_controllen = sizeof_control;

    cmptr = CMSG_FIRSTHDR(&msg);
    cmptr->cmsg_len = CMSG_LEN(sizeof(int));
    cmptr->cmsg_level = SOL_SOCKET;
    cmptr->cmsg_type = SCM_RIGHTS;

    memmove(CMSG_DATA(cmptr), &sendfd, sizeof(sendfd));

    msg.msg_name = (void *)&cliaddr;
    msg.msg_namelen = sizeof(struct sockaddr_un);

    iov[0].iov_base = (void *)"";
    iov[0].iov_len = 1;
    msg.msg_iov = iov;
```

(continues on next page)

(continued from previous page)

```

msg.msg_iovlen = 1;

ssize_t sendResult = sendmsg(socket, &msg, 0);
if (sendResult <= 0) {
    perror("IPC failure: Sending data over socket failed");
    free(control_un.control);
    return -1;
}

free(control_un.control);
return 0;
}

```

Send: OS-Specific IPC (WIN)

```

int ipcSendShareableHandle(HANDLE *handle, HANDLE &shareableHandle, PROCESS_
↪INFORMATION process) {
    HANDLE hProcess = OpenProcess(PROCESS_DUP_HANDLE, FALSE, process.
↪dwProcessId);
    HANDLE hDup = INVALID_HANDLE_VALUE;
    DuplicateHandle(GetCurrentProcess(), shareableHandle, hProcess, &hDup, 0,
↪FALSE, DUPLICATE_SAME_ACCESS);
    DWORD cbWritten;
    WriteFile(handle->hMailslot[i], &hDup, (DWORD)sizeof(hDup), &cbWritten,
↪(LPOVERLAPPED)NULL);
    CloseHandle(hProcess);
    return 0;
}

```

Send: Fabric IPC

```

MPI_Send(&fh, sizeof(CUmemFabricHandle), MPI_BYTE, 1, 0, MPI_COMM_WORLD);

```

Receive: OS-Specific IPC (Linux)

```

int ipcRecvShareableHandle(int socket, int* fd) {
    struct msghdr msg = {0};
    struct iovec iov[1];
    struct cmsghdr cm;

    // Union to guarantee alignment requirements for control array
    union {
        struct cmsghdr cm;
        // This will not work on QNX as QNX CMSG_SPACE calls __cmsg_alignbytes
        // And __cmsg_alignbytes is a runtime function instead of compile-time
↪macros
        // char control[CMSG_SPACE(sizeof(int))]
        char* control;
    } control_un;
}

```

(continues on next page)

(continued from previous page)

```

size_t sizeof_control = CMSG_SPACE(sizeof(int)) * sizeof(char);
control_un.control = (char*) malloc(sizeof_control);
struct cmsghdr *cmptr;
ssize_t n;
int receivedfd;
char dummy_buffer[1];
ssize_t sendResult;
msg.msg_control = control_un.control;
msg.msg_controllen = sizeof_control;

iov[0].iov_base = (void *)dummy_buffer;
iov[0].iov_len = sizeof(dummy_buffer);

msg.msg_iov = iov;
msg.msg_iovlen = 1;
if ((n = recvmsg(socket, &msg, 0)) <= 0) {
    perror("IPC failure: Receiving data over socket failed");
    free(control_un.control);
    return -1;
}

if (((cmptr = CMSG_FIRSTHDR(&msg)) != NULL) &&
    (cmptr->cmsg_len == CMSG_LEN(sizeof(int)))) {
    if ((cmptr->cmsg_level != SOL_SOCKET) || (cmptr->cmsg_type != SCM_
→RIGHTS)) {
        free(control_un.control);
        return -1;
    }

    memmove(&receivedfd, CMSG_DATA(cmptr), sizeof(receivedfd));
    *fd = receivedfd;
} else {
    free(control_un.control);
    return -1;
}

free(control_un.control);
return 0;
}

```

Receive: OS-Specific IPC (WIN)

```

int ipcRecvShareableHandle(HANDLE &handle, HANDLE *shareableHandle) {
    DWORD cbRead;
    ReadFile(handle, shareableHandle, (DWORD)sizeof(*shareableHandles), &
→cbRead, NULL);
    return 0;
}

```

Receive: Fabric IPC

```
MPI_Recv(&fh, sizeof(CUmemFabricHandle), MPI_BYTE, 1, 0, MPI_COMM_WORLD);
```

Importing Memory Handle Again, the user can import handles for OS-specific IPC or fabric-specific IPC. OS-specific IPC handles can only be used on a single-node. Fabric-specific handles can be used for single- or multi-node.

OS-Specific Handle (Linux)

```
CUmemAllocationHandleType handleType = CU_MEM_HANDLE_TYPE_POSIX_FILE_
    ↳DESCRIPTOR;
cuMemImportFromShareableHandle(handle, (void*) &fd, handleType);
```

Fabric Handle

```
CUmemAllocationHandleType handleType = CU_MEM_HANDLE_TYPE_FABRIC;
cuMemImportFromShareableHandle(handle, (void*) &fh, handleType);
```

4.16.3.3 Reserve and Map

Reserving a Virtual Address Range

Since notions of address and memory are distinct in VMM, applications must carve out an address range that can hold the memory allocations made by `cuMemCreate`. The address range reserved must be at least as large as the sum of the sizes of all the physical memory allocations the user plans to place in them.

Applications can reserve a virtual address range by passing appropriate parameters to `cuMemAddressReserve`. The address range obtained will not have any device or host physical memory associated with it. The reserved virtual address range can be mapped to memory chunks belonging to any device in the system, thus providing the application a continuous VA range backed and mapped by memory belonging to different devices. Applications are expected to return the virtual address range back to CUDA using `cuMemAddressFree`. Users must ensure that the entire VA range is unmapped before calling `cuMemAddressFree`. These functions are conceptually similar to `mmap` and `munmap` on Linux or `VirtualAlloc` and `VirtualFree` on Windows. The following code snippet illustrates the usage for the function:

```
CUdeviceptr ptr;
// `ptr` holds the returned start of virtual address range reserved.
CUresult result = cuMemAddressReserve(&ptr, size, 0, 0, 0); // alignment = 0
    ↳for default alignment
```

Mapping Memory

The allocated physical memory and the carved out virtual address space from the previous two sections represent the memory and address distinction introduced by the VMM APIs. For the allocated memory to be usable, the user must map the memory to the address space. The address range obtained from `cuMemAddressReserve` and the physical allocation obtained from `cuMemCreate` or `cuMemImportFromShareableHandle` must be associated with each other by using `cuMemMap`.

Users can associate allocations from multiple devices to reside in contiguous virtual address ranges as long as they have carved out enough address space. To decouple the physical allocation and the address range, users must unmap the address of the mapping with `cuMemUnmap`. Users can map and unmap memory to the same address range as many times as they want, so long as they ensure

that they don't attempt to create mappings on VA range reservations that are already mapped. The following code snippet illustrates the usage for the function:

```
CUdeviceptr ptr;
// `ptr`: address in the address range previously reserved by
// ↪ cuMemAddressReserve.
// `allocHandle`: CUmemGenericAllocationHandle obtained by a previous call to
// ↪ cuMemCreate.
CUresult result = cuMemMap(ptr, size, 0, allocHandle, 0);
```

4.16.3.4 Access Rights

CUDA's virtual memory management APIs enable applications to explicitly protect their VA ranges with access control mechanisms. Mapping the allocation to a region of the address range using `cuMemMap` does not make the address accessible, and would result in a program crash if accessed by a CUDA kernel. Users must specifically select access control using the `cuMemSetAccess` function on source and accessing devices. This allows or restricts access for specific devices to a mapped address range. The following code snippet illustrates the usage for the function:

```
void setAccessOnDevice(int device, CUdeviceptr ptr, size_t size) {
    CUmemAccessDesc accessDesc = {};
    accessDesc.location.type = CU_MEM_LOCATION_TYPE_DEVICE;
    accessDesc.location.id = device;
    accessDesc.flags = CU_MEM_ACCESS_FLAGS_PROT_READWRITE;

    // Make the address accessible
    cuMemSetAccess(ptr, size, &accessDesc, 1);
}
```

The access control mechanism exposed with VMM allows users to be explicit about which allocations they want to share with other peer devices on the system. As specified earlier, `cudaEnablePeerAccess` forces all prior and future allocations made with `cudaMalloc` to be mapped to the target peer device. This can be convenient in many cases as user doesn't have to worry about tracking the mapping state of every allocation to every device in the system. But this approach [has performance implications](#). With access control at allocation granularity, VMM allows peer mappings with minimal overhead.

The `vectorAddMMAP` [sample](#) can be used as an example for using the Virtual Memory Management APIs.

4.16.3.5 Releasing the Memory

To release the allocated memory and address space, both the source and target processes should use `cuMemUnmap`, `cuMemRelease`, and `cuMemAddressFree` functions in that order. The `cuMemUnmap` function un-maps a previously mapped memory region from an address range, effectively detaching the physical memory from the reserved virtual address space. Next, `cuMemRelease` deallocates the physical memory that was previously created, returning it to the system. Finally, `cuMemAddressFree` frees a virtual address range that was previously reserved, making it available for future use. This specific order ensures a clean and complete deallocation of both the physical memory and the virtual address space.

```
cuMemUnmap(ptr, size);
cuMemRelease(handle);
cuMemAddressFree(ptr, size);
```

Note

In the OS-specific case, the exported handle must be closed using `fclose`. This step is not applicable to the fabric-based case.

4.16.4. Multicast Memory Sharing

The [Multicast Object Management APIs](#) provide a way for the application to create multicast objects and, in combination with the [Virtual Memory Management APIs](#) described above, allow applications to leverage NVLink SHARP on supported NVLink connected GPUs connected with NVSwitch. NVLink SHARP allows CUDA applications to leverage in-fabric computing to accelerate operations like broadcast and reductions between GPUs connected with NVSwitch. For this to work, multiple NVLink connected GPUs form a multicast team and each GPU from the team backs up a multicast object with physical memory. So a multicast team of N GPUs has N physical replicas of a multicast object, each local to one participating GPU. The [multimem PTX instructions](#) using mappings of multicast objects work with all replicas of the multicast object.

To work with multicast objects, an application needs to

- ▶ Query multicast support.
- ▶ Create a multicast handle with `cuMulticastCreate`.
- ▶ Share the multicast handle with all processes that control a GPU which should participate in a multicast team. This works with `cuMemExportToShareableHandle` as described above.
- ▶ Add all GPUs that should participate in the multicast team with `cuMulticastAddDevice`.
- ▶ For each participating GPU, bind physical memory allocated with `cuMemCreate` as described above to the multicast handle. All devices need to be added to the multicast team before binding memory on any device.
- ▶ Reserve an address range, map the multicast handle and set access rights as described above for regular unicast mappings. Unicast and multicast mappings to the same physical memory are possible. See the [Virtual Aliasing Support](#) section on how to ensure consistency between multiple mappings to the same physical memory.
- ▶ Use the [multimem PTX instructions](#) with the multicast mappings.

The `multi_node_p2p` example in the [Multi GPU Programming Models](#) GitHub repository contains a complete example using fabric memory including multicast objects to leverage NVLink SHARP. Please note that this example is for developers of libraries like NCCL or NVSHMEM. It shows how higher-level programming models like NVSHMEM work internally within a (multi-node) NVLink domain. Application developers generally should use the higher-level MPI, NCCL, or NVSHMEM interfaces instead of this API.

4.16.4.1 Allocating Multicast Objects

Multicast objects can be created with `cuMulticastCreate`:

```
CUmemGenericAllocationHandle createMCHandle(int numDevices, size_t size) {
    CUmemAllocationProp mcProp = {};
    mcProp.numDevices = numDevices;
    mcProp.handleTypes = CU_MEM_HANDLE_TYPE_FABRIC; // or on single node CU_
    ↪ MEM_HANDLE_TYPE_POSIX_FILE_DESCRIPTOR
```

(continues on next page)

(continued from previous page)

```

    size_t granularity = 0;
    cuMulticastGetGranularity(&granularity, &mcProp, CU_MEM_ALLOC_GRANULARITY_
    ↪MINIMUM);

    // Ensure size matches granularity requirements for the allocation
    size_t padded_size = ROUND_UP(size, granularity);

    mcProp.size = padded_size;

    // Create Multicast Object this has no devices and no physical memory
    ↪associated yet
    CUmemGenericAllocationHandle mcHandle;
    cuMulticastCreate(&mcHandle, &mcProp);

    return mcHandle;
}

```

4.16.4.2 Add Devices to Multicast Objects

Devices can be added to a multicast team with `cuMulticastAddDevice`:

```
cuMulticastAddDevice(&mcHandle, device);
```

This step needs to be completed on all processes controlling devices that participate in a multicast team before memory on any device is bound to the multicast object.

4.16.4.3 Bind Memory to Multicast Objects

After a multicast object has been created and all participating devices have been added to the multicast object it needs to be backed with physical memory allocated with `cuMemCreate` for each device:

```
cuMulticastBindMem(mcHandle, mcOffset, memHandle, memOffset, size, 0 /*flags*/
    ↪);
```

4.16.4.4 Use Multicast Mappings

To use multicast mappings in CUDA C++, it is necessary to use the [multimem PTX instructions](#) with inline PTX:

```

__global__ void all_reduce_norm_barrier_kernel(float* l2_norm,
                                              float* partial_l2_norm_mc,
                                              unsigned int* arrival_counter_
    ↪uc, unsigned int* arrival_counter_mc,
                                              const unsigned int expected_
    ↪count) {
    ↪assert( 1 == blockDim.x * blockDim.y * blockDim.z * gridDim.x * gridDim.y
    ↪* gridDim.z );
    float l2_norm_sum = 0.0;
    #if __CUDA_ARCH__ >= 900
        // atomic reduction to all replicas
        // this can be conceptually thought of as __threadfence_system();
    
```

(continues on next page)

(continued from previous page)

```

→atomicAdd_system(arrival_counter_mc, 1);
    cuda::ptx::multimem_red(cuda::ptx::release_t, cuda::ptx::scope_sys_t,
→cuda::ptx::op_add_t, arrival_counter_mc, n);

    // Need a fence between Multicast (mc) and Unicast (uc) access to the same
→memory `arrival_counter_uc` and `arrival_counter_mc`:
    // - fence.proxy instructions establish an ordering between memory
→accesses that may happen through different proxies
    // - Value .alias of the .proxykind qualifier refers to memory accesses
→performed using virtually aliased addresses to the same memory location.
    // from https://docs.nvidia.com/cuda/parallel-thread-execution/#parallel-
→synchronization-and-communication-instructions-membar
    cuda::ptx::fence_proxy_alias();

    // spin wait with acquire ordering on UC mapping till all peers have
→arrived in this iteration
    // Note: all ranks need to reach another barrier after this kernel, such
→that it is not possible for the barrier to be unblocked by an
    // arrival of a rank for the next iteration if some other rank is slow.
    cuda::atomic_ref<unsigned int, cuda::thread_scope_system> ac(arrival_
→counter_uc);
    while (expected_count > ac.load(cuda::memory_order_acquire));

    // Atomic load reduction from all replicas. It does not provide ordering
→so it can be relaxed.
    asm volatile ("multimem.ld_reduce.relaxed.sys.global.add.f32 %0, [%1];" :
→"=f"(l2_norm_sum) : "l"(partial_l2_norm_mc) : "memory");

#else
    #error "ERROR: multimem instructions require compute capability 9.0 or
→larger."
#endif

    *l2_norm = std::sqrt(l2_norm_sum);
}

```

4.16.5. Advanced Configuration

4.16.5.1 Memory Type

VMM also provides a mechanism for applications to allocate special types of memory that certain devices may support. With `cuMemCreate`, applications can specify memory type requirements using the `CUMemAllocationProp::allocFlags` to opt-in to specific memory features. Applications must ensure that the requested memory type is supported by the device.

4.16.5.2 Compressible Memory

Compressible memory can be used to accelerate accesses to data with unstructured sparsity and other compressible data patterns. Compression can save DRAM bandwidth, L2 read bandwidth, and L2 capacity depending on the data. Applications that want to allocate compressible memory on devices that support compute data compres-

sion can do so by setting `CUmemAllocationProp::allocFlags::compressionType` to `CU_MEM_ALLOCATION_COMP_GENERIC`. Users must query if device supports Compute Data Compression by using `CU_DEVICE_ATTRIBUTE_GENERIC_COMPRESSION_SUPPORTED`. The following code snippet illustrates querying compressible memory support using `cuDeviceGetAttribute`.

```
int compressionSupported = 0;
cuDeviceGetAttribute(&compressionSupported, CU_DEVICE_ATTRIBUTE_GENERIC_
    ↪ COMPRESSION_SUPPORTED, device);
```

On devices that support compute data compression, users must opt in at allocation time as shown below:

```
prop.allocFlags.compressionType = CU_MEM_ALLOCATION_COMP_GENERIC;
```

For a variety of reasons such as limited hardware resources, the allocation may not have compression attributes. To verify that the flags worked, users should query the properties of the allocated memory using `cuMemGetAllocationPropertiesFromHandle`.

```
CUmemAllocationProp allocationProp = {};
cuMemGetAllocationPropertiesFromHandle(&allocationProp, allocationHandle);

if (allocationProp.allocFlags.compressionType == CU_MEM_ALLOCATION_COMP_
    ↪ GENERIC)
{
    // Obtained compressible memory allocation
}
```

4.16.5.3 Virtual Aliasing Support

The virtual memory management APIs provide a way to create multiple virtual memory mappings or “proxies” to the same allocation using multiple calls to `cuMemMap` with different virtual addresses. This is called virtual aliasing. Unless otherwise noted in the PTX ISA, writes to one proxy of the allocation are considered inconsistent and incoherent with any other proxy of the same memory until the writing device operation (grid launch, `memcpy`, `memset`, and so on) completes. Grids present on the GPU prior to a writing device operation but reading after the writing device operation completes are also considered to have inconsistent and incoherent proxies.

For example, the following snippet is considered undefined, assuming device pointers A and B are virtual aliases of the same memory allocation:

```
__global__ void foo(char *A, char *B) {
    *A = 0x1;
    printf("%d\n", *B);    // Undefined behavior! *B can take on either
    // the previous value or some value in-between.
}
```

The following is defined behavior, assuming these two kernels are ordered monotonically (by streams or events).

```
__global__ void foo1(char *A) {
    *A = 0x1;
}

__global__ void foo2(char *B) {
```

(continues on next page)

(continued from previous page)

```

    printf("%d\n", *B);    // *B == *A == 0x1 assuming foo2 waits for foo1
    // to complete before launching
}

cudaMemcpyAsync(B, input, size, stream1);    // Aliases are allowed at
// operation boundaries
foo1<<<1,1,0,stream1>>>(A);                // allowing foo1 to access A.
cudaEventRecord(event, stream1);
cudaStreamWaitEvent(stream2, event);
foo2<<<1,1,0,stream2>>>(B);
cudaStreamWaitEvent(stream3, event);
cudaMemcpyAsync(output, B, size, stream3);  // Both launches of foo2 and
// cudaMemcpy (which both
// read) wait for foo1 (which
→writes)
// to complete before proceeding

```

If accessing the same allocation through different “proxies” is required in the same kernel, a fence . proxy.alias can be used between the two accesses. The above example can thus be made legal with inline PTX assembly:

```

__global__ void foo(char *A, char *B) {
    *A = 0x1;
    cuda::ptx::fence_proxy_alias();
    printf("%d\n", *B);    // *B == *A == 0x1
}

```

4.16.5.4 OS-Specific Handle Details for IPC

With `cuMemCreate`, users can indicate at allocation time that they have earmarked a particular allocation for inter-process communication or graphics interop purposes. Applications can do this by setting `CUmemAllocationProp::requestedHandleTypes` to a platform-specific field. On Windows, when `CUmemAllocationProp::requestedHandleTypes` is set to `CU_MEM_HANDLE_TYPE_WIN32` applications must also specify an `LPSECURITY_ATTRIBUTES` attribute in `CUmemAllocationProp::win32HandleMetaData`. This security attribute defines the scope of which exported allocations may be transferred to other processes.

Users must ensure they query for support of the requested handle type before attempting to export memory allocated with `cuMemCreate`. The following code snippet illustrates query for handle type support in a platform-specific way.

```

int deviceSupportsIpcHandle;
#ifdef __linux__
    cuDeviceGetAttribute(&deviceSupportsIpcHandle, CU_DEVICE_ATTRIBUTE_HANDLE_
→TYPE_POSIX_FILE_DESCRIPTOR_SUPPORTED, device));
#else
    cuDeviceGetAttribute(&deviceSupportsIpcHandle, CU_DEVICE_ATTRIBUTE_HANDLE_
→TYPE_WIN32_HANDLE_SUPPORTED, device));
#endif

```

Users should set the `CUmemAllocationProp::requestedHandleTypes` appropriately as shown below:

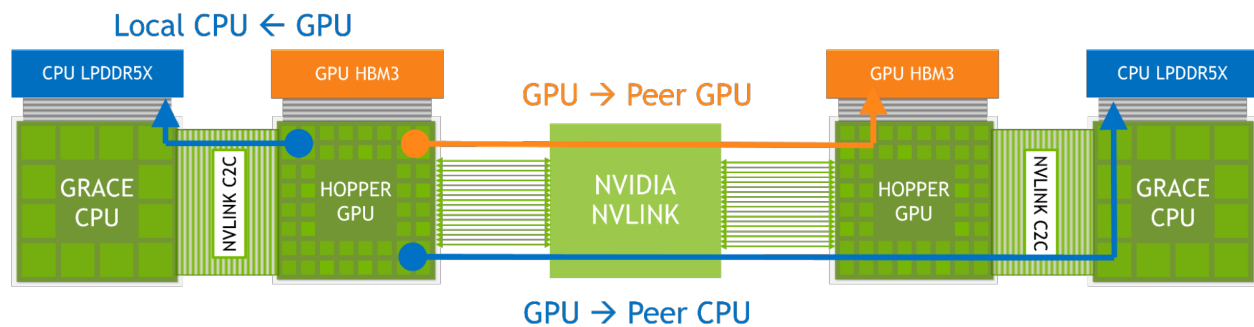
```

#if defined(__linux__)
    prop.requestedHandleTypes = CU_MEM_HANDLE_TYPE_POSIX_FILE_DESCRIPTOR;
#else
    prop.requestedHandleTypes = CU_MEM_HANDLE_TYPE_WIN32;
    prop.win32HandleMetaData = // Windows specific LPSECURITY_ATTRIBUTES
    ↪ attribute.
#endif

```

4.17. Extended GPU Memory

The Extended GPU Memory (EGM) feature, utilizing the high-bandwidth NVLink-C2C, facilitates efficient access to all system memory by GPUs, in both single-node and multi-node systems. EGM applies to integrated CPU-GPU NVIDIA systems by allowing physical memory allocation that can be accessed from any GPU thread within the setup. EGM ensures that all GPUs can access its resources at the speed of either GPU-GPU NVLink or NVLink-C2C.



In this setup, memory accesses occur via the local high-bandwidth NVLink-C2C. For remote memory accesses, GPU NVLink and, in some cases, NVLink-C2C are used. With EGM, GPU threads gain the capability to access all available memory resources, including CPU attached memory and HBM3, over the NVSwitch fabric.

4.17.1. Preliminaries

Before diving into API changes for EGM functionalities, we are going to cover currently supported topologies, identifier assignment, prerequisites for virtual memory management, and CUDA types for EGM.

4.17.1.1 EGM Platforms: System topology

Currently, EGM can be enabled in several platforms: **(1) Single-Node, Single-GPU**: Consists of an Arm-based CPU, CPU attached memory, and a GPU. Between the CPU and the GPU there is a high bandwidth C2C (Chip-to-Chip) interconnect. **(2) Single-Node, Multi-GPU**: Consists of ARM-based CPUs, each with attached memory, and multiple GPUs connected through an NVLink-based network. **(3) Multi-Node, Multi-GPU**: Two or more single-node systems, each as in (1) or (2) above, connected through an NVLink-based network.

Note

Using `cgroups` to limit available devices will block routing over EGM and cause performance issues. Use `CUDA_VISIBLE_DEVICES` instead.

4.17.1.2 Socket Identifiers: What are they? How to access them?

NUMA (Non-Uniform Memory Access) is a memory architecture used in multi-processor computer systems such that the memory is divided into multiple nodes. Each node has its own processors and memory. In such a system, NUMA divides the system into nodes and assigns a unique identifier (*numaID*) to every node.

EGM uses the NUMA node identifier which is assigned by the operating system. Note that, this identifier is different from the ordinal of a device and it is associated with the closest host node. In addition to the existing methods, the user can obtain the identifier of the host node (*numaID*) by calling `cuDeviceGetAttribute` with `CU_DEVICE_ATTRIBUTE_HOST_NUMA_ID` attribute type as follows:

```
int numaId;
cuDeviceGetAttribute(&numaId, CU_DEVICE_ATTRIBUTE_HOST_NUMA_ID,
    ↪ deviceOrdinal);
```

4.17.1.3 Allocators and EGM support

Mapping system memory as EGM does not cause any performance issues. In fact, accessing a remote socket's system memory mapped as EGM is going to be faster. Because, with EGM traffic is guaranteed to be routed over NVLinks. Currently, `cuMemCreate` and `cudaMemPoolCreate` allocators are supported with appropriate location type and NUMA identifiers.

4.17.1.4 Memory management extensions to current APIs

Currently, EGM memory can be mapped with Virtual Memory (`cuMemCreate`) or Stream Ordered Memory (`cudaMemPoolCreate`) allocators. The user is responsible for allocating physical memory and mapping it to a virtual memory address space on all sockets.

Note

Multi-node, multi-GPU platforms require interprocess communication. Therefore we encourage the reader to see [Chapter 4.15](#).

Note

We encourage readers to read CUDA Programming Guide's [Chapter 4.16](#) and [Chapter 4.3](#) for a better understanding.

New CUDA property types have been added to APIs for allowing those approaches to understand allocation locations using NUMA-like node identifiers:

CUDA Type	Used with
CU_MEM_LOCATION_TYPE_HOST_NUMA	CUmemAllocationProp for cuMemCreate
cudaMemLocationTypeHostNuma	cudaMemPoolProps for cudaMemPoolCreate

Note

Please see [CUDA Driver API](#) and [CUDA Runtime Data Types](#) to find more about NUMA specific CUDA types.

4.17.2. Using the EGM Interface

4.17.2.1 Single-Node, Single-GPU

Any of the existing CUDA host allocators as well as system allocated memory can be used to benefit from high-bandwidth C2C. To the user, local access is what a host allocation is today.

Note

Refer to the tuning guide for more information about memory allocators and page sizes.

4.17.2.2 Single-Node, Multi-GPU

In a multi-GPU system, the user has to provide host information for the placement. As we mentioned, a natural way to express that information would be by using NUMA node IDs and EGM follows this approach. Therefore, using the `cuDeviceGetAttribute` function the user should be able to learn the closest NUMA node id. (See [Socket Identifiers: What are they? How to access them?](#)). Then the user can allocate and manage EGM memory using VMM (Virtual Memory Management) API or CUDA Memory Pool.

4.17.2.2.1 Using VMM APIs

The first step in memory allocation using Virtual Memory Management APIs is to create a physical memory chunk that will provide a backing for the allocation. See CUDA Programming Guide's *Virtual Memory Management* section for more details. In EGM allocations the user has to explicitly provide `CU_MEM_LOCATION_TYPE_HOST_NUMA` as the location type and `numaId` as the location identifier. Also in EGM, allocations must be aligned to appropriate granularity of the platform. The following code snippet shows allocating physical memory with `cuMemCreate`:

```
CUmemAllocationProp prop{};
prop.type = CU_MEM_ALLOCATION_TYPE_PINNED;
prop.location.type = CU_MEM_LOCATION_TYPE_HOST_NUMA;
prop.location.id = numaId;
size_t granularity = 0;
cuMemGetAllocationGranularity(&granularity, &prop, MEM_ALLOC_GRANULARITY_
    ↪ MINIMUM);
size_t padded_size = ROUND_UP(size, granularity);
```

(continues on next page)

(continued from previous page)

```
CUmemGenericAllocationHandle allocHandle;
cuMemCreate(&allocHandle, padded_size, &prop, 0);
```

After physical memory allocation, we have to reserve an address space and map it to a pointer. These procedures do not have EGM-specific changes:

```
CUdeviceptr dptr;
cuMemAddressReserve(&dptr, padded_size, 0, 0, 0);
cuMemMap(dptr, padded_size, 0, allocHandle, 0);
```

Finally, the user has to explicitly protect mapped virtual address ranges. Otherwise access to the mapped space would result in a crash. Similar to the memory allocation, the user has to provide `CU_MEM_LOCATION_TYPE_HOST_NUMA` as the location type and *numaId* as the location identifier. Following code snippet create an access descriptors for the host node and the GPU to give read and write access for the mapped memory to both of them:

```
CUmemAccessDesc accessDesc[2]{{}};
accessDesc[0].location.type = CU_MEM_LOCATION_TYPE_HOST_NUMA;
accessDesc[0].location.id = numaId;
accessDesc[0].flags = CU_MEM_ACCESS_FLAGS_PROT_READWRITE;
accessDesc[1].location.type = CU_MEM_LOCATION_TYPE_DEVICE;
accessDesc[1].location.id = currentDev;
accessDesc[1].flags = CU_MEM_ACCESS_FLAGS_PROT_READWRITE;
cuMemSetAccess(dptr, size, accessDesc, 2);
```

4.17.2.2.2 Using CUDA Memory Pool

To define EGM, the user can create a memory pool on a node and give access to peers. In this case, the user has to explicitly define `cudaMemLocationTypeHostNuma` as the location type and *numaId* as the location identifier. The following code snippet shows creating a memory pool `cudaMemPoolCreate`:

```
cudaSetDevice(homeDevice);
cudaMemPoolProps props{};
props.allocType = cudaMemAllocationTypePinned;
props.location.type = cudaMemLocationTypeHostNuma;
props.location.id = numaId;
cudaMemPoolCreate(&memPool, &props);
```

Additionally, for direct connect peer access, it is also possible to use the existing peer access API, `cudaMemPoolSetAccess`. An example for an *accessingDevice* is shown in the following code snippet:

```
cudaMemAccessDesc desc{};
desc.flags = cudaMemAccessFlagsProtReadWrite;
desc.location.type = cudaMemLocationTypeDevice;
desc.location.id = accessingDevice;
cudaMemPoolSetAccess(memPool, &desc, 1);
```

When the memory pool is created, and accesses are given, the user can set created memory pool to the *residentDevice* and start allocating memory using `cudaMallocAsync`:

```
cudaDeviceSetMemPool(residentDevice, memPool);
cudaMallocAsync(&ptr, size, memPool, stream);
```

Note

EGM is mapped with 2MB pages. Therefore, users may encounter more TLB misses when accessing very large allocations.

4.17.2.3 Multi-Node, Multi-GPU

Beyond memory allocation, remote peer access does not have EGM-specific modification and it follows CUDA inter process (IPC) protocol. See [CUDA Programming Guide](#) for more details in IPC.

The user should allocate memory using `cuMemCreate` and again the user has to explicitly provide `CU_MEM_LOCATION_TYPE_HOST_NUMA` as the location type and `numaId` as the location identifier. In addition `CU_MEM_HANDLE_TYPE_FABRIC` should be defined as the requested handle type. The following code snippet shows allocating physical memory on Node A:

```
CUmemAllocationProp prop{};
prop.type = CU_MEM_ALLOCATION_TYPE_PINNED;
prop.requestedHandleTypes = CU_MEM_HANDLE_TYPE_FABRIC;
prop.location.type = CU_MEM_LOCATION_TYPE_HOST_NUMA;
prop.location.id = numaId;
size_t granularity = 0;
cuMemGetAllocationGranularity(&granularity, &prop,
                             MEM_ALLOC_GRANULARITY_MINIMUM);
size_t padded_size = ROUND_UP(size, granularity);
size_t page_size = ...;
assert(padded_size % page_size == 0);
CUmemGenericAllocationHandle allocHandle;
cuMemCreate(&allocHandle, padded_size, &prop, 0);
```

After creating allocation handle using `cuMemCreate` the user can export that handle to the other node, Node B, calling `cuMemExportToShareableHandle`:

```
cuMemExportToShareableHandle(&fabricHandle, allocHandle,
                             CU_MEM_HANDLE_TYPE_FABRIC, 0);
// At this point, fabricHandle should be sent to Node B via TCP/IP.
```

On Node B, the handle can be imported using `cuMemImportFromShareableHandle` and treated as any other fabric handle

```
// At this point, fabricHandle should be received from Node A via TCP/IP.
CUmemGenericAllocationHandle allocHandle;
cuMemImportFromShareableHandle(&allocHandle, &fabricHandle,
                               CU_MEM_HANDLE_TYPE_FABRIC);
```

When handle is imported at Node B, then the user can reserve an address space and map it locally in a regular fashion:

```
size_t granularity = 0;
cuMemGetAllocationGranularity(&granularity, &prop,
                             MEM_ALLOC_GRANULARITY_MINIMUM);
size_t padded_size = ROUND_UP(size, granularity);
size_t page_size = ...;
assert(padded_size % page_size == 0);
CUdeviceptr dptr;
```

(continues on next page)

(continued from previous page)

```
cuMemAddressReserve(&dpPtr, padded_size, 0, 0, 0);
cuMemMap(dpPtr, padded_size, 0, allocHandle, 0);
```

As the final step, the user should give appropriate accesses to each of the local GPUs at Node B. An example code snippet that gives read and write access to eight local GPUs:

```
// Give all 8 local GPUS access to exported EGM memory located on Node A.
CUmemAccessDesc accessDesc[8];
for (int i = 0; i < 8; i++) {
    accessDesc[i].location.type = CU_MEM_LOCATION_TYPE_DEVICE;
    accessDesc[i].location.id = i;
    accessDesc[i].flags = CU_MEM_ACCESS_FLAGS_PROT_READWRITE;
}
cuMemSetAccess(dpPtr, size, accessDesc, 8);
```

4.18. CUDA Dynamic Parallelism

4.18.1. Introduction

4.18.1.1 Overview

CUDA dynamic parallelism - often abbreviated CDP - is a feature of the CUDA programming model that enables code running on the GPU to create new GPU work. That is, new work can be added to the GPU in the form of additional kernel launches from device code already running on the GPU. This feature can reduce the need to transfer execution control and data between host and device, as launch configuration decisions can be made at runtime by threads executing on the device.

Data-dependent parallel work can be generated by a kernel at run-time. Before CDP was added to CUDA, some algorithms and programming patterns required modifications to eliminate recursion, irregular loop structure, or other constructs that do not fit a flat, single-level of parallelism. These program structures may be expressed more naturally using CUDA dynamic parallelism.

Note

This section documents the updated version of CUDA Dynamic Parallelism, sometimes called CDP2, which is the default in CUDA 12.0 and later. CDP2 is the only version of CUDA dynamic parallelism available on devices of CC 9.0 and higher. Developers can still opt in to the legacy CUDA Dynamic Parallelism, CDP1, for devices of CC lower than 9.0 with the compiler argument `-DCUDA_FORCE_CDP1_IF_SUPPORTED`. CDP1 documentation can be found in [Legacy versions of the CUDA Programming Guide](#). CDP1 is slated for removal from a future version of CUDA.

4.18.2. Execution Environment

Dynamic parallelism in CUDA allows GPU threads to configure, launch, and implicitly synchronize new grids. A grid is an instance of a kernel launch, including the specific shape of the thread blocks and the grid of thread blocks. The distinction between a kernel function itself and the specific invocation of that kernel, i.e. a grid, is important to note in the following sections.

4.18.2.1 Parent and Child Grids

A device thread that configures and launches a new grid belongs to the parent grid. The new grid that is created by the invocation is called a child grid.

The invocation and completion of child grids is properly nested, meaning that the parent grid is not considered complete until all child grids created by its threads have completed, and the runtime guarantees an implicit synchronization between the parent and child.

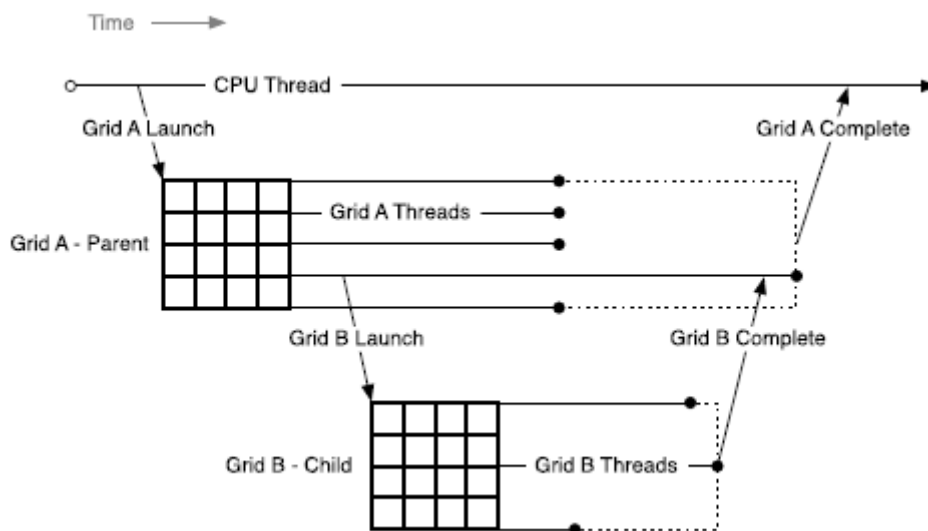


Figure 57: Parent-Child Launch Nesting

4.18.2.2 Scope of CUDA Primitives

CUDA Dynamic Parallelism relies on the *CUDA Device Runtime*, which enables calling a limited set of APIs which are syntactically similar to the CUDA Runtime API, but available in device code. The behavior of the device runtime APIs are similar to their host counterparts, but there are some differences. These differences are captured in the table in section *API Reference*.

On both host and device, the CUDA runtime offers an API for launching kernels and for tracking dependencies between launches via streams and events. On the device, launched kernels and CUDA objects are visible to all threads in the invoking grid. This means, for example, that a stream may be created by one thread and used by any other thread in the same grid. However, CUDA objects such as streams and events which were created by device API calls are only valid within the grid where they were created.

4.18.2.3 Streams and Events

CUDA *Streams* and *Events* allow control over dependencies between kernel launches: kernels launched into the same stream execute in-order, and events may be used to create dependencies between streams. Streams and events created on the device serve this exact same purpose.

Streams and events created within a grid exist within grid scope, but have undefined behavior when used outside of the grid where they were created. As described above, all work launched by a grid is implicitly synchronized when the grid exits; work launched into streams is included in this, with all dependencies resolved appropriately. The behavior of operations on a stream that has been modified outside of grid scope is undefined.

Streams and events created on the host have undefined behavior when used within any kernel, just as streams and events created by a parent grid have undefined behavior if used within a child grid.

4.18.2.4 Ordering and Concurrency

The ordering of kernel launches from the device runtime follows CUDA Stream ordering semantics. Within a grid, all kernel launches into the same stream (with the exception of *The Fire-and-Forget Stream*) are executed in-order. With multiple threads in the same grid launching into the same stream, the ordering within the stream is dependent on the thread scheduling within the grid, which may be controlled with synchronization primitives such as `__syncthreads()`.

Note that while named streams are shared by all threads within a grid, the implicit *NULL* stream is only shared by all threads within a thread block. If multiple threads in a thread block launch into the implicit stream, then these launches will be executed in-order. If threads in different thread blocks launch into the implicit stream, these launches may be executed concurrently. If concurrency is desired for launches from multiple threads within a thread block, explicit named streams should be used.

The device runtime introduces no new concurrency guarantees within the CUDA execution model. That is, there is no guarantee of concurrent execution between any number of different thread blocks on a device.

The lack of concurrency guarantee extends to a parent grid and their child grids. When a parent grid launches a child grid, the child may start to execute once stream dependencies are satisfied and hardware resources are available, but is not guaranteed to begin execution until the parent grid reaches an implicit synchronization point.

Concurrency may vary as a function of device configuration, application workload, and runtime scheduling. It is therefore unsafe to depend on any concurrency between different thread blocks.

4.18.3. Memory Coherence and Consistency

Parent and child grids share the same global and constant memory storage, but have distinct local and shared memory. The following table shows which memory spaces allow parent and child to access via the same pointers. Child grids can never access the local or shared memory of parent grids, nor can parent grids access local or shared memory of child grids.

Table 26: Dynamic Parallelism: Memory Scope Accessibility Between Parent and Child Grids

Memory Space	Parent/Child use same pointers?
Global Memory	Yes
Mapped Memory	Yes
Local Memory	No
Shared Memory	No
Texture Memory	Yes (read-only)

4.18.3.1 Global Memory

Parent and child grids have coherent access to global memory, with weak consistency guarantees between child and parent. There is only one point of time in the execution of a child grid when its view of memory is fully consistent with the parent thread: at the point when the child grid is invoked by the parent.

All global memory operations in the parent thread prior to the child grid's invocation are visible to the child grid. With the removal of `cudaDeviceSynchronize()`, it is no longer possible to access the modifications made by the threads in the child grid from the parent grid. The only way to access the modifications made by the threads in the child grid before the parent grid exits is via a kernel launched into the `cudaStreamTailLaunch` stream.

In the following example, the child grid executing `child_launch` is only guaranteed to see the modifications to data made before the child grid was launched. Since thread 0 of the parent is performing the launch, the child will be consistent with the memory seen by thread 0 of the parent. Due to the first `__syncthreads()` call, the child will see `data[0]=0`, `data[1]=1`, ..., `data[255]=255` (without the `__syncthreads()` call, only `data[0]=0` would be guaranteed to be seen by the child). The child grid is only guaranteed to return at an implicit synchronization. This means that the modifications made by the threads in the child grid are never guaranteed to become available to the parent grid. To access modifications made by `child_launch`, a `tail_launch` kernel is launched into the `cudaStreamTailLaunch` stream.

```
__global__ void tail_launch(int *data) {
    data[threadIdx.x] = data[threadIdx.x]+1;
}

__global__ void child_launch(int *data) {
    data[threadIdx.x] = data[threadIdx.x]+1;
}

__global__ void parent_launch(int *data) {
    data[threadIdx.x] = threadIdx.x;

    __syncthreads();

    if (threadIdx.x == 0) {
        child_launch<<< 1, 256 >>>(data);
        tail_launch<<< 1, 256, 0, cudaStreamTailLaunch >>>(data);
    }
}

void host_launch(int *data) {
    parent_launch<<< 1, 256 >>>(data);
}
```

4.18.3.2 Mapped Memory

Mapped system memory has identical coherence and consistency guarantees to global memory, and follows the semantics detailed above. A kernel may not allocate or free mapped memory, but may use pointers to mapped memory passed in from the host program.

4.18.3.3 Shared and Local Memory

Shared and Local memory is private to a thread block or thread, respectively, and is not visible or coherent between parent and child. Behavior is undefined when an object in one of these locations is referenced outside of the scope within which it belongs, and may cause an error.

The NVIDIA compiler will attempt to warn if it can detect that a pointer to local or shared memory is being passed as an argument to a kernel launch. At runtime, the programmer may use the `__is-Global()` intrinsic to determine whether a pointer references global memory and so may safely be passed to a child launch.

Calls to `cudaMemcpy*Async()` or `cudaMemset*Async()` may invoke new child kernels on the device in order to preserve stream semantics. As such, passing shared or local memory pointers to these APIs is illegal and will return an error.

4.18.3.4 Local Memory

Local memory is private storage for an executing thread, and is not visible outside of that thread. It is illegal to pass a pointer to local memory as a launch argument when launching a child kernel. The result of dereferencing such a local memory pointer from a child grid is undefined.

For example the following is illegal, with undefined behavior if `x_array` is accessed by `child_launch`:

```
int x_array[10];           // Creates x_array in parent's local memory
child_launch<<< 1, 1 >>>(x_array);
```

It is sometimes difficult for a programmer to be aware of when a variable is placed into local memory by the compiler. As a general rule, all storage passed to a child kernel should be allocated explicitly from the global-memory heap, either with `cudaMalloc()`, `new()` or by declaring `__device__` storage at global scope. For example:

```
// Correct - "value" is global storage
__device__ int value;
__device__ void x() {
    value = 5;
    child<<< 1, 1 >>>(&value);
}
```

```
// Invalid - "value" is local storage
__device__ void y() {
    int value = 5;
    child<<< 1, 1 >>>(&value);
}
```

4.18.3.4.1 Texture Memory

Writes to the global memory region over which a texture is mapped are incoherent with respect to texture accesses. Coherence for texture memory is enforced at the invocation of a child grid and when a child grid completes. This means that writes to memory prior to a child kernel launch are reflected in texture memory accesses of the child. Similarly to Global Memory above, writes to memory by a child are never guaranteed to be reflected in the texture memory accesses by a parent. The only way to access the modifications made by the threads in the child grid before the parent grid exits is via a kernel launched into the `cudaStreamTailLaunch` stream. Concurrent accesses by parent and child may result in inconsistent data.

4.18.4. Programming Interface

4.18.4.1 Basics

The following example shows a simple *Hello World* program incorporating dynamic parallelism:

```
#include <stdio.h>

__global__ void childKernel()
{
    printf("Hello ");
}

__global__ void tailKernel()
{
    printf("World!\n");
}

__global__ void parentKernel()
{
    // launch child
    childKernel<<<1,1>>>();
    if (cudaSuccess != cudaGetLastError()) {
        return;
    }

    // launch tail into cudaStreamTailLaunch stream
    // implicitly synchronizes: waits for child to complete
    tailKernel<<<1,1,0,cudaStreamTailLaunch>>>();
}

int main(int argc, char *argv[])
{
    // launch parent
    parentKernel<<<1,1>>>();
    if (cudaSuccess != cudaGetLastError()) {
        return 1;
    }

    // wait for parent to complete
    if (cudaSuccess != cudaDeviceSynchronize()) {
        return 2;
    }

    return 0;
}
```

This program may be built in a single step from the command line as follows:

```
$ nvcc -arch=sm_75 -rdc=true hello_world.cu -o hello -lcudadevrt
```

4.18.4.2 C++ Language Interface for CDP

The language interface and API available to CUDA kernels using CUDA C++ for Dynamic Parallelism is called the *CUDA Device Runtime*.

Where possible the syntax and semantics of the CUDA Runtime API have been retained in order to facilitate ease of code reuse for routines that may run in either the host or device environments.

As with all code in CUDA C++, the APIs and code outlined here is per-thread code. This enables each thread to make unique, dynamic decisions regarding what kernel or operation to execute next. There are no synchronization requirements between threads within a block to execute any of the provided device runtime APIs, which enables the device runtime API functions to be called in arbitrarily divergent kernel code without deadlock.

4.18.4.2.1 Device-Side Kernel Launch

Kernels may be launched from the device using the standard CUDA <<< >>> syntax:

```
kernel_name<<< Dg, Db, Ns, S >>>([kernel arguments]);
```

- ▶ Dg is of type `dim3` and specifies the dimensions and size of the grid
- ▶ Db is of type `dim3` and specifies the dimensions and size of each thread block
- ▶ Ns is of type `size_t` and specifies the number of bytes of shared memory that is dynamically allocated per thread block for this call in addition to statically allocated memory. Ns is an optional argument that defaults to 0.
- ▶ S is of type `cudaStream_t` and specifies the stream associated with this call. The stream must have been allocated in the same grid where the call is being made. S is an optional argument that defaults to the NULL stream.

4.18.4.2.1.1 Launches are Asynchronous

Identical to host-side launches, all device-side kernel launches are asynchronous with respect to the launching thread. That is to say, the <<<>>> launch command will return immediately and the launching thread will continue to execute until it hits an implicit launch-synchronization point, such as at a kernel launched into the `cudaStreamTailLaunch` stream (*The Tail Launch Stream*). The child grid may begin execution at any time after launch, but is not guaranteed to begin execution until the launching thread reaches an implicit launch-synchronization point.

Similar to host-side launch, work launched into separate streams may run concurrently, but actual concurrency is not guaranteed. Programs that depend upon concurrency between child kernels are not supported by the CUDA programming model and will have undefined behavior.

4.18.4.2.1.2 Launch Environment Configuration

All global device configuration settings (for example, shared memory and L1 cache size as returned from `cudaDeviceGetCacheConfig()`, and device limits returned from `cudaDeviceGetLimit()`) will be inherited from the parent. Likewise, device limits such as stack size will remain as-configured.

For host-launched kernels, per-kernel configurations set from the host will take precedence over the global setting. These configurations will be used when the kernel is launched from the device as well. It is not possible to reconfigure a kernel's environment from the device.

4.18.4.2.2 Events

Only the inter-stream synchronization capabilities of CUDA events are supported. This means that `cudaStreamWaitEvent()` is supported, but `cudaEventSynchronize()`, `cudaEventElapsedTime()`, and `cudaEventQuery()` are not. As `cudaEventElapsedTime()` is not supported, `cudaEvents` must be created via `cudaEventCreateWithFlags()`, passing the `cudaEventDisableTiming` flag.

As with streams, event objects may be shared between all threads within the grid which created them but are local to that grid and may not be passed to other kernels. Event handles are not guaranteed to be unique between grids, so using an event handle within a grid that did not create it will result in undefined behavior.

4.18.4.2.3 Synchronization

It is up to the program to perform sufficient inter-thread synchronization, for example via a CUDA Event, if the calling thread is intended to synchronize with child grids invoked from other threads.

As it is not possible to explicitly synchronize child work from a parent thread, there is no way to guarantee that changes occurring in child grids are visible to threads within the parent grid.

4.18.4.2.4 Device Management

Only the device on which a kernel is running will be controllable from that kernel. This means that device APIs such as `cudaSetDevice()` are not supported by the device runtime. The active device as seen from the GPU (returned from `cudaGetDevice()`) will have the same device number as seen from the host system. The `cudaDeviceGetAttribute()` call may request information about another device as this API allows specification of a device ID as a parameter of the call. Note that the catch-all `cudaGetDeviceProperties()` API is not offered by the device runtime - properties must be queried individually.

4.18.5. Programming Guidelines

4.18.5.1 Performance

4.18.5.1.1 Dynamic-parallelism-enabled Kernel Overhead

System software which is active when controlling dynamic launches may impose an overhead on any kernel which is running at the time, whether or not it invokes kernel launches of its own. This overhead arises from the device runtime's execution tracking and management software and may result in decreased performance. This overhead is, in general, incurred for applications that link against the device runtime library.

4.18.5.2 Implementation Restrictions and Limitations

Dynamic Parallelism guarantees all semantics described in this document, however, certain hardware and software resources are implementation-dependent and limit the scale, performance and other properties of a program which uses the device runtime.

4.18.5.2.1 Runtime

4.18.5.2.1.1 Memory Footprint

The device runtime system software reserves memory for various management purposes, in particular a reservation for tracking pending grid launches. Configuration controls are available to reduce the

size of this reservation in exchange for certain launch limitations. See [Configuration Options](#), below, for details.

4.18.5.2.1.2 Pending Kernel Launches

When a kernel is launched, all associated configuration and parameter data is tracked until the kernel completes. This data is stored within a system-managed launch pool.

The size of the fixed-size launch pool is configurable by calling `cudaDeviceSetLimit()` from the host and specifying `cudaLimitDevRuntimePendingLaunchCount`.

4.18.5.3 Compatibility and Interoperability

CDP2 is the default. Functions can be compiled with `-DCUDA_FORCE_CDP1_IF_SUPPORTED` to opt-out of using CDP2 on devices of compute capability less than 9.0.

	Function compiled with CUDA 12.0 and newer (default)	Function compiled with pre-CUDA 12.0 or with CUDA 12.0 and newer with <code>-DCUDA_FORCE_CDP1_IF_SUPPORTED</code> specified
Compilation	Compile error if device code references <code>cudaDeviceSynchronize</code> .	Compile error if code references <code>cudaStreamTailLaunch</code> or <code>cudaStreamFireAndForget</code> . Compile error if device code references <code>cudaDeviceSynchronize</code> and code is compiled for sm_90 or newer.
Compute capability < 9.0	New interface is used.	Legacy interface is used.
Compute capability 9.0 and higher	New interface is used.	New interface is used. If function references <code>cudaDeviceSynchronize</code> in device code, function load returns <code>cudaErrorSymbolNotFound</code> (this could happen if the code is compiled for devices of compute capability less than 9.0, but run on devices of compute capability 9.0 or higher using JIT).

Functions using CDP1 and CDP2 may be loaded and run simultaneously in the same context. The CDP1 functions are able to use CDP1-specific features (e.g. `cudaDeviceSynchronize`) and CDP2 functions are able to use CDP2-specific features (e.g. tail launch and fire-and-forget launch).

A function using CDP1 cannot launch a function using CDP2, and vice versa. If a function that would use CDP1 contains in its call graph a function that would use CDP2, or vice versa, `cudaErrorCdpVersionMismatch` would result during function load.

The behavior of legacy CDP1 is not included in this document. For information on CDP1, refer back to a [legacy version of the CUDA Programming Guide](#)

4.18.6. Device-side Launch from PTX

The previous sections have discussed using the [CUDA Device Runtime](#) to achieve dynamic parallelism. Dynamic parallelism can also be performed from PTX. For the programming language and compiler implementers who target *Parallel Thread Execution* (PTX) and plan to support *Dynamic Parallelism* in

their language, this section provides the low-level details related to supporting kernel launches at the PTX level.

4.18.6.1 Kernel Launch APIs

Device-side kernel launches can be implemented using the following two APIs accessible from PTX: `cudaLaunchDevice()` and `cudaGetParameterBuffer()`. `cudaLaunchDevice()` launches the specified kernel with the parameter buffer that is obtained by calling `cudaGetParameterBuffer()` and filled with the parameters to the launched kernel. The parameter buffer can be NULL, i.e., no need to invoke `cudaGetParameterBuffer()`, if the launched kernel does not take any parameters.

4.18.6.1.1 `cudaLaunchDevice`

At the PTX level, `cudaLaunchDevice()` needs to be declared in one of the two forms shown below before it is used.

```
// PTX-level Declaration of cudaLaunchDevice() when .address_size is 64
.extern .func(.param .b32 func_retval0) cudaLaunchDevice
(
    .param .b64 func,
    .param .b64 parameterBuffer,
    .param .align 4 .b8 gridDimension[12],
    .param .align 4 .b8 blockDimension[12],
    .param .b32 sharedMemSize,
    .param .b64 stream
)
```

The CUDA-level declaration below is mapped to one of the aforementioned PTX-level declarations and is found in the system header file `cuda_device_runtime_api.h`. The function is defined in the `cudadevrt` system library, which must be linked with a program in order to use device-side kernel launch functionality.

```
// CUDA-level declaration of cudaLaunchDevice()
extern "C" __device__
cudaError_t cudaLaunchDevice(void *func, void *parameterBuffer,
                             dim3 gridDimension, dim3 blockDimension,
                             unsigned int sharedMemSize,
                             cudaStream_t stream);
```

The first parameter is a pointer to the kernel to be launched, and the second parameter is the parameter buffer that holds the actual parameters to the launched kernel. The layout of the parameter buffer is explained in [Parameter Buffer Layout](#), below. Other parameters specify the launch configuration, i.e., as grid dimension, block dimension, shared memory size, and the stream associated with the launch (please refer to [Kernel Configuration](#) for the detailed description of launch configuration).

4.18.6.1.2 `cudaGetParameterBuffer`

`cudaGetParameterBuffer()` needs to be declared at the PTX level before it's used. The PTX-level declaration must be in one of the two forms given below, depending on address size:

```
// PTX-level Declaration of cudaGetParameterBuffer() when .address_size is 64
.extern .func(.param .b64 func_retval0) cudaGetParameterBuffer
(
```

(continues on next page)

(continued from previous page)

```
.param .b64 alignment,
.param .b64 size
)
;
```

The following CUDA-level declaration of `cudaGetParameterBuffer()` is mapped to the aforementioned PTX-level declaration:

```
// CUDA-level Declaration of cudaGetParameterBuffer()
extern "C" __device__
void *cudaGetParameterBuffer(size_t alignment, size_t size);
```

The first parameter specifies the alignment requirement of the parameter buffer and the second parameter the size requirement in bytes. In the current implementation, the parameter buffer returned by `cudaGetParameterBuffer()` is always guaranteed to be 64-byte aligned, and the alignment requirement parameter is ignored. However, it is recommended to pass the correct alignment requirement value - which is the largest alignment of any parameter to be placed in the parameter buffer - to `cudaGetParameterBuffer()` to ensure portability in the future.

4.18.6.2 Parameter Buffer Layout

Parameter reordering in the parameter buffer is prohibited, and each individual parameter placed in the parameter buffer is required to be aligned. That is, each parameter must be placed at the n^{th} byte in the parameter buffer, where n is the smallest multiple of the parameter size that is greater than the offset of the last byte taken by the preceding parameter. The maximum size of the parameter buffer is 4KB.

For a more detailed description of PTX code generated by the CUDA compiler, please refer to the PTX-3.5 specification.

4.19. CUDA Interoperability with APIs

Directly accessing GPU data from APIs in CUDA allows to read and write the data with CUDA kernels and thereby offering CUDA features while consuming them from other APIs. There are two main concepts: the direct approach, *Graphics Interoperability* with OpenGL and Direct3D[9-11] which enables to map the resources from the OpenGL and the Direct3D to the CUDA address space; and the more flexible *External resource interoperability*, where memory and synchronization objects can be accessed by importing and exporting through OS-level handles. This is supported for the following APIs, Direct3D[11-12], Vulkan and the NVIDIA Software Communication Interface Interoperability.

4.19.1. Graphics Interoperability

Before accessing a Direct3D or an OpenGL resource, for example a VBO (vertex buffer object) with CUDA, it must be registered and mapped. The registering with the according CUDA functions, see examples below, returns a CUDA graphics resource of type `struct cudaGraphicsResource`, which holds a CUDA device pointer or array. To access the device data in a kernel, the resource must be mapped. While the resource is registered, it can be mapped and unmapped as many times as necessary. A mapped resource is accessed by kernels using the device memory address returned by `cudaGraphicsResourceGetMappedPointer()` for buffers and `cudaGraphicsSubResourceGetMappedArray()` for CUDA arrays. Once the resource is no longer needed by CUDA, it can be unregistered.

These are the main steps:

1. Register the graphics buffer with CUDA
2. Map the resource
3. Access the device pointer or array of the mapped resource
4. Use device pointer or array in a CUDA kernel
5. Unmap the resource
6. Unregister the resource

Note that, registering a resource is costly and therefore ideally only called once per resource, however for each CUDA context which intends to use the resource, it is required to register the resource separately. `cudaGraphicsResourceSetMapFlags()` can be called to specify usage hints (write-only, read-only) that the CUDA driver can use to optimize resource management. Further note, that when accessing a resource through OpenGL, Direct3D, or a different CUDA context while it is mapped, it produces undefined results.

4.19.1.1 OpenGL Interoperability

The OpenGL resources that can be mapped into the address space of CUDA are OpenGL buffer, texture, and renderbuffer objects. A buffer object is registered using `cudaGraphicsGLRegisterBuffer()`, in CUDA, it appears as a normal device pointer. A texture or renderbuffer object is registered using `cudaGraphicsGLRegisterImage()`, in CUDA, it appears as a CUDA array.

If a texture or render buffer object has been registered with the `cudaGraphicsRegisterFlagsSurfaceLoadStore` flag, it can be written to. `cudaGraphicsGLRegisterImage()` supports all texture formats with 1, 2, or 4 components and an internal type of float (for example, `GL_RGBA_FLOAT32`), normalized integer (for example, `GL_RGBA8`, `GL_INTENSITY16`), and unnormalized integer (for example, `GL_RGBA8UI`).

Example:simpleGL interoperability

The following code sample uses a kernel to dynamically modify a 2D width x height grid of vertices stored in a vertex buffer object (VBO), and goes through the following main steps:

1. Register the VBO with CUDA
2. Loop: Map the VBO for writing from CUDA
3. Loop: Run CUDA kernel to modify the vertex positions
4. Loop: Unmap the VBO
5. Loop: Render the results using OpenGL
6. Unregister and delete VBO

The full example, `simpleGL`, of this section can be found here, https://github.com/NVIDIA/cuda-samples/tree/master/Samples/5_Domain_Specific/simpleGL.

```
__global__ void simple_vbo_kernel(float4 *pos, unsigned int width, unsigned
↪int height, float time)
{
    unsigned int x = blockIdx.x * blockDim.x + threadIdx.x;
    unsigned int y = blockIdx.y * blockDim.y + threadIdx.y;

    // calculate uv coordinates
    float u = x / (float)width;
```

(continues on next page)

(continued from previous page)

```

float v = y / (float)height;
u = u * 2.0f - 1.0f;
v = v * 2.0f - 1.0f;

// calculate simple sine wave pattern
float freq = 4.0f;
float w = sinf(u * freq + time) * cosf(v * freq + time) * 0.5f;

// write output vertex
pos[y * width + x] = make_float4(u, w, v, 1.0f);
}

int main(int argc, char **argv)
{
    char *ref_file = NULL;

    pArgc = &argc;
    pArgv = argv;

#ifdef __linux__
    setenv("DISPLAY", ":0", 0);
#endif

    printf("%s starting...\n", sSDKsample);

    if (argc > 1) {
        if (checkCmdLineFlag(argc, (const char **)argv, "file")) {
            // In this mode, we are running non-OpenGL and doing a compare of
            → the VBO was generated correctly
            getCmdLineArgumentString(argc, (const char **)argv, "file", (char
            → **) &ref_file);
        }
    }

    printf("\n");

    // First initialize OpenGL context
    if (false == initGL(&argc, argv)) {
        return false;
    }

    // register callbacks
    glutDisplayFunc(display);
    glutKeyboardFunc(keyboard);
    glutMouseFunc(mouse);
    glutMotionFunc(motion);
    glutCloseFunc(cleanup);

    // Create an empty vertex buffer object (VBO)
    // 1. Register the VBO with CUDA
    createVBO(&vbo, &cuda_vbo_resource, cudaGraphicsMapFlagsWriteDiscard);

```

(continues on next page)

(continued from previous page)

```

    // start rendering mainloop
    // 5. Render the results using OpenGL
    glutMainLoop();

    printf("%s completed, returned %s\n", sSDKsample, (g_TotalErrors == 0) ?
    ↪ "OK" : "ERROR!");
    exit(g_TotalErrors == 0 ? EXIT_SUCCESS : EXIT_FAILURE);
}

void createVBO(GLuint *vbo, struct cudaGraphicsResource **vbo_res, unsigned
    ↪ int vbo_res_flags)
{
    assert(vbo);

    // create buffer object
    glGenBuffers(1, vbo);
    glBindBuffer(GL_ARRAY_BUFFER, *vbo);

    // initialize buffer object
    unsigned int size = mesh_width * mesh_height * 4 * sizeof(float);
    glBufferData(GL_ARRAY_BUFFER, size, 0, GL_DYNAMIC_DRAW);

    glBindBuffer(GL_ARRAY_BUFFER, 0);

    // register this buffer object with CUDA
    checkCudaErrors(cudaGraphicsGLRegisterBuffer(vbo_res, *vbo, vbo_res_
    ↪ flags));

    SDK_CHECK_ERROR_GL();
}

void display()
{
    float4 *dptr;
    // 2. Map the VBO for writing from CUDA
    checkCudaErrors(cudaGraphicsMapResources(1, &cuda_vbo_resource, 0));
    size_t num_bytes;
    checkCudaErrors(cudaGraphicsResourceGetMappedPointer((void **)&dptr, &num_
    ↪ bytes, cuda_vbo_resource));

    // 3. Run CUDA kernel to modify the vertex positions
    // call the CUDA kernel
    dim3 block(8, 8, 1);
    dim3 grid(mesh_width / block.x, mesh_height / block.y, 1);
    simple_vbo_kernel<<<grid, block>>>(dptr, mesh_width, mesh_height, g_
    ↪ fAnim);

    // 4. Unmap the VBO
    checkCudaErrors(cudaGraphicsUnmapResources(1, &cuda_vbo_resource, 0));

    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

```

(continues on next page)

(continued from previous page)

```

// set view matrix
glMatrixMode(GL_MODELVIEW);
glLoadIdentity();
glTranslatef(0.0, 0.0, translate_z);
glRotatef(rotate_x, 1.0, 0.0, 0.0);
glRotatef(rotate_y, 0.0, 1.0, 0.0);

// 5. Render the updated using OpenGL
glBindBuffer(GL_ARRAY_BUFFER, vbo);
glVertexPointer(4, GL_FLOAT, 0, 0);

glEnableClientState(GL_VERTEX_ARRAY);
glColor3f(1.0, 0.0, 0.0);
glDrawArrays(GL_POINTS, 0, mesh_width * mesh_height);
glDisableClientState(GL_VERTEX_ARRAY);

glutSwapBuffers();

g_fAnim += 0.01f;
}

void deleteVBO(GLuint *vbo, struct cudaGraphicsResource *vbo_res)
{
    // 6. Unregister and delete VBO
    checkCudaErrors(cudaGraphicsUnregisterResource(vbo_res));

    glBindBuffer(1, *vbo);
    glDeleteBuffers(1, vbo);

    *vbo = 0;
}

void cleanup()
{
    if (vbo) {
        deleteVBO(&vbo, cuda_vbo_resource);
    }
}

```

Limitations and considerations.

- The OpenGL context whose resources are being shared has to be current to the host thread making any OpenGL interoperability API calls.
- When an OpenGL texture is made bindless (say for example by requesting an image or texture handle using the `glGetTextureHandle` or `glGetImageHandle` APIs) it cannot be registered with CUDA. The application needs to register the texture for interop before requesting an image or texture handle.

4.19.1.2 Direct3D Interoperability

Direct3D interoperability is supported for Direct3D9, Direct3D10, and Direct3D11 but not Direct3D12, here we focus on Direct3D11, for Direct3D9 and Direct3D10 please refer to the CUDA programming guide 12.9. The Direct3D resources that may be mapped into the address space of CUDA are Direct3D buffers, textures, and surfaces. These resources are registered using `cudaGraphicsD3D11RegisterResource()`.

A CUDA context may interoperate only with Direct3D11 devices created with `DriverType` set to `D3D_DRIVER_TYPE_HARDWARE`.

Example: 2D Texture Direct3D11 interoperability

The following code snippets are from the `simpleD3D11Texture` example, https://github.com/NVIDIA/cuda-samples/tree/master/Samples/5_Domain_Specific/simpleD3D11Texture. The full example includes a lot of boiler plate DX11 code, here we focus on the CUDA side.

The CUDA kernel `cuda_kernel_texture_2d` paints a 2D texture with a moving red/green hatch pattern on a strobing blue background, it is dependent on the previous texture values. The underlying data is a 2D CUDA array, where the row offsets are defined by the pitch.

```
/*
 * Paint a 2D texture with a moving red/green hatch pattern on a
 * strobing blue background. Note that this kernel reads to and
 * writes from the texture, hence why this texture was not mapped
 * as WriteDiscard.
 */
__global__ void cuda_kernel_texture_2d(unsigned char *surface, int width,
                                       int height, size_t pitch, float t) {
    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;
    float *pixel;

    // in the case where, due to quantization into grids, we have
    // more threads than pixels, skip the threads which don't
    // correspond to valid pixels
    if (x >= width || y >= height) return;

    // get a pointer to the pixel at (x,y)
    pixel = (float *) (surface + y * pitch) + 4 * x;

    // populate it
    float value_x = 0.5f + 0.5f * cos(t + 10.0f * ((2.0f * x) / width - 1.0f));
    float value_y = 0.5f + 0.5f * cos(t + 10.0f * ((2.0f * y) / height - 1.0f));
    pixel[0] = 0.5 * pixel[0] + 0.5 * pow(value_x, 3.0f); // red
    pixel[1] = 0.5 * pixel[1] + 0.5 * pow(value_y, 3.0f); // green
    pixel[2] = 0.5f + 0.5f * cos(t); // blue
    pixel[3] = 1; // alpha
}

extern "C" void cuda_texture_2d(void *surface, int width, int height,
                                size_t pitch, float t) {
    cudaError_t error = cudaSuccess;

    dim3 Db = dim3(16, 16); // block dimensions are fixed to be 256 threads
    dim3 Dg = dim3((width + Db.x - 1) / Db.x, (height + Db.y - 1) / Db.y);
```

(continues on next page)

(continued from previous page)

```

cuda_kernel_texture_2d<<<Dg, Db>>>((unsigned char *)surface, width, height,
                                     pitch, t);

error = cudaGetLastError();

if (error != cudaSuccess) {
    printf("cuda_kernel_texture_2d() failed to launch error = %d\n", error);
}
}

```

To keep the pointers and data buffers belonging together the following data structure is used:

```

// Data structure for 2D texture shared between DX11 and CUDA
struct {
    ID3D11Texture2D *pTexture;
    ID3D11ShaderResourceView *pSRView;
    cudaGraphicsResource *cudaResource;
    void *cudaLinearMemory;
    size_t pitch;
    int width;
    int height;
    int offsetInShader;
} g_texture_2d;

```

After the initialization of the Direct3D device and the textures, the resources are registered with CUDA once. To match the Direct3D pixel format, the CUDA array is allocated with the same width and height, and a pitch matching the Direct3D texture row pitch.

```

// register the Direct3D resources that are used in the CUDA kernel
// we'll read to and write from g_texture_2d, so don't set any special map
→ flags for it
    cudaGraphicsD3D11RegisterResource(&g_texture_2d.cudaResource,
                                     g_texture_2d.pTexture,
                                     cudaGraphicsRegisterFlagsNone);
    getLastCudaError("cudaGraphicsD3D11RegisterResource (g_texture_2d) failed
→");
    // CUDA cannot write into the texture directly : the texture is seen as a
    // cudaArray and can only be mapped as a texture
    // Create a buffer so that CUDA can write into it
    // the pixel fmt is DXGI_FORMAT_R32G32B32A32_FLOAT
    cudaMallocPitch(&g_texture_2d.cudaLinearMemory, &g_texture_2d.pitch,
                  g_texture_2d.width * sizeof(float) * 4,
                  g_texture_2d.height);
    getLastCudaError("cudaMallocPitch (g_texture_2d) failed");
    cudaMemset(g_texture_2d.cudaLinearMemory, 1,
              g_texture_2d.pitch * g_texture_2d.height);

```

In the rendering loop, the resources are mapped, the CUDA kernel is launched to update the texture data, and then the resources are unmapped. After this step the Direct3D device is used to draw the updated textures on the screen.

```

cudaStream_t stream = 0;
const int nbResources = 3;
cudaGraphicsResource *ppResources[nbResources] = {
    g_texture_2d.cudaResource, g_texture_3d.cudaResource,
    g_texture_cube.cudaResource,
};
cudaGraphicsMapResources(nbResources, ppResources, stream);
getLastCudaError("cudaGraphicsMapResources(3) failed");

// run kernels which will populate the contents of those textures
RunKernels();

// unmap the resources
cudaGraphicsUnmapResources(nbResources, ppResources, stream);
getLastCudaError("cudaGraphicsUnmapResources(3) failed");

```

Finally, once the resources are no longer needed in CUDA, they are unregistered and the device array freed.

```

// unregister the Cuda resources
cudaGraphicsUnregisterResource(g_texture_2d.cudaResource);
getLastCudaError("cudaGraphicsUnregisterResource (g_texture_2d) failed");
cudaFree(g_texture_2d.cudaLinearMemory);
getLastCudaError("cudaFree (g_texture_2d) failed");

```

4.19.1.3 Interoperability in a Scalable Link Interface (SLI) configuration

In a system with multiple GPUs, all CUDA-enabled GPUs are accessible via the CUDA driver and runtime as separate devices. This is different when the system is in SLI mode. SLI is a hardware configured multi-GPU configuration that offers increased rendering performance by dividing the workload across multiple GPUs. Implicit SLI mode, where the driver makes assumption is no longer supported, however explicit SLI is still supported. Explicit SLI means applications know and manage the SLI state through APIs (e.g. Vulkan, DirectX, GL) for all devices in the SLI group.

There are special considerations when the system is in SLI mode:

- ▶ An allocation in one CUDA device on one GPU will consume memory on other GPUs that are part of the SLI configuration of the Direct3D or OpenGL device. Because of this, allocations may fail earlier than otherwise expected.
- ▶ An application should create multiple CUDA contexts, one for each GPU in the SLI configuration. While this is not a strict requirement, it avoids unnecessary data transfers between devices. The application can use the `cudaD3D[9|10|11]GetDevices()` for Direct3D and `cudaGLGetDevices()` for OpenGL set of calls to identify the CUDA device handles for the devices that are performing the rendering in the current and next frame. Given this information the application will typically choose the appropriate device and map Direct3D or OpenGL resources to the CUDA device returned by `cudaD3D[9|10|11]GetDevices()` or `cudaGLGetDevices()` when the `deviceList` parameter is set to `cudaD3D[9|10|11]DeviceListCurrentFrame` or `cudaGLDeviceListCurrentFrame`.
- ▶ Resource returned from `cudaGraphicsD3D[9|10|11]RegisterResource` and `cudaGraphicsGLRegister[Buffer|Image]` must be only used on the device where the registration happened. Therefore, in SLI configurations when data for different frames is computed on different CUDA devices it is necessary to register the resources for each separately.

4.19.2. External resource interoperability

External resource interoperability allows CUDA to import certain resources that are explicitly exported by APIs. These objects are typically exported using handles native to the Operating System, like file descriptors on Linux or NT handles on Windows. This allows to efficiently share the resource between other APIs and CUDA without the need to copy or duplicate in between. It is supported for the following APIs, Direct3D[11-12], Vulkan and the NVIDIA Software Communication Interface Interoperability. There are two types of resources that can be imported:

► Memory objects

can be imported into CUDA using `cudaImportExternalMemory()`. An imported memory object can then be accessed from within kernels using device pointers mapped onto the memory object with `cudaExternalMemoryGetMappedBuffer()` or CUDA mipmapped arrays mapped with `cudaExternalMemoryGetMappedMipmappedArray()`. Depending on the type of memory object, it may be possible for more than one mapping to be setup on a single memory object. The mappings must match the mappings setup of the exporting API. Any mismatched mappings result in undefined behavior. Imported memory objects must be freed using `cudaDestroyExternalMemory()`. Freeing a memory object does not free any mappings to that object. Therefore, any device pointers mapped onto that object must be explicitly freed using `cudaFree()` and any CUDA mipmapped arrays mapped onto that object must be explicitly freed using `cudaFreeMipmappedArray()`. It is illegal to access mappings to an object after it has been destroyed.

► Synchronization objects

can be imported into CUDA using `cudaImportExternalSemaphore()`. An imported synchronization object can then be signaled using `cudaSignalExternalSemaphoresAsync()` and waited on using `cudaWaitExternalSemaphoresAsync()`. It is illegal to issue a wait before the corresponding signal has been issued. Also, depending on the type of the imported synchronization object, there may be additional constraints imposed on how they can be signaled and waited on, as described in subsequent sections. Imported semaphore objects must be freed using `cudaDestroyExternalSemaphore()`. All outstanding signals and waits must have completed before the semaphore object is destroyed.

4.19.2.1 Vulkan interoperability

Coupled execution of Vulkan graphics and compute workloads on the same hardware can maximize GPU utilization and avoid unnecessary copies. Note, this is not a Vulkan guide, we only focus on the interoperability with CUDA, for a Vulkan guide please refer to <https://www.vulkan.org/learn#vulkan-tutorials>.

The main steps to get a Vulkan-CUDA interoperability working involve:

1. Initialize Vulkan, create and export the external buffers and/or synchronization objects
2. Set the CUDA device where Vulkan is running with the matching devices UUIDs
3. Get the memory and/or synchronization handle
4. Import the memory and/or synchronization object in CUDA using these handles
5. Map the device pointer or mipmapped array onto the memory object
6. Use the imported memory objects in CUDA and Vulkan interchangeably by defining an order of execution through signaling and waiting on the synchronization objects.

In this section the steps above are explained with the help of the *simpleVulkan* example, https://github.com/NVIDIA/cuda-samples/tree/master/Samples/5_Domain_Specific/simpleVulkan. We walk through the example step by step, focusing on the parts needed for the CUDA interoperability. Some variation are explained with standalone snippets.

Note

The code example used in this section, uses the direct memory allocation and resource creation. Which is not state of the art due to several reasons, including the limitation to the number of instances that can be created. However, to understand the interoperability, one needs to know the underlying Vulkan code and the specific flags. For a more state of the art example, using the [VulkanMemoryAllocator](#) please refer to the *sample_cuda_interop* in the [NVProSamples](#) repository.

The following data structure is used throughout the example:

```
class VulkanCudaSineWave : public VulkanBaseApp {
    typedef struct UniformBufferObject_st {
        mat4x4 modelViewProj;
    } UniformBufferObject;

    VkBuffer m_heightBuffer, m_xyBuffer, m_indexBuffer;
    VkDeviceMemory m_heightMemory, m_xyMemory, m_indexMemory;
    UniformBufferObject m_ubo;
    VkSemaphore m_vkWaitSemaphore, m_vkSignalSemaphore;
    SineWaveSimulation m_sim;
    cudaStream_t m_stream;
    cudaExternalSemaphore_t m_cudaWaitSemaphore, m_cudaSignalSemaphore, m_
    ↪ cudaTimelineSemaphore;
    cudaExternalMemory_t m_cudaVertMem;
    float *m_cudaHeightMap;
    // ...
}
```

4.19.2.1.1 Setting up a Vulkan device

In order to export memory objects, a Vulkan instance must be created with the `VK_KHR_external_memory_capabilities` extension enabled and the device with `VK_KHR_external_memory`. In addition to the platform specific handle types must be enabled, for Windows `VK_KHR_external_memory_win32` and for UNIX based systems `VK_KHR_external_memory_fd`.

Similarly for exporting synchronization objects, on the device level `VK_KHR_external_semaphore_capabilities` and `VK_KHR_external_semaphore` on the instance level need to be enabled. As well as the platform specific extensions for the handles, that is `VK_KHR_external_semaphore_win32` for Windows and `VK_KHR_external_semaphore_fd` for Unix based systems.

In the *simpleVulkan* example these extensions are enabled with the following enums.

```
std::vector<const char*> getRequiredExtensions() const {
    std::vector<const char*> extensions;
    extensions.push_back(VK_KHR_EXTERNAL_MEMORY_CAPABILITIES_EXTENSION_NAME);
    extensions.push_back(VK_KHR_EXTERNAL_SEMAPHORE_CAPABILITIES_EXTENSION_
    ↪ NAME);
    return extensions;
}

std::vector<const char*> getRequiredDeviceExtensions() const {
    std::vector<const char*> extensions;
```

(continues on next page)

(continued from previous page)

```

    extensions.push_back(VK_KHR_EXTERNAL_MEMORY_EXTENSION_NAME);
    extensions.push_back(VK_KHR_EXTERNAL_SEMAPHORE_EXTENSION_NAME);
    extensions.push_back(VK_KHR_TIMELINE_SEMAPHORE_EXTENSION_NAME);
#ifdef _WIN64
    extensions.push_back(VK_KHR_EXTERNAL_MEMORY_WIN32_EXTENSION_NAME);
    extensions.push_back(VK_KHR_EXTERNAL_SEMAPHORE_WIN32_EXTENSION_NAME);
#else
    extensions.push_back(VK_KHR_EXTERNAL_MEMORY_FD_EXTENSION_NAME);
    extensions.push_back(VK_KHR_EXTERNAL_SEMAPHORE_FD_EXTENSION_NAME);
#endif /* _WIN64 */
    return extensions;
}

```

These are then added to the Vulkan instance and device creation info, please see the *simpleVulkan* example for details.

4.19.2.1.2 Initializing CUDA with matching device UUIDs

When importing memory and synchronization objects exported by Vulkan, they must be imported and mapped on the same device as they were created on. The CUDA device that corresponds to the Vulkan physical device on which the objects were created can be determined by comparing the UUID of a CUDA device with that of the Vulkan physical device, as shown in the following code snippet from the *simpleVulkan* example, where `vkDeviceUUID` is the member of the Vulkan API structure `vkPhysicalDeviceIDProperties.deviceUUID` and defines the physical devices id of the current Vulkan instance.

```

// from the CUDA example `simpleVulkan`
int SinWaveSimulation::initCuda(uint8_t *vkDeviceUUID, size_t UUID_SIZE) {
    int current_device = 0;
    int device_count = 0;
    int devices_prohibited = 0;

    cudaDeviceProp deviceProp;
    checkCudaErrors(cudaGetDeviceCount(&device_count));

    if (device_count == 0) {
        fprintf(stderr, "CUDA error: no devices supporting CUDA.\n");
        exit(EXIT_FAILURE);
    }

    // Find the GPU which is selected by Vulkan
    while (current_device < device_count) {
        cudaGetDeviceProperties(&deviceProp, current_device);

        if ((deviceProp.computeMode != cudaComputeModeProhibited)) {
            // Compare the cuda device UUID with vulkan UUID
            int ret = memcmp((void *)&deviceProp.uuid, vkDeviceUUID, UUID_SIZE);
            if (ret == 0) {
                checkCudaErrors(cudaSetDevice(current_device));
                checkCudaErrors(cudaGetDeviceProperties(&deviceProp, current_device));
                printf("GPU Device %d: \"%s\" with compute capability %d.%d\n\n",
                    current_device, deviceProp.name, deviceProp.major,

```

(continues on next page)

(continued from previous page)

```

        deviceProp.minor);

    return current_device;
}

} else {
    devices_prohibited++;
}

current_device++;
}

if (devices_prohibited == device_count) {
    fprintf(stderr,
        "CUDA error:"
        " No Vulkan-CUDA Interop capable GPU found.\n");
    exit(EXIT_FAILURE);
}

return -1;
}

```

Note that the Vulkan physical device should not be part of a device group that contains more than one Vulkan physical device. That is, the device group as returned by `vkEnumeratePhysicalDeviceGroups` that contains the given Vulkan physical device must have a physical device count of 1.

4.19.2.1.3 Exporting Vulkan memory objects

In order to export a Vulkan memory object, a buffer with the according export flags must be created. Note that the enums for the handle types are platform specific.

```

void VulkanBaseApp::createExternalBuffer(
    VkDeviceSize size, VkBufferUsageFlags usage,
    VkMemoryPropertyFlags properties,
    VkExternalMemoryHandleTypeFlagsKHR extMemHandleType, VkBuffer &buffer,
    VkDeviceMemory &bufferMemory) {
    VkBufferCreateInfo bufferInfo = {};
    bufferInfo.sType = VK_STRUCTURE_TYPE_BUFFER_CREATE_INFO;
    bufferInfo.size = size;
    bufferInfo.usage = usage;
    bufferInfo.sharingMode = VK_SHARING_MODE_EXCLUSIVE;

    VkExternalMemoryBufferCreateInfo externalMemoryBufferInfo = {};
    externalMemoryBufferInfo.sType =
        VK_STRUCTURE_TYPE_EXTERNAL_MEMORY_BUFFER_CREATE_INFO;
    externalMemoryBufferInfo.handleTypes = extMemHandleType;
    bufferInfo.pNext = &externalMemoryBufferInfo;

    if (vkCreateBuffer(m_device, &bufferInfo, nullptr, &buffer) != VK_SUCCESS) {
        throw std::runtime_error("failed to create buffer!");
    }
}

```

(continues on next page)

(continued from previous page)

```

VkMemoryRequirements memRequirements;
vkGetBufferMemoryRequirements(m_device, buffer, &memRequirements);

#ifdef _WIN64
    WindowsSecurityAttributes winSecurityAttributes;

    VkExportMemoryWin32HandleInfoKHR vulkanExportMemoryWin32HandleInfoKHR = {};
    vulkanExportMemoryWin32HandleInfoKHR.sType =
        VK_STRUCTURE_TYPE_EXPORT_MEMORY_WIN32_HANDLE_INFO_KHR;
    vulkanExportMemoryWin32HandleInfoKHR.pNext = NULL;
    vulkanExportMemoryWin32HandleInfoKHR.pAttributes = &winSecurityAttributes;
    vulkanExportMemoryWin32HandleInfoKHR.dwAccess =
        DXGI_SHARED_RESOURCE_READ | DXGI_SHARED_RESOURCE_WRITE;
    vulkanExportMemoryWin32HandleInfoKHR.name = (LPCWSTR)NULL;
#endif /* _WIN64 */
    VkExportMemoryAllocateInfoKHR vulkanExportMemoryAllocateInfoKHR = {};
    vulkanExportMemoryAllocateInfoKHR.sType =
        VK_STRUCTURE_TYPE_EXPORT_MEMORY_ALLOCATE_INFO_KHR;
#ifdef _WIN64
    vulkanExportMemoryAllocateInfoKHR.pNext =
        extMemHandleType & VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_BIT_KHR
        ? &vulkanExportMemoryWin32HandleInfoKHR
        : NULL;
    vulkanExportMemoryAllocateInfoKHR.handleTypes = extMemHandleType;
#else
    vulkanExportMemoryAllocateInfoKHR.pNext = NULL;
    vulkanExportMemoryAllocateInfoKHR.handleTypes =
        VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_FD_BIT;
#endif /* _WIN64 */
    VkMemoryAllocateInfo allocInfo = {};
    allocInfo.sType = VK_STRUCTURE_TYPE_MEMORY_ALLOCATE_INFO;
    allocInfo.pNext = &vulkanExportMemoryAllocateInfoKHR;
    allocInfo.allocationSize = memRequirements.size;
    allocInfo.memoryTypeIndex = findMemoryType(
        m_physicalDevice, memRequirements.memoryTypeBits, properties);

    if (vkAllocateMemory(m_device, &allocInfo, nullptr, &bufferMemory) !=
        VK_SUCCESS) {
        throw std::runtime_error("failed to allocate external buffer memory!");
    }

    vkBindBufferMemory(m_device, buffer, bufferMemory, 0);
}

```

4.19.2.1.4 Exporting Vulkan synchronization objects

Vulkan API calls which are executed on the GPU are asynchronous. To define an order of execution there are semaphores and fences available in Vulkan which can be shared with CUDA. Similar to the memory objects, semaphores can be exported by Vulkan, they need to be created with the export flags depending on the type of semaphore. There are binary and timeline semaphores. Binary semaphores only have a 1 bit counter, either signaled or not. Timeline semaphores have a 64 bit counter, which can be used to define an order of execution with the same semaphore. In the *simpleVulkan* example there

are code paths for both timeline and binary semaphores.

```
void VulkanBaseApp::createExternalSemaphore(
    VkSemaphore &semaphore, VkExternalSemaphoreHandleTypeFlagBits handleType)
{
    VkSemaphoreCreateInfo semaphoreInfo = {};
    semaphoreInfo.sType = VK_STRUCTURE_TYPE_SEMAPHORE_CREATE_INFO;
    VkExportSemaphoreCreateInfoKHR exportSemaphoreCreateInfo = {};
    exportSemaphoreCreateInfo.sType =
        VK_STRUCTURE_TYPE_EXPORT_SEMAPHORE_CREATE_INFO_KHR;

#ifdef _VK_TIMELINE_SEMAPHORE
    VkSemaphoreTypeCreateInfo timelineCreateInfo;
    timelineCreateInfo.sType = VK_STRUCTURE_TYPE_SEMAPHORE_TYPE_CREATE_INFO;
    timelineCreateInfo.pNext = NULL;
    timelineCreateInfo.semaphoreType = VK_SEMAPHORE_TYPE_TIMELINE;
    timelineCreateInfo.initialValue = 0;
    exportSemaphoreCreateInfo.pNext = &timelineCreateInfo;
#else
    exportSemaphoreCreateInfo.pNext = NULL;
#endif /* _VK_TIMELINE_SEMAPHORE */
    exportSemaphoreCreateInfo.handleTypes = handleType;
    semaphoreInfo.pNext = &exportSemaphoreCreateInfo;

    if (vkCreateSemaphore(m_device, &semaphoreInfo, nullptr, &semaphore) !=
        VK_SUCCESS) {
        throw std::runtime_error(
            "failed to create synchronization objects for a CUDA-Vulkan!");
    }
}
```

4.19.2.1.5 Importing memory objects

Both dedicated and non-dedicated memory objects exported by Vulkan can be imported into CUDA. When importing a Vulkan dedicated memory object, the flag `cudaExternalMemoryDedicated` must be set.

In Windows, a Vulkan memory object exported using `VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_BIT` can be imported into CUDA using the NT handle associated with that object as shown below. Note that CUDA does not assume ownership of the NT handle and it is the application's responsibility to close the handle when it is not required anymore. The NT handle holds a reference to the resource, so it must be explicitly freed before the underlying memory can be freed.

In Linux, a Vulkan memory object exported using `VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_FD_BIT` can be imported into CUDA using the file descriptor associated with that object as shown below. Note that CUDA assumes ownership of the file descriptor once it is imported. Using the file descriptor after a successful import results in undefined behavior.

```
// from the CUDA example `simpleVulkan`
void importCudaExternalMemory(void **cudaPtr, cudaExternalMemory_t &cudaMem,
    VkDeviceMemory &vkMem, VkDeviceSize size,
    VkExternalMemoryHandleTypeFlagBits
    handleType) {
    cudaExternalMemoryHandleDesc externalMemoryHandleDesc = {};
```

(continues on next page)

(continued from previous page)

```

if (handleType & VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_BIT) {
    externalMemoryHandleDesc.type = cudaExternalMemoryHandleTypeOpaqueWin32;
} else if (handleType &
           VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT) {
    externalMemoryHandleDesc.type =
        cudaExternalMemoryHandleTypeOpaqueWin32Kmt;
} else if (handleType & VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_FD_BIT) {
    externalMemoryHandleDesc.type = cudaExternalMemoryHandleTypeOpaqueFd;
} else {
    throw std::runtime_error("Unknown handle type requested!");
}

externalMemoryHandleDesc.size = size;

#ifdef _WIN64
    externalMemoryHandleDesc.handle.win32.handle =
        (HANDLE)getMemHandle(vkMem, handleType);
#else
    externalMemoryHandleDesc.handle.fd =
        (int)(uintptr_t)getMemHandle(vkMem, handleType);
#endif

    checkCudaErrors(
        cudaImportExternalMemory(&cudaMem, &externalMemoryHandleDesc));

```

A Vulkan memory object exported using VK_EXTERNAL_MEMORY_HANDLE_TYPE_OPAQUE_WIN32_BIT can also be imported using a named handle if one exists as shown in the standalone snippet below.

```

cudaExternalMemory_t importVulkanMemoryObjectFromNamedNTHandle(LPCWSTR name,
↳ unsigned long long size, bool isDedicated) {
    cudaExternalMemory_t extMem = NULL;
    cudaExternalMemoryHandleDesc desc = {};

    memset(&desc, 0, sizeof(desc));

    desc.type = cudaExternalMemoryHandleTypeOpaqueWin32;
    desc.handle.win32.name = (void *)name;
    desc.size = size;
    if (isDedicated) {
        desc.flags |= cudaExternalMemoryDedicated;
    }

    cudaImportExternalMemory(&extMem, &desc);

    return extMem;
}

```

4.19.2.1.6 Mapping buffers onto imported memory objects

After importing a memory object, they have to be mapped before they can be used. A device pointer can be mapped onto an imported memory object as shown below. The offset and size of the mapping must match that specified when creating the mapping using the corresponding Vulkan API. All mapped device pointers must be freed using `cudaFree()`.

```
// from the CUDA example 'simpleVulkan', continuation of function
→`importCudaExternalMemory`
    cudaExternalMemoryBufferDesc externalMemBufferDesc = {};
    externalMemBufferDesc.offset = 0;
    externalMemBufferDesc.size = size;
    externalMemBufferDesc.flags = 0;

    checkCudaErrors(cudaExternalMemoryGetMappedBuffer(cudaPtr, cudaMem,
&
→externalMemBufferDesc));
}
```

4.19.2.1.7 Mapping mipmapped arrays onto imported memory objects

A CUDA mipmapped array can be mapped onto an imported memory object as shown below. The offset, dimensions, format and number of mip levels must match that specified when creating the mapping using the corresponding Vulkan API. Additionally, if the mipmapped array is bound as a color target in Vulkan, the flag `cudaArrayColorAttachment` must be set. All mapped mipmapped arrays must be freed using `cudaFreeMipmappedArray()`. The following code standalone snippet shows how to convert Vulkan parameters into the corresponding CUDA parameters when mapping mipmapped arrays onto imported memory objects.

```
cudaMipmappedArray_t mapMipmappedArrayOntoExternalMemory(cudaExternalMemory_t
→extMem, unsigned long long offset, cudaChannelFormatDesc *formatDesc,
→cudaExtent *extent, unsigned int flags, unsigned int numLevels) {
    cudaMipmappedArray_t mipmap = NULL;
    cudaExternalMemoryMipmappedArrayDesc desc = {};

    memset(&desc, 0, sizeof(desc));

    desc.offset = offset;
    desc.formatDesc = *formatDesc;
    desc.extent = *extent;
    desc.flags = flags;
    desc.numLevels = numLevels;

    // Note: 'mipmap' must eventually be freed using cudaFreeMipmappedArray()
    cudaExternalMemoryGetMappedMipmappedArray(&mipmap, extMem, &desc);

    return mipmap;
}
//end mapMipmappedArrayOntoExternalMemory

//begin getCudaChannelFormatDescForVulkanFormat
cudaChannelFormatDesc getCudaChannelFormatDescForVulkanFormat(VkFormat format)
{
```

(continues on next page)

(continued from previous page)

```

cudaChannelFormatDesc d;

memset(&d, 0, sizeof(d));

switch (format) {
    case VK_FORMAT_R8_UINT:          d.x = 8;  d.y = 0;  d.z = 0;  d.w
→= 0; d.f = cudaChannelFormatKindUnsigned; break;
    case VK_FORMAT_R8_SINT:          d.x = 8;  d.y = 0;  d.z = 0;  d.w
→= 0; d.f = cudaChannelFormatKindSigned;   break;
    case VK_FORMAT_R8G8_UINT:        d.x = 8;  d.y = 8;  d.z = 0;  d.w
→= 0; d.f = cudaChannelFormatKindUnsigned; break;
    case VK_FORMAT_R8G8_SINT:        d.x = 8;  d.y = 8;  d.z = 0;  d.w
→= 0; d.f = cudaChannelFormatKindSigned;   break;
    case VK_FORMAT_R8G8B8A8_UINT:    d.x = 8;  d.y = 8;  d.z = 8;  d.w
→= 8; d.f = cudaChannelFormatKindUnsigned; break;
    case VK_FORMAT_R8G8B8A8_SINT:    d.x = 8;  d.y = 8;  d.z = 8;  d.w
→= 8; d.f = cudaChannelFormatKindSigned;   break;
    case VK_FORMAT_R16_UINT:         d.x = 16; d.y = 0;  d.z = 0;  d.w
→= 0; d.f = cudaChannelFormatKindUnsigned; break;
    case VK_FORMAT_R16_SINT:         d.x = 16; d.y = 0;  d.z = 0;  d.w
→= 0; d.f = cudaChannelFormatKindSigned;   break;
    case VK_FORMAT_R16G16_UINT:      d.x = 16; d.y = 16; d.z = 0;  d.w
→= 0; d.f = cudaChannelFormatKindUnsigned; break;
    case VK_FORMAT_R16G16_SINT:      d.x = 16; d.y = 16; d.z = 0;  d.w
→= 0; d.f = cudaChannelFormatKindSigned;   break;
    case VK_FORMAT_R16G16B16A16_UINT: d.x = 16; d.y = 16; d.z = 16; d.w
→= 16; d.f = cudaChannelFormatKindUnsigned; break;
    case VK_FORMAT_R16G16B16A16_SINT: d.x = 16; d.y = 16; d.z = 16; d.w
→= 16; d.f = cudaChannelFormatKindSigned;   break;
    case VK_FORMAT_R32_UINT:         d.x = 32; d.y = 0;  d.z = 0;  d.w
→= 0; d.f = cudaChannelFormatKindUnsigned; break;
    case VK_FORMAT_R32_SINT:         d.x = 32; d.y = 0;  d.z = 0;  d.w
→= 0; d.f = cudaChannelFormatKindSigned;   break;
    case VK_FORMAT_R32_SFLOAT:       d.x = 32; d.y = 0;  d.z = 0;  d.w
→= 0; d.f = cudaChannelFormatKindFloat;    break;
    case VK_FORMAT_R32G32_UINT:      d.x = 32; d.y = 32; d.z = 0;  d.w
→= 0; d.f = cudaChannelFormatKindUnsigned; break;
    case VK_FORMAT_R32G32_SINT:      d.x = 32; d.y = 32; d.z = 0;  d.w
→= 0; d.f = cudaChannelFormatKindSigned;   break;
    case VK_FORMAT_R32G32_SFLOAT:    d.x = 32; d.y = 32; d.z = 0;  d.w
→= 0; d.f = cudaChannelFormatKindFloat;    break;
    case VK_FORMAT_R32G32B32A32_UINT: d.x = 32; d.y = 32; d.z = 32; d.w
→= 32; d.f = cudaChannelFormatKindUnsigned; break;
    case VK_FORMAT_R32G32B32A32_SINT: d.x = 32; d.y = 32; d.z = 32; d.w
→= 32; d.f = cudaChannelFormatKindSigned;   break;
    case VK_FORMAT_R32G32B32A32_SFLOAT: d.x = 32; d.y = 32; d.z = 32; d.w
→= 32; d.f = cudaChannelFormatKindFloat;    break;
    default: assert(0);
}
return d;
}
//end getCudaChannelFormatDescForVulkanFormat

```

(continues on next page)

(continued from previous page)

```

//begin getCudaExtentForVulkanExtent
cudaExtent getCudaExtentForVulkanExtent(VkExtent3D vkExt, uint32_t
→arrayLayers, VkImageViewType vkImageViewType) {
    cudaExtent e = { 0, 0, 0 };

    switch (vkImageViewType) {
        case VK_IMAGE_VIEW_TYPE_1D:          e.width = vkExt.width; e.height =
→0;          e.depth = 0;          break;
        case VK_IMAGE_VIEW_TYPE_2D:          e.width = vkExt.width; e.height =
→vkExt.height; e.depth = 0;          break;
        case VK_IMAGE_VIEW_TYPE_3D:          e.width = vkExt.width; e.height =
→vkExt.height; e.depth = vkExt.depth; break;
        case VK_IMAGE_VIEW_TYPE_CUBE:        e.width = vkExt.width; e.height =
→vkExt.height; e.depth = arrayLayers; break;
        case VK_IMAGE_VIEW_TYPE_1D_ARRAY:    e.width = vkExt.width; e.height =
→0;          e.depth = arrayLayers; break;
        case VK_IMAGE_VIEW_TYPE_2D_ARRAY:    e.width = vkExt.width; e.height =
→vkExt.height; e.depth = arrayLayers; break;
        case VK_IMAGE_VIEW_TYPE_CUBE_ARRAY:  e.width = vkExt.width; e.height =
→vkExt.height; e.depth = arrayLayers; break;
        default: assert(0);
    }

    return e;
}
//end getCudaExtentForVulkanExtent

//begin getCudaMipmappedArrayFlagsForVulkanImage
unsigned int getCudaMipmappedArrayFlagsForVulkanImage(VkImageViewType
→vkImageViewType,
                                                    VkImageUsageFlags
→vkImageUsageFlags,
                                                    bool
→allowSurfaceLoadStore) {
    unsigned int flags = 0;

    switch (vkImageViewType) {
        case VK_IMAGE_VIEW_TYPE_CUBE:        flags |= cudaArrayCubemap;
→          break;
        case VK_IMAGE_VIEW_TYPE_CUBE_ARRAY:  flags |= cudaArrayCubemap |
→cudaArrayLayered; break;
        case VK_IMAGE_VIEW_TYPE_1D_ARRAY:    flags |= cudaArrayLayered;
→          break;
        case VK_IMAGE_VIEW_TYPE_2D_ARRAY:    flags |= cudaArrayLayered;
→          break;
        default: break;
    }
    if (vkImageUsageFlags & VK_IMAGE_USAGE_COLOR_ATTACHMENT_BIT) {
        flags |= cudaArrayColorAttachment;
    }
}

```

(continues on next page)

(continued from previous page)

```

    if (allowSurfaceLoadStore) {
        flags |= cudaArraySurfaceLoadStore;
    }

    return flags;
}

```

4.19.2.1.8 Importing Synchronization Objects

A Vulkan semaphore object exported using `VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_FD_BIT` can be imported into CUDA using the file descriptor associated with that object as shown below. Note that CUDA assumes ownership of the file descriptor once it is imported. Using the file descriptor after a successful import results in undefined behavior.

Whereas a Vulkan semaphore object exported using `VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_BIT` can be imported into CUDA using the NT handle associated with that object as shown below. Note that CUDA does not assume ownership of the NT handle and it is the application's responsibility to close the handle when it is not required anymore. The NT handle holds a reference to the resource, so it must be explicitly freed before the underlying semaphore can be freed.

And, a Vulkan semaphore object exported using `VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT` can be imported into CUDA using the globally shared D3DKMT handle associated with that object as shown below. Since a globally shared D3DKMT handle does not hold a reference to the underlying semaphore it is automatically destroyed when all other references to the resource are destroyed.

```

void importCudaExternalSemaphore(
    cudaExternalSemaphore_t &cudaSem, VkSemaphore &vkSem,
    VkExternalSemaphoreHandleTypeFlagBits handleType) {
    cudaExternalSemaphoreHandleDesc externalSemaphoreHandleDesc = {};

#ifdef _VK_TIMELINE_SEMAPHORE
    if (handleType & VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_BIT) {
        externalSemaphoreHandleDesc.type =
            cudaExternalSemaphoreHandleTypeTimelineSemaphoreWin32;
    } else if (handleType &
        VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT) {
        externalSemaphoreHandleDesc.type =
            cudaExternalSemaphoreHandleTypeTimelineSemaphoreWin32;
    } else if (handleType & VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_FD_BIT) {
        externalSemaphoreHandleDesc.type =
            cudaExternalSemaphoreHandleTypeTimelineSemaphoreFd;
    }
#else
    if (handleType & VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_BIT) {
        externalSemaphoreHandleDesc.type =
            cudaExternalSemaphoreHandleTypeOpaqueWin32;
    } else if (handleType &
        VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_WIN32_KMT_BIT) {
        externalSemaphoreHandleDesc.type =
            cudaExternalSemaphoreHandleTypeOpaqueWin32Kmt;
    } else if (handleType & VK_EXTERNAL_SEMAPHORE_HANDLE_TYPE_OPAQUE_FD_BIT) {
        externalSemaphoreHandleDesc.type =

```

(continues on next page)

(continued from previous page)

```

        cudaExternalSemaphoreHandleTypeOpaqueFd;
    }
#endif /* _VK_TIMELINE_SEMAPHORE */
    else {
        throw std::runtime_error("Unknown handle type requested!");
    }

#ifdef _WIN64
    externalSemaphoreHandleDesc.handle.win32.handle =
        (HANDLE)getSemaphoreHandle(vkSem, handleType);
#else
    externalSemaphoreHandleDesc.handle.fd =
        (int)(uintptr_t)getSemaphoreHandle(vkSem, handleType);
#endif

    externalSemaphoreHandleDesc.flags = 0;

    checkCudaErrors(
        cudaImportExternalSemaphore(&cudaSem, &externalSemaphoreHandleDesc));
}

```

4.19.2.1.9 Signaling/Waiting on Imported Synchronization Objects

An imported Vulkan semaphore can be signaled and waited on as shown below. Signaling a semaphore sets it to the signaled state and in the case of timeline semaphores it sets the counter to the value specified in the signal call. The corresponding wait that waits on this signal must be issued in Vulkan. Additionally, in the case of a binary semaphore, the wait that waits on this signal must be issued after this signal has been issued.

Waiting on a semaphore waits until it reaches the signaled state or the assigned wait value. A signaled binary semaphore then resets it back to the unsignaled state. The corresponding signal that this wait is waiting on must be issued in Vulkan. Additionally, in the case of a binary semaphore, the signal must be issued before this wait can be issued.

In the following code extract from the *simpleVulkan* example the simulation step / the CUDA kernel is only called once the semaphore around the vertex buffers is signaled by Vulkan. After the simulation step another semaphore is signaled, or in the case of the timeline semaphore the same one is increased by CUDA, such that the Vulkan part that is waiting on this semaphore can continue rendering with the updated vertex buffers.

```

#ifdef _VK_TIMELINE_SEMAPHORE
    static uint64_t waitValue = 1;
    static uint64_t signalValue = 2;

    cudaExternalSemaphoreWaitParams waitParams = {};
    waitParams.flags = 0;
    waitParams.params.fence.value = waitValue;

    cudaExternalSemaphoreSignalParams signalParams = {};
    signalParams.flags = 0;
    signalParams.params.fence.value = signalValue;
    // Wait for vulkan to complete it's work

```

(continues on next page)

(continued from previous page)

```

        checkCudaErrors(cudaWaitExternalSemaphoresAsync(&m_cudaTimelineSemaphore,
                                                         &waitParams, 1, m_
→stream));
        // Now step the simulation, call CUDA kernel
        m_sim.stepSimulation(time, m_stream);
        // Signal vulkan to continue with the updated buffers
        checkCudaErrors(cudaSignalExternalSemaphoresAsync(
            &m_cudaTimelineSemaphore, &signalParams, 1, m_stream));

        waitValue += 2;
        signalValue += 2;
    #else
        cudaExternalSemaphoreWaitParams waitParams = {};
        waitParams.flags = 0;
        waitParams.params.fence.value = 0;

        cudaExternalSemaphoreSignalParams signalParams = {};
        signalParams.flags = 0;
        signalParams.params.fence.value = 0;

        // Wait for vulkan to complete it's work
        checkCudaErrors(cudaWaitExternalSemaphoresAsync(&m_cudaWaitSemaphore,
                                                         &waitParams, 1, m_
→stream));
        // Now step the simulation, call CUDA kernel
        m_sim.stepSimulation(time, m_stream);
        // Signal vulkan to continue with the updated buffers
        checkCudaErrors(cudaSignalExternalSemaphoresAsync(
            &m_cudaSignalSemaphore, &signalParams, 1, m_stream));
    #endif /* _VK_TIMELINE_SEMAPHORE */

```

4.19.2.1.10 OpenGL Interoperability

Traditional OpenGL-CUDA interop as outlined in [OpenGL Interoperability](#) works by CUDA directly consuming handles created in OpenGL. However, since OpenGL can also consume memory and synchronization objects created in Vulkan, there exists an alternative approach to doing OpenGL-CUDA interop. Essentially, memory and synchronization objects exported by Vulkan could be imported into both, OpenGL and CUDA, and then used to coordinate memory accesses between OpenGL and CUDA. Please refer to the following OpenGL extensions for further details on how to import memory and synchronization objects exported by Vulkan:

- ▶ GL_EXT_memory_object
- ▶ GL_EXT_memory_object_fd
- ▶ GL_EXT_memory_object_win32
- ▶ GL_EXT_semaphore
- ▶ GL_EXT_semaphore_fd
- ▶ GL_EXT_semaphore_win32

4.19.2.2 Direct3D Interoperability

Importing Direct3D[11|12] resources to CUDA is supported for Direct3D11 and Direct3D12. We are only looking at the Direct3D12, for Direct3D11 please refer to the CUDA programming guide 12.9.

4.19.2.2.1 Matching Device LUIDs

When importing memory and synchronization objects exported by Direct3D12, they must be imported and mapped on the same device as they were created on. The CUDA device that corresponds to the Direct3D12 device on which the objects were created can be determined by comparing the LUID of a CUDA device with that of the Direct3D12 device, as shown in the following code sample. Note that the Direct3D12 device must not be created on a linked node adapter, i.e. the node count as returned by `ID3D12Device::GetNodeCount` must be 1.

```
int getCudaDeviceForD3D12Device(ID3D12Device *d3d12Device) {
    LUID d3d12Luid = d3d12Device->GetAdapterLuid();

    int cudaDeviceCount;
    cudaGetDeviceCount(&cudaDeviceCount);

    for (int cudaDevice = 0; cudaDevice < cudaDeviceCount; cudaDevice++) {
        cudaDeviceProp deviceProp;
        cudaGetDeviceProperties(&deviceProp, cudaDevice);
        char *cudaLuid = deviceProp.luid;

        if (!memcmp(&d3d12Luid.LowPart, cudaLuid, sizeof(d3d12Luid.LowPart)) &
            ↪& !memcmp(&d3d12Luid.HighPart, cudaLuid + sizeof(d3d12Luid.LowPart),
            ↪ sizeof(d3d12Luid.HighPart))) {
            return cudaDevice;
        }
    }
    return cudaInvalidDeviceId;
}
```

4.19.2.2.2 Importing Memory Objects

There are several different ways how to import memory objects from NT handles. Note that it is the application's responsibility to close the NT handle when it is not required anymore. The NT handle holds a reference to the resource, so it must be explicitly freed before the underlying memory can be freed. When importing a Direct3D resource, the flag `cudaExternalMemoryDedicated` must be set as in the snippets below.

A shareable Direct3D12 heap memory object, created by setting the flag `D3D12_HEAP_FLAG_SHARED` in the call to `ID3D12Device::CreateHeap`, can be imported into CUDA using the NT handle associated with that object as shown below.

```
cudaExternalMemory_t importD3D12HeapFromNTHandle(HANDLE handle, unsigned long
    ↪long size) {
    cudaExternalMemory_t extMem = NULL;
    cudaExternalMemoryHandleDesc desc = {};

    memset(&desc, 0, sizeof(desc));
```

(continues on next page)

(continued from previous page)

```

desc.type = cudaExternalMemoryHandleTypeD3D12Heap;
desc.handle.win32.handle = (void *)handle;
desc.size = size;

cudaImportExternalMemory(&extMem, &desc);

// Input parameter 'handle' should be closed if it's not needed anymore
CloseHandle(handle);

return extMem;
}

```

A shareable Direct3D12 heap memory object can also be imported using a named handle if one exists:

```

cudaExternalMemory_t importD3D12HeapFromNamedNTHandle(LPCWSTR name, unsigned
→long long size) {
    cudaExternalMemory_t extMem = NULL;
    cudaExternalMemoryHandleDesc desc = {};

    memset(&desc, 0, sizeof(desc));

    desc.type = cudaExternalMemoryHandleTypeD3D12Heap;
    desc.handle.win32.name = (void *)name;
    desc.size = size;

    cudaImportExternalMemory(&extMem, &desc);

    return extMem;
}

```

A shareable Direct3D12 committed resource, created by setting the flag `D3D12_HEAP_FLAG_SHARED` in the call to `D3D12Device::CreateCommittedResource`, can be imported into CUDA using the NT handle associated with that object as shown below. When importing a Direct3D12 committed resource, the flag `cudaExternalMemoryDedicated` must be set.

```

cudaExternalMemory_t importD3D12CommittedResourceFromNTHandle(HANDLE handle,
→unsigned long long size) {
    cudaExternalMemory_t extMem = NULL;
    cudaExternalMemoryHandleDesc desc = {};

    memset(&desc, 0, sizeof(desc));

    desc.type = cudaExternalMemoryHandleTypeD3D12Resource;
    desc.handle.win32.handle = (void *)handle;
    desc.size = size;
    desc.flags |= cudaExternalMemoryDedicated;

    cudaImportExternalMemory(&extMem, &desc);

    // Input parameter 'handle' should be closed if it's not needed anymore
    CloseHandle(handle);
}

```

(continues on next page)

(continued from previous page)

```

    return extMem;
}

```

A shareable Direct3D12 committed resource can also be imported using a named handle if one exists as shown below.

```

cudaExternalMemory_t importD3D12CommittedResourceFromNamedNTHandle(LPCWSTR
↪name, unsigned long long size) {
    cudaExternalMemory_t extMem = NULL;
    cudaExternalMemoryHandleDesc desc = {};

    memset(&desc, 0, sizeof(desc));

    desc.type = cudaExternalMemoryHandleTypeD3D12Resource;
    desc.handle.win32.name = (void *)name;
    desc.size = size;
    desc.flags |= cudaExternalMemoryDedicated;

    cudaImportExternalMemory(&extMem, &desc);

    return extMem;
}

```

4.19.2.2.3 Mapping Buffers onto Imported Memory Objects

A device pointer can be mapped onto an imported memory object as shown below. The offset and size of the mapping must match that specified when creating the mapping using the corresponding Direct3D12 API. All mapped device pointers must be freed using `cudaFree()`.

```

void * mapBufferOntoExternalMemory(cudaExternalMemory_t extMem, unsigned long
↪long offset, unsigned long long size) {
    void *ptr = NULL;
    cudaExternalMemoryBufferDesc desc = {};

    memset(&desc, 0, sizeof(desc));

    desc.offset = offset;
    desc.size = size;

    cudaExternalMemoryGetMappedBuffer(&ptr, extMem, &desc);

    // Note: 'ptr' must eventually be freed using cudaFree()
    return ptr;
}

```

4.19.2.2.4 Mapping Mipmapped Arrays onto Imported Memory Objects

A CUDA mipmapped array can be mapped onto an imported memory object as shown below. The offset, dimensions, format and number of mip levels must match that specified when creating the mapping using the corresponding Direct3D12 API. Additionally, if the mipmapped array can be bound as a render target in Direct3D12, the flag `cudaArrayColorAttachment` must be set. All mapped mipmapped arrays must be freed using `cudaFreeMipmappedArray()`. The following code

sample shows how to convert parameters into the corresponding CUDA parameters when mapping mipmapped arrays onto imported memory objects.

```
cudaMipmappedArray_t mapMipmappedArrayOntoExternalMemory(cudaExternalMemory_t
↳extMem, unsigned long long offset, cudaChannelFormatDesc *formatDesc,
↳cudaExtent *extent, unsigned int flags, unsigned int numLevels) {
    cudaMipmappedArray_t mipmap = NULL;
    cudaExternalMemoryMipmappedArrayDesc desc = {};

    memset(&desc, 0, sizeof(desc));

    desc.offset = offset;
    desc.formatDesc = *formatDesc;
    desc.extent = *extent;
    desc.flags = flags;
    desc.numLevels = numLevels;

    // Note: 'mipmap' must eventually be freed using cudaFreeMipmappedArray()
    cudaExternalMemoryGetMappedMipmappedArray(&mipmap, extMem, &desc);

    return mipmap;
}
```

```
cudaChannelFormatDesc getCudaChannelFormatDescForDxgiFormat(DXGI_FORMAT
↳dxgiFormat)
{
    cudaChannelFormatDesc d;

    memset(&d, 0, sizeof(d));

    switch (dxgiFormat) {
        case DXGI_FORMAT_R8_UINT:           d.x = 8;  d.y = 0;  d.z = 0;  d.
↳w = 0; d.f = cudaChannelFormatKindUnsigned; break;
        case DXGI_FORMAT_R8_SINT:           d.x = 8;  d.y = 0;  d.z = 0;  d.
↳w = 0; d.f = cudaChannelFormatKindSigned;   break;
        case DXGI_FORMAT_R8G8_UINT:         d.x = 8;  d.y = 8;  d.z = 0;  d.
↳w = 0; d.f = cudaChannelFormatKindUnsigned; break;
        case DXGI_FORMAT_R8G8_SINT:         d.x = 8;  d.y = 8;  d.z = 0;  d.
↳w = 0; d.f = cudaChannelFormatKindSigned;   break;
        case DXGI_FORMAT_R8G8B8A8_UINT:     d.x = 8;  d.y = 8;  d.z = 8;  d.
↳w = 8; d.f = cudaChannelFormatKindUnsigned; break;
        case DXGI_FORMAT_R8G8B8A8_SINT:     d.x = 8;  d.y = 8;  d.z = 8;  d.
↳w = 8; d.f = cudaChannelFormatKindSigned;   break;
        case DXGI_FORMAT_R16_UINT:          d.x = 16; d.y = 0;  d.z = 0;  d.
↳w = 0; d.f = cudaChannelFormatKindUnsigned; break;
        case DXGI_FORMAT_R16_SINT:          d.x = 16; d.y = 0;  d.z = 0;  d.
↳w = 0; d.f = cudaChannelFormatKindSigned;   break;
        case DXGI_FORMAT_R16G16_UINT:       d.x = 16; d.y = 16; d.z = 0;  d.
↳w = 0; d.f = cudaChannelFormatKindUnsigned; break;
        case DXGI_FORMAT_R16G16_SINT:       d.x = 16; d.y = 16; d.z = 0;  d.
↳w = 0; d.f = cudaChannelFormatKindSigned;   break;
        case DXGI_FORMAT_R16G16B16A16_UINT: d.x = 16; d.y = 16; d.z = 16; d.
↳w = 16; d.f = cudaChannelFormatKindUnsigned; break;
    }
```

(continues on next page)

(continued from previous page)

```

        case DXGI_FORMAT_R16G16B16A16_SINT: d.x = 16; d.y = 16; d.z = 16; d.
→w = 16; d.f = cudaChannelFormatKindSigned; break;
        case DXGI_FORMAT_R32_UINT: d.x = 32; d.y = 0; d.z = 0; d.
→w = 0; d.f = cudaChannelFormatKindUnsigned; break;
        case DXGI_FORMAT_R32_SINT: d.x = 32; d.y = 0; d.z = 0; d.
→w = 0; d.f = cudaChannelFormatKindSigned; break;
        case DXGI_FORMAT_R32_FLOAT: d.x = 32; d.y = 0; d.z = 0; d.
→w = 0; d.f = cudaChannelFormatKindFloat; break;
        case DXGI_FORMAT_R32G32_UINT: d.x = 32; d.y = 32; d.z = 0; d.
→w = 0; d.f = cudaChannelFormatKindUnsigned; break;
        case DXGI_FORMAT_R32G32_SINT: d.x = 32; d.y = 32; d.z = 0; d.
→w = 0; d.f = cudaChannelFormatKindSigned; break;
        case DXGI_FORMAT_R32G32_FLOAT: d.x = 32; d.y = 32; d.z = 0; d.
→w = 0; d.f = cudaChannelFormatKindFloat; break;
        case DXGI_FORMAT_R32G32B32A32_UINT: d.x = 32; d.y = 32; d.z = 32; d.
→w = 32; d.f = cudaChannelFormatKindUnsigned; break;
        case DXGI_FORMAT_R32G32B32A32_SINT: d.x = 32; d.y = 32; d.z = 32; d.
→w = 32; d.f = cudaChannelFormatKindSigned; break;
        case DXGI_FORMAT_R32G32B32A32_FLOAT: d.x = 32; d.y = 32; d.z = 32; d.
→w = 32; d.f = cudaChannelFormatKindFloat; break;
        default: assert(0);
    }
    return d;
}

```

```

cudaExtent getCudaExtentForD3D12Extent(UINT64 width, UINT height, UINT16
→depthOrArraySize, D3D12_SRV_DIMENSION d3d12SRVDimension) {
    cudaExtent e = { 0, 0, 0 };

    switch (d3d12SRVDimension) {
        case D3D12_SRV_DIMENSION_TEXTURE1D: e.width = width; e.height
→= 0; e.depth = 0; break;
        case D3D12_SRV_DIMENSION_TEXTURE2D: e.width = width; e.height
→= height; e.depth = 0; break;
        case D3D12_SRV_DIMENSION_TEXTURE3D: e.width = width; e.height
→= height; e.depth = depthOrArraySize; break;
        case D3D12_SRV_DIMENSION_TEXTURECUBE: e.width = width; e.height
→= height; e.depth = depthOrArraySize; break;
        case D3D12_SRV_DIMENSION_TEXTURE1DARRAY: e.width = width; e.height
→= 0; e.depth = depthOrArraySize; break;
        case D3D12_SRV_DIMENSION_TEXTURE2DARRAY: e.width = width; e.height
→= height; e.depth = depthOrArraySize; break;
        case D3D12_SRV_DIMENSION_TEXTURECUBEARRAY: e.width = width; e.height
→= height; e.depth = depthOrArraySize; break;
        default: assert(0);
    }

    return e;
}

```

```

unsigned int getCudaMipmappedArrayFlagsForD3D12Resource(D3D12_SRV_DIMENSION
→d3d12SRVDimension, D3D12_RESOURCE_FLAGS d3d12ResourceFlags, bool

```

(continues on next page)

(continued from previous page)

```

→allowSurfaceLoadStore) {
    unsigned int flags = 0;

    switch (d3d12SRVDimension) {
        case D3D12_SRV_DIMENSION_TEXTURECUBE:      flags |= cudaArrayCubemap;
→           break;
        case D3D12_SRV_DIMENSION_TEXTURECUBEARRAY: flags |= cudaArrayCubemap
→| cudaArrayLayered; break;
        case D3D12_SRV_DIMENSION_TEXTURE1DARRAY:   flags |= cudaArrayLayered;
→           break;
        case D3D12_SRV_DIMENSION_TEXTURE2DARRAY:   flags |= cudaArrayLayered;
→           break;
        default: break;
    }

    if (d3d12ResourceFlags & D3D12_RESOURCE_FLAG_ALLOW_RENDER_TARGET) {
        flags |= cudaArrayColorAttachment;
    }
    if (allowSurfaceLoadStore) {
        flags |= cudaArraySurfaceLoadStore;
    }

    return flags;
}

```

4.19.2.2.5 Importing Synchronization Objects

A shareable Direct3D12 fence object, created by setting the flag `D3D12_FENCE_FLAG_SHARED` in the call to `ID3D12Device::CreateFence`, can be imported into CUDA using the NT handle associated with that object as shown below. Note that it is the application's responsibility to close the handle when it is not required anymore. The NT handle holds a reference to the resource, so it must be explicitly freed before the underlying semaphore can be freed.

```

cudaExternalSemaphore_t importD3D12FenceFromNTHandle(HANDLE handle) {
    cudaExternalSemaphore_t extSem = NULL;
    cudaExternalSemaphoreHandleDesc desc = {};

    memset(&desc, 0, sizeof(desc));

    desc.type = cudaExternalSemaphoreHandleTypeD3D12Fence;
    desc.handle.win32.handle = handle;

    cudaImportExternalSemaphore(&extSem, &desc);

    // Input parameter 'handle' should be closed if it's not needed anymore
    CloseHandle(handle);

    return extSem;
}

```

A shareable Direct3D12 fence object can also be imported using a named handle if one exists as shown below.

```
cudaExternalSemaphore_t importD3D12FenceFromNamedNTHandle(LPCWSTR name) {
    cudaExternalSemaphore_t extSem = NULL;
    cudaExternalSemaphoreHandleDesc desc = {};

    memset(&desc, 0, sizeof(desc));

    desc.type = cudaExternalSemaphoreHandleTypeD3D12Fence;
    desc.handle.win32.name = (void *)name;

    cudaImportExternalSemaphore(&extSem, &desc);

    return extSem;
}
```

4.19.2.2.6 Signaling/Waiting on Imported Synchronization Objects

Once the semaphores with fences have been imported from Direct3D12 they can be signaled and waited on.

Signaling a fence object sets its value. The corresponding wait that waits on this signal must be issued in Direct3D12. Note that the wait that waits on this signal must be issued after this signal has been issued.

```
void signalExternalSemaphore(cudaExternalSemaphore_t extSem, unsigned long
→long value, cudaStream_t stream) {
    cudaExternalSemaphoreSignalParams params = {};

    memset(&params, 0, sizeof(params));

    params.params.fence.value = value;

    cudaSignalExternalSemaphoresAsync(&extSem, &params, 1, stream);
}
```

A fence object waits until its value becomes greater than or equal to the specified value. The corresponding signal that it is waiting on must be issued in Direct3D12. Note that, the signal must be issued before this wait can be issued.

```
void waitExternalSemaphore(cudaExternalSemaphore_t extSem, unsigned long long
→value, cudaStream_t stream) {
    cudaExternalSemaphoreWaitParams params = {};

    memset(&params, 0, sizeof(params));

    params.params.fence.value = value;

    cudaWaitExternalSemaphoresAsync(&extSem, &params, 1, stream);
}
```

4.19.2.3 NVIDIA Software Communication Interface Interoperability (NVSCI)

NvSciBuf and NvSciSync are interfaces developed for serving the following purposes:

- ▶ NvSciBuf: Allows applications to allocate and exchange buffers in memory
- ▶ NvSciSync: Allows applications to manage synchronization objects at operation boundaries

More details on these interfaces are available at: <https://docs.nvidia.com/drive>.

4.19.2.3.1 Importing Memory Objects

For allocating an NvSciBuf object compatible with a given CUDA device, the corresponding GPU id must be set with NvSciBufGeneralAttrKey_GpuId in the NvSciBuf attribute list as shown below. Optionally, applications can specify the following attributes -

- ▶ NvSciBufGeneralAttrKey_NeedCpuAccess: Specifies if CPU access is required for the buffer
- ▶ NvSciBufRawBufferAttrKey_Align: Specifies the alignment requirement of NvSciBufType_RawBuffer
- ▶ NvSciBufGeneralAttrKey_RequiredPerm: Different access permissions can be configured for different UMDs per NvSciBuf memory object instance. For example, to provide the GPU with read-only access permissions to the buffer, create a duplicate NvSciBuf object using NvSciBufObjDupWithReducePerm() with NvSciBufAccessPerm_Readonly as the input parameter. Then import this newly created duplicate object with reduced permission into CUDA as shown
- ▶ NvSciBufGeneralAttrKey_EnableGpuCache: To control GPU L2 cacheability
- ▶ NvSciBufGeneralAttrKey_EnableGpuCompression: To specify GPU compression

Note

For more details on these attributes and their valid input options, refer to NvSciBuf Documentation.

The following code snippet illustrates their sample usage.

```
NvSciBufObj createNvSciBufObject() {
    // Raw Buffer Attributes for CUDA
    NvSciBufType bufType = NvSciBufType_RawBuffer;
    uint64_t rawsize = SIZE;
    uint64_t align = 0;
    bool cpuaccess_flag = true;
    NvSciBufAttrValAccessPerm perm = NvSciBufAccessPerm_ReadWrite;

    NvSciRmGpuId gpuid[] = {};
    CUuuid uuid;
    cuDeviceGetUuid(&uuid, dev);

    memcpy(&gpuid[0].bytes, &uuid.bytes, sizeof(uuid.bytes));
    // Disable cache on dev
    NvSciBufAttrValGpuCache gpuCache[] = {{gpuid[0], false}};
    NvSciBufAttrValGpuCompression gpuCompression[] = {{gpuid[0],
    ↪ NvSciBufCompressionType_GenericCompressible}};
    // Fill in values
    NvSciBufAttrKeyValuePair rawbuffattrs[] = {
```

(continues on next page)

(continued from previous page)

```

        { NvSciBufGeneralAttrKey_Types, &bufType, sizeof(bufType) },
        { NvSciBufRawBufferAttrKey_Size, &rawsize, sizeof(rawsize) },
        { NvSciBufRawBufferAttrKey_Align, &align, sizeof(align) },
        { NvSciBufGeneralAttrKey_NeedCpuAccess, &cpuaccess_flag,
→ sizeof(cpuaccess_flag) },
        { NvSciBufGeneralAttrKey_RequiredPerm, &perm, sizeof(perm) },
        { NvSciBufGeneralAttrKey_GpuId, &gpuid, sizeof(gpuid) },
        { NvSciBufGeneralAttrKey_EnableGpuCache, &gpuCache, sizeof(gpuCache)
→ },
        { NvSciBufGeneralAttrKey_EnableGpuCompression, &gpuCompression,
→ sizeof(gpuCompression) }
    };

    // Create list by setting attributes
    err = NvSciBufAttrListSetAttrs(attrListBuffer, rawbuffattrs,
        sizeof(rawbuffattrs)/sizeof(NvSciBufAttrKeyValuePair));

    NvSciBufAttrListCreate(NvSciBufModule, &attrListBuffer);

    // Reconcile And Allocate
    NvSciBufAttrListReconcile(&attrListBuffer, 1, &attrListReconciledBuffer,
        &attrListConflictBuffer)
    NvSciBufObjAlloc(attrListReconciledBuffer, &bufferObjRaw);
    return bufferObjRaw;
}

```

```

NvSciBufObj bufferObjRo; // Readonly NvSciBuf memory obj
// Create a duplicate handle to the same memory buffer with reduced permissions
NvSciBufObjDupWithReducePerm(bufferObjRaw, NvSciBufAccessPerm_Readonly, &
→ bufferObjRo);
return bufferObjRo;

```

The allocated NvSciBuf memory object can be imported in CUDA using the NvSciBufObj handle as shown below. Application should query the allocated NvSciBufObj for attributes required for filling CUDA External Memory Descriptor. Note that the attribute list and NvSciBuf objects should be maintained by the application. If the NvSciBuf object imported into CUDA is also mapped by other drivers, then based on NvSciBufGeneralAttrKey_GpuSwNeedCacheCoherency output attribute value the application must use NvSciSync objects (refer to [Importing Synchronization Objects](#)) as appropriate barriers to maintain coherence between CUDA and the other drivers.

Note

For more details on how to allocate and maintain NvSciBuf objects refer to [NvSciBuf API Documentation](#).

```

cudaExternalMemory_t importNvSciBufObject (NvSciBufObj bufferObjRaw) {

    /***** Query NvSciBuf Object *****/
    NvSciBufAttrKeyValuePair bufattrs[] = {
        { NvSciBufRawBufferAttrKey_Size, NULL, 0 },
        { NvSciBufGeneralAttrKey_GpuSwNeedCacheCoherency, NULL, 0 },

```

(continues on next page)

(continued from previous page)

```

        { NvSciBufGeneralAttrKey_EnableGpuCompression, NULL, 0 }
    };
    NvSciBufAttrListGetAttrs(retList, bufattrs,
        sizeof(bufattrs)/sizeof(NvSciBufAttrKeyValuePair));
    ret_size = *(static_cast<const uint64_t*>(bufattrs[0].value));

    // Note cache and compression are per GPU attributes, so read values for
    ↪ specific gpu by comparing UUID
    // Read cacheability granted by NvSciBuf
    int numGpus = bufattrs[1].len / sizeof(NvSciBufAttrValGpuCache);
    NvSciBufAttrValGpuCache[] cacheVal = (NvSciBufAttrValGpuCache
    ↪ *)bufattrs[1].value;
    bool ret_cacheVal;
    for (int i = 0; i < numGpus; i++) {
        if (memcmp(gpuid[0].bytes, cacheVal[i].gpuId.bytes, sizeof(CUuuid))
    ↪ == 0) {
            ret_cacheVal = cacheVal[i].cacheability);
        }
    }

    // Read compression granted by NvSciBuf
    numGpus = bufattrs[2].len / sizeof(NvSciBufAttrValGpuCompression);
    NvSciBufAttrValGpuCompression[] compVal = (NvSciBufAttrValGpuCompression
    ↪ *)bufattrs[2].value;
    NvSciBufCompressionType ret_compVal;
    for (int i = 0; i < numGpus; i++) {
        if (memcmp(gpuid[0].bytes, compVal[i].gpuId.bytes, sizeof(CUuuid)) ==
    ↪ 0) {
            ret_compVal = compVal[i].compressionType);
        }
    }

    /***** NvSciBuf Registration With CUDA *****/

    // Fill up CUDA_EXTERNAL_MEMORY_HANDLE_DESC
    cudaExternalMemoryHandleDesc memHandleDesc;
    memset(&memHandleDesc, 0, sizeof(memHandleDesc));
    memHandleDesc.type = cudaExternalMemoryHandleTypeNvSciBuf;
    memHandleDesc.handle.nvSciBufObject = bufferObjRaw;
    // Set the NvSciBuf object with required access permissions in this step
    memHandleDesc.handle.nvSciBufObject = bufferObjRo;
    memHandleDesc.size = ret_size;
    cudaImportExternalMemory(&extMemBuffer, &memHandleDesc);
    return extMemBuffer;
}

```

4.19.2.3.2 Mapping Buffers onto Imported Memory Objects

A device pointer can be mapped onto an imported memory object as shown below. The offset and size of the mapping can be filled as per the attributes of the allocated NvSciBufObj. All mapped device pointers must be freed using cudaFree().

```

void * mapBufferOntoExternalMemory(cudaExternalMemory_t extMem, unsigned long
↳ long offset, unsigned long long size) {
    void *ptr = NULL;
    cudaExternalMemoryBufferDesc desc = {};

    memset(&desc, 0, sizeof(desc));

    desc.offset = offset;
    desc.size = size;

    cudaExternalMemoryGetMappedBuffer(&ptr, extMem, &desc);

    // Note: 'ptr' must eventually be freed using cudaFree()
    return ptr;
}

```

4.19.2.3.3 Mapping Mipmapped Arrays onto Imported Memory Objects

A CUDA mipmapped array can be mapped onto an imported memory object as shown below. The offset, dimensions and format can be filled as per the attributes of the allocated `NvSciBufObj`. All mapped mipmapped arrays must be freed using `cudaFreeMipmappedArray()`. The following code sample shows how to convert `NvSciBuf` attributes into the corresponding CUDA parameters when mapping mipmapped arrays onto imported memory objects.

Note

The number of mip levels must be 1.

```

cudaMipmappedArray_t mapMipmappedArrayOntoExternalMemory(cudaExternalMemory_t
↳ extMem, unsigned long long offset, cudaChannelFormatDesc *formatDesc,
↳ cudaExtent *extent, unsigned int flags, unsigned int numLevels) {
    cudaMipmappedArray_t mipmap = NULL;
    cudaExternalMemoryMipmappedArrayDesc desc = {};

    memset(&desc, 0, sizeof(desc));

    desc.offset = offset;
    desc.formatDesc = *formatDesc;
    desc.extent = *extent;
    desc.flags = flags;
    desc.numLevels = numLevels;

    // Note: 'mipmap' must eventually be freed using cudaFreeMipmappedArray()
    cudaExternalMemoryGetMappedMipmappedArray(&mipmap, extMem, &desc);

    return mipmap;
}

```

4.19.2.3.4 Importing Synchronization Objects

NvSciSync attributes that are compatible with a given CUDA device can be generated using `cudaDeviceGetNvSciSyncAttributes()`. The returned attribute list can be used to create a `NvSciSyncObj` that is guaranteed compatibility with a given CUDA device.

```
NvSciSyncObj createNvSciSyncObject() {
    NvSciSyncObj nvSciSyncObj
    int cudaDev0 = 0;
    int cudaDev1 = 1;
    NvSciSyncAttrList signalerAttrList = NULL;
    NvSciSyncAttrList waiterAttrList = NULL;
    NvSciSyncAttrList reconciledList = NULL;
    NvSciSyncAttrList newConflictList = NULL;

    NvSciSyncAttrListCreate(module, &signalerAttrList);
    NvSciSyncAttrListCreate(module, &waiterAttrList);
    NvSciSyncAttrList unreconciledList[2] = {NULL, NULL};
    unreconciledList[0] = signalerAttrList;
    unreconciledList[1] = waiterAttrList;

    cudaDeviceGetNvSciSyncAttributes(signalerAttrList, cudaDev0, CUDA_
    ↪ NVSCISYNC_ATTR_SIGNAL);
    cudaDeviceGetNvSciSyncAttributes(waiterAttrList, cudaDev1, CUDA_NVSCISYNC_
    ↪ ATTR_WAIT);

    NvSciSyncAttrListReconcile(unreconciledList, 2, &reconciledList, &
    ↪ newConflictList);

    NvSciSyncObjAlloc(reconciledList, &nvSciSyncObj);

    return nvSciSyncObj;
}
```

An `NvSciSync` object (created as above) can be imported into CUDA using the `NvSciSyncObj` handle as shown below. Note that ownership of the `NvSciSyncObj` handle continues to lie with the application even after it is imported.

```
cudaExternalSemaphore_t importNvSciSyncObject(void* nvSciSyncObj) {
    cudaExternalSemaphore_t extSem = NULL;
    cudaExternalSemaphoreHandleDesc desc = {};

    memset(&desc, 0, sizeof(desc));

    desc.type = cudaExternalSemaphoreHandleTypeNvSciSync;
    desc.handle.nvSciSyncObj = nvSciSyncObj;

    cudaImportExternalSemaphore(&extSem, &desc);

    // Deleting/Freeing the nvSciSyncObj beyond this point will lead to
    ↪ undefined behavior in CUDA

    return extSem;
}
```

4.19.2.3.5 Signaling/Waiting on Imported Synchronization Objects

An imported `NvSciSyncObj` object can be signaled as outlined below. Signaling `NvSciSync` backed semaphore object initializes the `fence` parameter passed as input. This fence parameter is waited upon by a wait operation that corresponds to the aforementioned signal. Additionally, the wait that waits on this signal must be issued after this signal has been issued. If the flags are set to `cudaExternalSemaphoreSignalSkipNvSciBufMemSync` then memory synchronization operations (over all the imported `NvSciBuf` in this process) that are executed as a part of the signal operation by default are skipped. When `NvsciBufGeneralAttrKey_GpuSwNeedCacheCoherency` is `FALSE`, this flag should be set.

```
void signalExternalSemaphore(cudaExternalSemaphore_t extSem, cudaStream_t
→stream, void *fence) {
    cudaExternalSemaphoreSignalParams signalParams = {};

    memset(&signalParams, 0, sizeof(signalParams));

    signalParams.params.nvSciSync.fence = (void*)fence;
    signalParams.flags = 0; //OR cudaExternalSemaphoreSignalSkipNvSciBufMemSync

    cudaSignalExternalSemaphoresAsync(&extSem, &signalParams, 1, stream);
}
```

An imported `NvSciSyncObj` object can be waited upon as outlined below. Waiting on `NvSciSync` backed semaphore object waits until the input `fence` parameter is signaled by the corresponding signaler. Additionally, the signal must be issued before the wait can be issued. If the flags are set to `cudaExternalSemaphoreWaitSkipNvSciBufMemSync` then memory synchronization operations (over all the imported `NvSciBuf` in this process) that are executed as a part of the signal operation by default are skipped. When `NvsciBufGeneralAttrKey_GpuSwNeedCacheCoherency` is `FALSE`, this flag should be set.

```
void waitExternalSemaphore(cudaExternalSemaphore_t extSem, cudaStream_t
→stream, void *fence) {
    cudaExternalSemaphoreWaitParams waitParams = {};

    memset(&waitParams, 0, sizeof(waitParams));

    waitParams.params.nvSciSync.fence = (void*)fence;
    waitParams.flags = 0; //OR cudaExternalSemaphoreWaitSkipNvSciBufMemSync

    cudaWaitExternalSemaphoresAsync(&extSem, &waitParams, 1, stream);
}
```

4.20. Driver Entry Point Access

4.20.1. Introduction

The Driver Entry Point Access APIs provide a way to retrieve the address of a CUDA driver function. Starting from CUDA 11.3, users can call into available CUDA driver APIs using function pointers obtained from these APIs.

These APIs provide functionality similar to their counterparts, `dlsym` on POSIX platforms and `GetProcAddress` on Windows. The provided APIs will let users:

- ▶ Retrieve the address of a driver function using the CUDA Driver API.
- ▶ Retrieve the address of a driver function using the CUDA Runtime API.
- ▶ Request *per-thread default stream* version of a CUDA driver function. For more details, see [Retrieve Per-thread Default Stream Versions](#).
- ▶ Access new CUDA features on older toolkits but with a newer driver.

4.20.2. Driver Function Typedefs

To help retrieve the CUDA Driver API entry points, the CUDA Toolkit provides access to headers containing the function pointer definitions for all CUDA driver APIs. These headers are installed with the CUDA Toolkit and are made available in the toolkit's `include/` directory. The table below summarizes the header files containing the typedefs for each CUDA API header file.

Table 27: Typedefs header files for CUDA driver APIs

API header file	API Typedef header file
<code>cuda.h</code>	<code>cudaTypedefs.h</code>
<code>cudaGL.h</code>	<code>cudaGLTypedefs.h</code>
<code>cudaProfiler.h</code>	<code>cudaProfilerTypedefs.h</code>
<code>cudaVDPAU.h</code>	<code>cudaVDPAUTypedefs.h</code>
<code>cudaEGL.h</code>	<code>cudaEGLTypedefs.h</code>
<code>cudaD3D9.h</code>	<code>cudaD3D9Typedefs.h</code>
<code>cudaD3D10.h</code>	<code>cudaD3D10Typedefs.h</code>
<code>cudaD3D11.h</code>	<code>cudaD3D11Typedefs.h</code>

The above headers do not define actual function pointers themselves; they define the typedefs for function pointers. For example, `cudaTypedefs.h` has the below typedefs for the driver API `cuMemAlloc`:

```
typedef CUresult (CUDAAPI *PFN_cuMemAlloc_v3020)(CUdeviceptr_v2 *dptr, size_t
↪ bytesize);
typedef CUresult (CUDAAPI *PFN_cuMemAlloc_v2000)(CUdeviceptr_v1 *dptr,
↪ unsigned int bytesize);
```

CUDA driver symbols have a version based naming scheme with a `_v*` extension in its name except for the first version. When the signature or the semantics of a specific CUDA driver API changes, we increment the version number of the corresponding driver symbol. In the case of the `cuMemAlloc` driver API, the first driver symbol name is `cuMemAlloc` and the next symbol name is `cuMemAlloc_v2`. The typedef for the first version which was introduced in CUDA 2.0 (2000) is `PFN_cuMemAlloc_v2000`. The typedef for the next version which was introduced in CUDA 3.2 (3020) is `PFN_cuMemAlloc_v3020`.

The typedefs can be used to more easily define a function pointer of the appropriate type in code:

```
PFN_cuMemAlloc_v3020 pfn_cuMemAlloc_v2;
PFN_cuMemAlloc_v2000 pfn_cuMemAlloc_v1;
```

4.20.3. Driver Function Retrieval

Using the Driver Entry Point Access APIs and the appropriate typedef, we can get the function pointer to any CUDA driver API.

4.20.3.1 Using the Driver API

The driver API requires CUDA version as an argument to get the ABI compatible version for the requested driver symbol. CUDA Driver APIs have a per-function ABI denoted with a `_v*` extension. For example, consider the versions of `cuStreamBeginCapture` and their corresponding typedefs from `cudaTypedefs.h`:

```
// cuda.h
CUDAResult CUDAAPI cuStreamBeginCapture(CUstream hStream);
CUDAResult CUDAAPI cuStreamBeginCapture_v2(CUstream hStream,
    ↪ CUstreamCaptureMode mode);

// cudaTypedefs.h
typedef CUDAResult (CUDAAPI *PFN_cuStreamBeginCapture_v10000)(CUstream hStream);
typedef CUDAResult (CUDAAPI *PFN_cuStreamBeginCapture_v10010)(CUstream hStream,
    ↪ CUstreamCaptureMode mode);
```

From the above typedefs in the code snippet, version suffixes `_v10000` and `_v10010` indicate that the above APIs were introduced in CUDA 10.0 and CUDA 10.1 respectively.

```
#include <cudaTypedefs.h>

// Declare the entry points for cuStreamBeginCapture
PFN_cuStreamBeginCapture_v10000 pfn_cuStreamBeginCapture_v1;
PFN_cuStreamBeginCapture_v10010 pfn_cuStreamBeginCapture_v2;

// Get the function pointer to the cuStreamBeginCapture driver symbol
cuGetProcAddress("cuStreamBeginCapture", &pfn_cuStreamBeginCapture_v1, 10000,
    ↪ CU_GET_PROC_ADDRESS_DEFAULT, &driverStatus);
// Get the function pointer to the cuStreamBeginCapture_v2 driver symbol
cuGetProcAddress("cuStreamBeginCapture", &pfn_cuStreamBeginCapture_v2, 10010,
    ↪ CU_GET_PROC_ADDRESS_DEFAULT, &driverStatus);
```

Referring to the code snippet above, to retrieve the address to the `_v1` version of the driver API `cuStreamBeginCapture`, the CUDA version argument should be exactly 10.0 (10000). Similarly, the CUDA version for retrieving the address to the `_v2` version of the API should be 10.1 (10010). Specifying a higher CUDA version for retrieving a specific version of a driver API might not always be portable. For example, using 11030 here would still return the `_v2` symbol, but if a hypothetical `_v3` version is released in CUDA 11.3, the `cuGetProcAddress` API would start returning the newer `_v3` symbol instead when paired with a CUDA 11.3 driver. Since the ABI and function signatures of the `_v2` and `_v3` symbols might differ, calling the `_v3` function using the `_v10010` typedef intended for the `_v2` symbol would exhibit undefined behavior.

Note that requesting a driver API with an invalid CUDA version will return an error `CUDA_ERROR_NOT_FOUND`. In the above code examples, passing in a version less than 10000

(CUDA 10.0) would be invalid.

4.20.3.2 Using the Runtime API

The runtime API `cudaGetDriverEntryPointByVersion` uses the provided CUDA version to get the ABI compatible version for the requested driver symbol in the same way `cuGetProcAddress` does. In the below code snippet, the minimum CUDA version required would be CUDA 11.2 as `cuMemAllocAsync` was introduced then.

```
#include <cudaTypedefs.h>

int cudaVersion;
// Ensure a CUDA driver >= 11.2 is installed or we will get an error from
// cuGetProcAddress
status = cuDriverGetVersion(&cudaVersion);
if (cudaVersion >= 11020) {

    // Declare the entry point
    PFN_cuMemAllocAsync_v11020 pfn_cuMemAllocAsync;

    // Initialize the entry point
    cudaGetDriverEntryPointByVersion("cuMemAllocAsync", &pfn_cuMemAllocAsync,
    // 11020, cudaEnableDefault, &driverStatus);

    // Call the entry point
    if(driverStatus == cudaDriverEntryPointSuccess && pfn_cuMemAllocAsync) {
        pfn_cuMemAllocAsync(...);
    }
}
```

4.20.3.3 Retrieve Per-thread Default Stream Versions

Some CUDA driver APIs can be configured to have *default stream* or *per-thread default stream* semantics. Driver APIs having *per-thread default stream* semantics are suffixed with `_ptsz` or `_ptds` in their name. For example, `cuLaunchKernel` has a *per-thread default stream* variant named `cuLaunchKernel_ptsz`. With the Driver Entry Point Access APIs, users can request for the *per-thread default stream* version of the driver API `cuLaunchKernel` instead of the *default stream* version. Configuring the CUDA driver APIs for *default stream* or *per-thread default stream* semantics affects the synchronization behavior. More details can be found [here](#).

The *default stream* or *per-thread default stream* versions of a driver API can be obtained by one of the following ways:

- ▶ Use the compilation flag `--default-stream per-thread` or define the macro `CUDA_API_PER_THREAD_DEFAULT_STREAM` to get *per-thread default stream* behavior.
- ▶ Force *default stream* or *per-thread default stream* behavior using the flags `CU_GET_PROC_ADDRESS_LEGACY_STREAM/cudaEnableLegacyStream` or `CU_GET_PROC_ADDRESS_PER_THREAD_DEFAULT_STREAM/cudaEnablePerThreadDefaultStream` respectively.

4.20.3.4 Access New CUDA features

It is always recommended to install the latest CUDA toolkit to access new CUDA driver features, but if for some reason, a user does not want to update or does not have access to the latest toolkit, the API can be used to access new CUDA features with only an updated CUDA driver. For discussion, let us assume the user is on CUDA 12.3 and wants to use a new driver API `cuFoo` available in the CUDA 12.5 driver. The below code snippet illustrates this use-case:

```
int main()
{
    // Manually define the prototype as cudaTypedefs.h in CUDA 12.3 does not
    ↪ have the cuFoo typedef
    typedef CUresult (CUDAAPI *PFN_cuFoo_v12050)(...);
    PFN_cuFoo_v12050 pfn_cuFoo = NULL;
    CUdriverProcAddressQueryResult driverStatus;
    int cudaVersion;

    // Ensure a CUDA driver >= 12.5 is installed or we will get an error from
    ↪ cuGetProcAddress
    CUresult status = cuDriverGetVersion(&cudaVersion);
    if (cudaVersion >= 12050) {
        // Get the address for cuFoo API using cuGetProcAddress. Specify CUDA
        ↪ version as
        // 12050 since cuFoo was introduced then
        CUresult status = cuGetProcAddress("cuFoo", &pfn_cuFoo, 12050, CU_GET_
        ↪ PROC_ADDRESS_DEFAULT, &driverStatus);

        if (status == CUDA_SUCCESS && pfn_cuFoo) {
            pfn_cuFoo(...);
        }
        else {
            printf("Cannot retrieve the address to cuFoo - driverStatus = %d\n
            ↪ ", driverStatus);
            assert(0);
        }
    }

    // rest of code here
}
```

In the next example, we discuss how to get a new version of an API released in a minor version of the CUDA Toolkit. Note that in the `cuda.h` header the version macro that would bump `cuDeviceGetUuid` to `_v2` is not done until a major boundary. So during the 11.4+ releases the following example illustrates how to get the `_v2` version.

Note in this case the original (not the `_v2` version) typedef looks like:

```
typedef CUresult (CUDAAPI *PFN_cuDeviceGetUuid_v9020)(CUuuid *uuid, CUdevice_
    ↪ v1 dev);
```

But the `_v2` version typedef looks like:

```
typedef CUresult (CUDAAPI *PFN_cuDeviceGetUuid_v11040)(CUuuid *uuid, CUdevice_
    ↪ v1 dev);
```



```

#include <cudaTypedefs.h>

CUuuid uuid;
CUdevice dev;
CUresult status;
int cudaVersion;
CUdriverProcAddressQueryResult driverStatus;

status = cuDeviceGet(&dev, 0); // Get device 0
// handle status

// Ensure a CUDA driver >= 11.4 is installed or we will get an error from
// cuGetProcAddress
status = cuDriverGetVersion(&cudaVersion);
if (cudaVersion >= 11040) {
    PFN_cuDeviceGetUuid_v11040 pfn_cuDeviceGetUuid;
    status = cuGetProcAddress("cuDeviceGetUuid", &pfn_cuDeviceGetUuid, 11040,
    CU_GET_PROC_ADDRESS_DEFAULT, &driverStatus);
    if (CUDA_SUCCESS == status && pfn_cuDeviceGetUuid) {
        pfn_cuDeviceGetUuid(&uuid, dev);
    }
}

```

4.20.4. Guidelines for cuGetProcAddress

Below are guidelines to keep in mind when using cuGetProcAddress.

- ▶ Code the CUDA version passed to cuGetProcAddress to match the typedef version (do not use a compile time constant such as CUDA_VERSION or a dynamic version such as returned from cuDriverGetVersion)
- ▶ Check the current driver version (such as from cuDriverGetVersion) is sufficient before calling cuGetProcAddress or an error is expected or an unexpected symbol may be returned

4.20.4.1 Guidelines for Runtime API Usage

Unless specified otherwise, the CUDA runtime API cudaGetDriverEntryPointByVersion will have similar guidelines as the driver entry point cuGetProcAddress since it allows for the user to request a specific CUDA driver version.

4.20.5. Determining cuGetProcAddress Failure Reasons

There are two types of errors with cuGetProcAddress. Those are (1) API/usage errors and (2) inability to find the driver API requested. The first error type will return error codes from the API via the CUresult return value. Things like passing NULL as the pfn variable or passing invalid flags.

The second error type encodes in the CUdriverProcAddressQueryResult *symbolStatus and can be used to help distinguish potential issues with the driver not being able to find the symbol requested. Take the following example:

```

// cuDeviceGetExecAffinitySupport was introduced in release CUDA 11.4
#include <cuda.h>

```

(continues on next page)

(continued from previous page)

```

CUdriverProcAddressQueryResult driverStatus;
cudaVersion = ...;
status = cuGetProcAddress("cuDeviceGetExecAffinitySupport", &pfn, cudaVersion,
→ 0, &driverStatus);
if (CUDA_SUCCESS == status) {
    if (CU_GET_PROC_ADDRESS_VERSION_NOT_SUFFICIENT == driverStatus) {
        printf("We can use the new feature when you upgrade cudaVersion to 11.
→4, but CUDA driver is good to go!\n");
        // Indicating cudaVersion was < 11.4 but run against a CUDA driver >=
→11.4
    }
    else if (CU_GET_PROC_ADDRESS_SYMBOL_NOT_FOUND == driverStatus) {
        printf("Please update both CUDA driver and cudaVersion to at least 11.
→4 to use the new feature!\n");
        // Indicating driver is < 11.4 since string not found, doesn't matter
→what cudaVersion was
    }
    else if (CU_GET_PROC_ADDRESS_SUCCESS == driverStatus && pfn) {
        printf("You're using cudaVersion and CUDA driver >= 11.4, using new
→feature!\n");
        pfn();
    }
}

```

The first case with the return code `CU_GET_PROC_ADDRESS_VERSION_NOT_SUFFICIENT` indicates that the symbol was found when searching in the CUDA driver but it was added later than the `cudaVersion` supplied. In the example, specifying `cudaVersion` as anything 11030 or less and when running against a CUDA driver $\geq$ CUDA 11.4 would give this result of `CU_GET_PROC_ADDRESS_VERSION_NOT_SUFFICIENT`. This is because `cuDeviceGetExecAffinitySupport` was added in CUDA 11.4 (11040).

The second case with the return code `CU_GET_PROC_ADDRESS_SYMBOL_NOT_FOUND` indicates that the symbol was not found when searching in the CUDA driver. This can be due to a few reasons such as unsupported CUDA function due to older driver as well as just having a typo. In the latter, similar to the last example if the user had put `symbol` as `CUDeviceGetExecAffinitySupport` - notice the capital CU to start the string - `cuGetProcAddress` would not be able to find the API because the string doesn't match. In the former case an example might be the user developing an application against a CUDA driver supporting the new API, and deploying the application against an older CUDA driver. Using the last example, if the developer developed against CUDA 11.4 or later but was deployed against a CUDA 11.3 driver, during their development they may have had a successful `cuGetProcAddress`, but when deploying an application running against a CUDA 11.3 driver the call would no longer work with the `CU_GET_PROC_ADDRESS_SYMBOL_NOT_FOUND` returned in `driverStatus`.

Chapter 5. Technical Appendices

5.1. Compute Capabilities

The general specifications and features of a compute device depend on its compute capability (see *Compute Capability and Streaming Multiprocessor Versions*).

Table 29, Table 30, and Table 31 show the features and technical specifications associated with each compute capability that is currently supported.

All NVIDIA GPU architectures use a little-endian representation.

5.1.1. Obtain the GPU Compute Capability

The *CUDA GPU Compute Capability* page provides a comprehensive mapping from NVIDIA GPU models to their compute capability.

Alternatively, the *nvidia-smi* tool, provided with the *NVIDIA Driver*, can be used to get the compute capability of a GPU. For example, the following command will output the GPU names and compute capabilities available on the system:

```
nvidia-smi --query-gpu=name,compute_cap
```

At runtime, the compute capability can be obtained using the CUDA Runtime API *cudaDeviceGetAttribute()*, CUDA Driver API *cuDeviceGetAttribute()*, or NVML API *nvmlDeviceGetCudaComputeCapability()*:

```
#include <cuda_runtime_api.h>
```

```
int computeCapabilityMajor, computeCapabilityMinor;
cudaDeviceGetAttribute(&computeCapabilityMajor,
    ↪ cudaDevAttrComputeCapabilityMajor, device_id);
cudaDeviceGetAttribute(&computeCapabilityMinor,
    ↪ cudaDevAttrComputeCapabilityMinor, device_id);
```

```
#include <cuda.h>
```

```
int computeCapabilityMajor, computeCapabilityMinor;
cuDeviceGetAttribute(&computeCapabilityMajor, CU_DEVICE_ATTRIBUTE_COMPUTE_
    ↪ CAPABILITY_MAJOR, device_id);
cuDeviceGetAttribute(&computeCapabilityMinor, CU_DEVICE_ATTRIBUTE_COMPUTE_
    ↪ CAPABILITY_MINOR, device_id);
```

```
#include <nvml.h> // required linking with -lnvidia-ml

int computeCapabilityMajor, computeCapabilityMinor;
nvmlDeviceGetCudaComputeCapability(nvmlDevice, &computeCapabilityMajor, &
    ↪ computeCapabilityMinor);
```

5.1.2. Feature Availability

Most compute features introduced with a compute architecture are intended to be available on all subsequent architectures. This is shown in [Table 29](#) by the “yes” for availability of a feature on compute capabilities subsequent to its introduction.

5.1.2.1 Architecture-Specific Features

Beginning with devices of Compute Capability 9.0, specialized compute features that are introduced with an architecture may not be guaranteed to be available on all subsequent compute capabilities. These features are called *architecture-specific* features and target acceleration of specialized operations, such as Tensor Core operations, which are not intended for all classes of compute capabilities or may significantly change in future generations. Code must be compiled with an architecture-specific compiler target (see [Feature Set Compiler Targets](#)) to enable architecture-specific features. Code compiled with an architecture-specific compiler target can only be run on the exact compute capability it was compiled for.

5.1.2.2 Family-Specific Features

Beginning with devices of Compute Capability 10.0, some architecture-specific features are common to devices of more than one compute capability. The devices that contain these features are part of the same family and these features can also be called *family-specific* features. Family-specific features are guaranteed to be available on all devices in the same family. A family-specific compiler target is required to enable family-specific features. See [Section 5.1.2.3](#). Code compiled for a family-specific target can only be run on GPUs which are members of that family.

5.1.2.3 Feature Set Compiler Targets

There are three sets of compute features which the compiler can target:

Baseline Feature Set: The predominant set of compute features that are introduced with the intent to be available for subsequent compute architectures. These features and their availability are summarized in [Table 29](#).

Architecture-Specific Feature Set: A small and highly specialized set of features called architecture-specific, that are introduced to accelerate specialized operations, which are not guaranteed to be available or might change significantly on subsequent compute architectures. These features are summarized in the respective “Compute Capability #.#” subsections. The architecture-specific feature set is a superset of the family-specific feature set. Architecture-specific compiler targets were introduced with Compute Capability 9.0 devices and are selected by using an **a** suffix in the compilation target, for example by specifying `compute_100a` or `compute_120a` as the compute target.

Family-Specific Feature Set: Some architecture-specific features are common to GPUs of more than one compute capability. These features are summarized in the respective “Compute Capability #.#” subsections. With a few exceptions, later-generation devices with the same major compute capability are in the same family. [Table 28](#) indicates the compatibility of family-specific targets with device compute capability, including exceptions. The family-specific feature set is a superset of the baseline feature set. Family-specific compiler targets were introduced with Compute Capability 10.0 devices and

are selected by using an **f** suffix in the compilation target, for example by specifying `compute_100f` or `compute_120f` as the compute target.

All devices starting from compute capability 9.0 have a set of features that are architecture-specific. To utilize the complete set of these features on a specific GPU, the architecture-specific compiler target with the suffix **a** must be used. Additionally, starting from compute capability 10.0, there are sets of features that appear in multiple devices with different minor compute capabilities. These sets of instructions are called family-specific features, and the devices which share these features are said to be part of the same family. The family-specific features are a subset of the architecture-specific features that are shared by all members of that GPU family. The family-specific compiler target with the suffix **f** allows the compiler to generate code that uses this common subset of architecture-specific features.

For example:

- ▶ The `compute_100` compilation target does not allow the use of architecture-specific features. This target will be compatible with all devices of compute capability 10.0 and later.
- ▶ The `compute_100f` *family-specific* compilation target allows the use of the subset of architecture-specific features that are common across the GPU family. This target will only be compatible with devices that are part of the GPU family. In this example, it is compatible with devices of Compute Capability 10.0 and Compute Capability 10.3. The features available in the family-specific `compute_100f` target are a superset of the features available in the baseline `compute_100` target.
- ▶ The `compute_100a` *architecture-specific* compilation target allows the use of the complete set of architecture-specific features in Compute Capability 10.0 devices. This target will only be compatible with devices of Compute Capability 10.0 and no others. The features available in the `compute_100a` target form a superset of the features available in the `compute_100f` target.

Table 28: Family-Specific Compatibility

Compilation Target	Compatible with Compute Capability	
<code>compute_100f</code>	10.0	10.3
<code>compute_103f</code>	10.3 ¹	
<code>compute_110f</code>	11.0 ¹	
<code>compute_120f</code>	12.0	12.1
<code>compute_121f</code>	12.1 ¹	

5.1.3. Features and Technical Specifications

¹ Some families only contain a single member when they are created. They may be expanded in the future to include more devices.

Table 29: Feature Support per Compute Capability

Feature Support	Compute Capability					
	7.x	8.x	9.0	10.x	11.0	12.x
(Unlisted features are supported for all compute capabilities)						
Atomic functions operating on 128-bit integer values in shared and global memory (<i>Atomic Functions</i>)	No		Yes			
Atomic addition operating on float2 and float4 floating point vectors in global memory (<i>atomicAdd()</i>)	No		Yes			
Warp reduce functions (<i>Warp Reduce Functions</i>)	No	Yes				
Bfloat16-precision floating-point operations	No	Yes				

continues on next page

Table 29 – continued from previous page

Feature Support	Compute Capability			
	1.0	2.0	3.0	3.5
128-bit-precision floating-point operations	No			Yes
Hardware-accelerated mem-cpy_async (<i>Pipelines</i>)	No	Yes		
Hardware-accelerated Split Arrive/Wait Barrier (<i>Asynchronous Barriers</i>)	No	Yes		
L2 Cache Residency Management (<i>L2 Cache Control</i>)	No	Yes		
DPX Instructions for Accelerated Dynamic Programming (<i>Dynamic Programming eXtension (DPX) Instructions</i>)	Multiple Instr.		Native	Multiple Instr.
Distributed Shared Memory	No		Yes	

continues on next page

Table 29 – continued from previous page

Feature Support	Compute Capability	
Thread Block Cluster (<i>Thread Block Clusters</i>)	No	Yes
Tensor Memory Accelerator (TMA) unit (<i>Using the Tensor Memory Accelerator (TMA)</i>)	No	Yes

Note that the KB and K units used in the following tables correspond to 1024 bytes (i.e., a KiB) and 1024 respectively.

Table 30: Device and Streaming Multiprocessor (SM) Information per Compute Capability

	Compute Capability										
	7.5	8.0	8.6	8.7	8.9	9.0	10.0	10.3	11.0	12.x	
Ratio of FP32 to FP64 Throughput ²	32:1	2:1	64:1			2:1		64:1			
Maximum number of resident grids per device (Concurrent Kernel Execution)	128										
Maximum dimensionality of a grid	3										
Maximum x-dimension of a grid	2 ³¹ -1										
Maximum y- or z-dimension of a grid	65535										
Maximum dimensionality of a thread block	3										
Maximum x- or y-dimensionality of a thread block	1024										
Maximum z-dimension of a thread block	64										
Maximum number of threads per block	1024										
Warp size	32										
Maximum number of resident blocks per SM	16	32	16		24	32			24		

continues on next page

Table 30 – continued from previous page

Compute Capability					
Maximum number of resident warps per SM	32	64	48	64	48
Maximum number of resident threads per SM	1024	2048	1536	2048	1536
Green contexts: minimum SM partition size for useFlags 0	2	4		8	
Green contexts: SM co-scheduled alignment per partition for useFlags 0	2			8	

Table 31: Memory Information per Compute Capability

Compute Capability										
	7.5	8.0	8.6	8.7	8.9	9.0	10.x	11.0	12.x	
Number of 32-bit registers per SM	64 K									
Maximum number of 32-bit registers per thread block	64 K									
Maximum number of 32-bit registers per thread	255									
Maximum amount of shared memory per SM	64 KB	164 KB	100 KB	164 KB	100 KB	228 KB			100 KB	
Maximum amount of shared memory per thread block ³	64 KB	163 KB	99 KB	163 KB	99 KB	227 KB			99 KB	
Number of shared memory banks	32									
Maximum amount of local memory per thread	512 KB									
Constant memory size	64 KB									
Cache working set per SM for constant memory	8 KB									
Cache working set per SM for texture memory	32 or 64 KB	28 KB ~ 192 KB	28 KB ~ 128 KB	28 KB ~ 192 KB	28 KB ~ 128 KB	28 KB ~ 128 KB	28 KB ~ 256 KB		28 KB ~ 128 KB	

² Non-Tensor Core throughputs. For more information on throughput see the [CUDA Best Practices Guide](#)

³ Kernels relying on shared memory allocations over 48 KB per block must use dynamic shared memory and require an explicit opt-in, see [Configuring L1/Shared Memory Balance](#).

Table 32: Shared Memory Capacity per Compute Capability

Compute Capability	Unified Data Cache Size (KB)	SMEM Capacity Sizes (KB)
7.5	96	32, 64
8.0	192	0, 8, 16, 32, 64, 100, 132, 164
8.6	128	0, 8, 16, 32, 64, 100
8.7	192	0, 8, 16, 32, 64, 100, 132, 164
8.9	128	0, 8, 16, 32, 64, 100
9.0	256	0, 8, 16, 32, 64, 100, 132, 164, 196, 228
10.x	256	0, 8, 16, 32, 64, 100, 132, 164, 196, 228
11.0	256	0, 8, 16, 32, 64, 100, 132, 164, 196, 228
12.x	128	0, 8, 16, 32, 64, 100

Table 33 shows the input data types supported by Tensor Core acceleration. The Tensor Core feature set is available within the CUDA compilation toolchain through inline PTX. It is strongly recommended that applications use this feature set through CUDA-X libraries such as cuDNN, cuBLAS, and cuFFT, for example, or through CUTLASS, a collection of CUDA C++ template abstractions and Python domain-specific languages (DSLs) designed to enable high-performance matrix-matrix multiplication (GEMM) and related computations across all levels within CUDA.

Table 33: Input Data Types Supported by Tensor Core Acceleration per Compute Capability

Compute Capability	Tensor Core Input Data Types								
	FP64	TF32	BF16	FP16	FP8	FP6	FP4	INT8	INT4
7.5				Yes				Yes	Yes
8.0	Yes	Yes	Yes	Yes				Yes	Yes
8.6		Yes	Yes	Yes				Yes	Yes
8.7		Yes	Yes	Yes				Yes	Yes
8.9		Yes	Yes	Yes	Yes			Yes	Yes
9.0	Yes	Yes	Yes	Yes	Yes			Yes	
10.0	Yes	Yes	Yes	Yes	Yes	Yes	Yes	Yes	
10.3		Yes	Yes	Yes	Yes	Yes	Yes	Yes	
11.0		Yes	Yes	Yes	Yes	Yes	Yes	Yes	
12.x		Yes	Yes	Yes	Yes	Yes	Yes	Yes	

5.2. CUDA Environment Variables

The following section lists the CUDA environment variables. Those related to the Multi-Process Service (MPS) are documented in the [GPU Deployment and Management Guide](#).

5.2.1. Device Enumeration and Properties

5.2.1.1 CUDA_VISIBLE_DEVICES

The environment variable controls which GPU devices are visible to a CUDA application and in what order they are enumerated.

- ▶ If the variable is not set, all GPU devices are visible.
- ▶ If the variable is set to an empty string, no GPU devices are visible.

Possible Values: A comma-separated sequence of GPU identifiers.

GPU identifiers are provided as:

- ▶ **Integer indices:** These correspond to the ordinal number of the GPU in the system, as determined by `nvidia-smi`, starting from 0. For example, setting `CUDA_VISIBLE_DEVICES=2,1` makes device 0 not visible and enumerates device 2 before device 1.
 - ▶ If an invalid index is encountered, only devices with indices that appear before the invalid index in the list are visible. For example, setting `CUDA_VISIBLE_DEVICES=0,2,-1,1` makes devices 0 and 2 visible, while device 1 is not visible because it appears after the invalid index -1.
- ▶ **GPU UUID strings:** These should follow the same format as given by `nvidia-smi -L`, such as `GPU-8932f937-d72c-4106-c12f-20bd9faed9f6`. However, for convenience, abbreviated forms are allowed; simply specify enough digits from the beginning of the GPU UUID to uniquely identify that GPU in the target system. For example, `CUDA_VISIBLE_DEVICES=GPU-8932f937` may be a valid way to refer to the above GPU UUID, assuming no other GPU in the system shares this prefix.
- ▶ **Multi-Instance GPU (MIG) support:** `MIG-<GPU-UUID>/<GPU instance ID>/<compute instance ID>`. For example, `MIG-GPU-8932f937-d72c-4106-c12f-20bd9faed9f6/1/2`. Only single MIG instance enumeration is supported.

The device count returned by the `cudaGetDeviceCount()` API includes only the visible devices, so CUDA APIs that use integer device identifiers only support ordinals in the range `[0, visible device count - 1]`. The enumeration order of the GPU devices determines the ordinal values. For example, with `CUDA_VISIBLE_DEVICES=2,1`, calling `cudaSetDevice(0)` will set device 2 as the current device, as it is enumerated first and assigned an ordinal of 0. Calling `cudaGetDevice(&device_ordinal)` after that will also set `device_ordinal` to 0, which corresponds to device 2.

Examples:

```
nvidia-smi -L # Get list of GPU UUIDs
CUDA_VISIBLE_DEVICES=0,1
CUDA_VISIBLE_DEVICES=GPU-8932f937-d72c-4106-c12f-20bd9faed9f6
CUDA_VISIBLE_DEVICES=MIG-GPU-8932f937-d72c-4106-c12f-20bd9faed9f6/1/2
```

5.2.1.2 CUDA_DEVICE_ORDER

The environment variable controls the order in which CUDA enumerates the available devices.

Possible Values:

- ▶ **FASTEST_FIRST:** The available devices are enumerated from fastest to slowest using a simple heuristic (default).
- ▶ **PCI_BUS_ID:** The available devices are enumerated by PCI bus ID in ascending order. The PCI bus IDs can be obtained with `nvidia-smi --query-gpu=name,pci.bus_id`.

Examples:

```
CUDA_DEVICE_ORDER=FASTEST_FIRST
CUDA_DEVICE_ORDER=PCI_BUS_ID
nvidia-smi --query-gpu=name,pci.bus_id # Get list of PCI bus IDs
```

5.2.1.3 CUDA_MANAGED_FORCE_DEVICE_ALLOC

The environment variable alters how *Unified Memory* is physically stored in multi-GPU systems.

Possible Values: Numerical value, either zero or non-zero.

- ▶ **Non-zero value:** Forces the driver to use device memory for physical storage. All devices used in the process that support managed memory must be peer-to-peer compatible. Otherwise, `cudaErrorInvalidDevice` is returned.
- ▶ **0:** Default behavior.

Examples:

```
CUDA_MANAGED_FORCE_DEVICE_ALLOC=0
CUDA_MANAGED_FORCE_DEVICE_ALLOC=1 # force device memory
```

5.2.2. JIT Compilation

5.2.2.1 CUDA_CACHE_DISABLE

The environment variable controls the behavior of the on-disk *Just-In-Time (JIT) compilation* cache. Disabling the JIT cache forces PTX to CUBIN compilation for a CUDA application each time it is executed, unless the CUBIN code for the running architecture is found in the binary.

Disabling the JIT cache increases an application's load time during initial execution. However, it can be useful for reducing the application's disk space and for diagnosing differences across driver versions or build flags.

Possible Values:

- ▶ **1:** Disables PTX JIT caching.
- ▶ **0:** Enables PTX JIT caching (default).

Examples:

```
CUDA_CACHE_DISABLE=1 # disables caching
CUDA_CACHE_DISABLE=0 # enables caching
```

5.2.2.2 CUDA_CACHE_PATH

The environment variable specifies the directory path for the *Just-In-Time (JIT) compilation* cache.

Possible Values: The absolute path to the cache directory (with appropriate access permissions). The default values are:

- ▶ on Windows, %APPDATA%\NVIDIA\ComputeCache
- ▶ on Linux, ~/.nv/ComputeCache

Example:

```
CUDA_CACHE_PATH=~/.tmp
```

5.2.2.3 CUDA_CACHE_MAXSIZE

The environment variable specifies the *Just-In-Time (JIT) compilation* cache size in bytes. Binaries that exceed this size are not cached. If needed, older binaries are evicted from the cache to make room for newer ones.

Possible Values: Number of bytes. The default values are:

- ▶ On desktop/server platforms, 1073741824 (1 GiB)
- ▶ On embedded platforms, 268435456 (256 MiB)

4294967296 (4 GiB) is the maximum size.

Example:

```
CUDA_CACHE_MAXSIZE=268435456 # 256 MiB
```

5.2.2.4 CUDA_FORCE_PTX_JIT and CUDA_FORCE_JIT

The environment variables instruct the CUDA driver to ignore any CUBIN embedded in an application and perform *Just-In-Time (JIT) compilation* of the embedded PTX code instead.

Forcing JIT compilation increases an application's load time during initial execution. However, it can be used to validate that PTX code is embedded in an application and that its Just-In-Time compilation is functioning properly. This ensures *forward compatibility* with future architectures.

CUDA_FORCE_PTX_JIT overrides CUDA_FORCE_JIT.

Possible Values:

- ▶ 1: Forces PTX JIT compilation.
- ▶ 0: Default behavior.

Example:

```
CUDA_FORCE_PTX_JIT=1
```

5.2.2.5 CUDA_DISABLE_PTX_JIT and CUDA_DISABLE_JIT

The environment variables disable the *Just-In-Time (JIT) compilation* of embedded PTX code and use the compatible CUBIN embedded in an application.

A kernel will fail to load if it does not have embedded binary code, or if the embedded binary was compiled for an incompatible architecture. These environment variables can be used to validate that an application has compatible CUBIN code generated for each kernel. See the *Binary Compatibility* section for more details.

CUDA_DISABLE_PTX_JIT overrides CUDA_DISABLE_JIT.

Possible Values:

- ▶ 1: Disables PTX JIT compilation.
- ▶ 0: Default behavior.

Example:

```
CUDA_DISABLE_PTX_JIT=1
```

5.2.2.6 CUDA_FORCE_PRELOAD_LIBRARIES

The environment variable affects the preloading of libraries required for NVVM and *Just-In-Time (JIT) compilation*.

Possible Values:

- ▶ 1: This forces the driver to preload the libraries required for NVVM and *Just-In-Time (JIT) compilation* during initialization. This increases the memory footprint and the time required for CUDA driver initialization. Setting this environment variable is necessary to avoid certain deadlock situations involving multiple threads.
- ▶ 0: Default behavior.

Example:

```
CUDA_FORCE_PRELOAD_LIBRARIES=1
```

5.2.3. Execution

5.2.3.1 CUDA_LAUNCH_BLOCKING

The environment variable specifies whether to disable or enable asynchronous kernel launches.

Disabling asynchronous execution results in slower execution but is useful for debugging. It forces GPU work to run synchronously from the CPU's perspective. This allows CUDA API errors to be observed at the exact API call that triggered them, rather than later in the execution. Synchronous execution is useful for debugging purposes.

Possible Values:

- 1: Disables asynchronous execution.
- 0: Asynchronous execution (default).

Example:

```
CUDA_LAUNCH_BLOCKING=1
```

5.2.3.2 CUDA_DEVICE_MAX_CONNECTIONS

The environment variable controls the number of concurrent compute and copy engine connections (work queues), setting both to the specified value. If independent GPU tasks, namely kernels or copy operations launched from different CUDA streams, map to the same work queue, a false dependency is created which can lead to GPU work serialization, since the same underlying resource(s) are used. To reduce the probability of such false dependencies, it is recommended that the work queue count, controlled via this environment variable, be greater than or equal to the number of active CUDA streams per context.

Setting this environment variable also modifies the number of copy connections, unless they are explicitly set via the `CUDA_DEVICE_MAX_COPY_CONNECTIONS` environment variable.

Possible Values: 1 to 32 connections, default is 8 (assumes no MPS)

Example:

```
CUDA_DEVICE_MAX_CONNECTIONS=16
```

5.2.3.3 CUDA_DEVICE_MAX_COPY_CONNECTIONS

The environment variable controls the number of concurrent copy connections (work queues) involved in copy operations. It affects only devices of *compute capability* 8.0 and above.

The `CUDA_DEVICE_MAX_COPY_CONNECTIONS` overrides the value of copy connections set via `CUDA_DEVICE_MAX_CONNECTIONS`, if both were set.

Possible Values: 1 to 32 connections, default is 8 (assumes no MPS)

Example:

```
CUDA_DEVICE_MAX_COPY_CONNECTIONS=16
```

5.2.3.4 CUDA_SCALE_LAUNCH_QUEUES

The environment variable specifies the scaling factor for the size of the queues available for launching work (command buffer), namely the total number of pending kernels or host/device copy operations that can be enqueued on a device.

Possible Values: 0.25x, 0.5x, 2x, 4x

- Any value other than 0.25x, 0.5x, 2x or 4x is interpreted as 1x.

Example:

```
CUDA_SCALE_LAUNCH_QUEUES=2x
```

5.2.3.5 CUDA_GRAPH_USE_NODE_PRIORITY

The environment variable controls the CUDA graph's execution priority relative to the stream priority it inherits from the stream in which it is launched.

CUDA_GRAPH_USE_NODE_PRIORITY overrides the `cudaGraphInstantiateFlagUseNodePriority` flag on graph instantiation.

Possible Values:

- ▶ 0: Inherit the priority of the stream the graph is launched into (default).
- ▶ 1: Honor per-node launch priorities. The CUDA runtime treats node-level priorities as a scheduling hint for ready-to-run graph nodes.

Example:

```
CUDA_GRAPH_USE_NODE_PRIORITY=1
```

5.2.3.6 CUDA_DEVICE_WAITS_ON_EXCEPTION

The environment variable controls the behavior of a CUDA application when an exception (error) occurs.

When enabled, a CUDA application will halt and wait when a device-side exception occurs, allowing a debugger, such as `cuda-gdb`, to be attached to inspect the live GPU state before the process exits or continues.

Possible Values:

- ▶ 0: Default behavior.
- ▶ 1: Halt when a device exception occurs.

Example:

```
CUDA_DEVICE_WAITS_ON_EXCEPTION=1
```

5.2.3.7 CUDA_DEVICE_DEFAULT_PERSISTING_L2_CACHE_PERCENTAGE_LIMIT

The environment variable controls the default “set-aside” portion of the GPU's L2 cache reserved for *persistent accesses*, expressed as a percentage of the L2 size.

It is relevant for GPUs that support persistent L2 cache, specifically devices with *compute capability* 8.0 or higher when using the `CUDA Multi-Process Service (MPS)`. The environment variable must be set before starting the CUDA MPS Control Daemon, namely before running the `nvidia-cuda-mps-control -d` command.

Possible Values: Percentage value between 0 and 100, default is 0.

Example:

```
CUDA_DEVICE_DEFAULT_PERSISTING_L2_CACHE_PERCENTAGE_LIMIT=25 # 25%
```

5.2.3.8 CUDA_DISABLE_PERF_BOOST

On Linux hosts, setting this environment variable to 1 prevents boosting the device performance state, instead pstate can be selected implicitly based on various heuristics. This option can potentially be used to reduce power consumption, but may result in higher latency in certain scenarios due to dynamic performance state selection.

Example:

```
CUDA_DISABLE_PERF_BOOST=1 # perf boost disabled, Linux only.
CUDA_DISABLE_PERF_BOOST=0 # default behavior
```

5.2.3.9 CUDA_AUTO_BOOST [[deprecated]]

The environment variable affects the GPU clock “auto boost” behavior, namely dynamic clock boosting. It overrides the “auto boost” option of the `nvidia-smi` tool, namely `nvidia-smi --auto-boost-default=0`.

Note

This environment variable is deprecated. It is strongly suggested to use `nvidia-smi --applications-clocks=<memory,graphics>` or the [NVML API](#) instead of the `CUDA_AUTO_BOOST` environment variable.

5.2.4. Module Loading

5.2.4.1 CUDA_MODULE_LOADING

The environment variable affects how the CUDA runtime loads modules, specifically how it initializes the device code.

Possible Values:

- ▶ **DEFAULT:** Default behavior, equivalent to LAZY.
- ▶ **LAZY:** The loading of specific kernels is delayed until a CUDA function handle, `CUfunc`, is extracted using the `cuModuleGetFunction()` or `cuKernelGetFunction()` API calls. In this case, the data from the CUBIN is loaded when the first kernel in the CUBIN is loaded or when the first variable in the CUBIN is accessed.
 - ▶ The driver loads the required code on the first call to a kernel; subsequent calls incur no extra overhead. This reduces startup time and GPU memory footprint.
- ▶ **EAGER:** Fully loads CUDA modules and kernels at program initialization. All kernels and data from a CUBIN, FATBIN, or PTX file are fully loaded upon the corresponding `cuModuleLoad*` and `cuLibraryLoad*` driver API call.
 - ▶ Higher startup time and GPU memory footprint. Kernel launch overhead is predictable.

Examples:

```
CUDA_MODULE_LOADING=EAGER  
CUDA_MODULE_LOADING=LAZY
```

5.2.4.2 CUDA_MODULE_DATA_LOADING

The environment variable affects how the CUDA runtime loads data associated to modules.

This is a complementary setting to the kernel-focused setting in `CUDA_MODULE_LOADING`. This environment variable does not affect the LAZY or EAGER loading of kernels. Data loading behavior is inherited from `CUDA_MODULE_LOADING` if this environment variable is not set.

Possible Values:

- ▶ **DEFAULT:** Default behavior, equivalent to LAZY.
- ▶ **LAZY:** The loading of module data is delayed until a CUDA function handle, `CUfunc`, is required. In this case, the data from the CUBIN is loaded when the first kernel in the CUBIN is loaded or when the first variable in the CUBIN is accessed.
 - ▶ Lazy data loads can require context synchronization, which can slow down concurrent execution.
- ▶ **EAGER:** All data from a CUBIN, FATBIN, or PTX file are fully loaded upon the corresponding `cuModuleLoad*` and `cuLibraryLoad*` API call.

Example:

```
CUDA_MODULE_DATA_LOADING=EAGER
```

5.2.4.3 CUDA_BINARY_LOADER_THREAD_COUNT

Sets the number of CPU threads to use when loading device binaries. When set to 0, the number of CPU threads used is set to a default value of 1.

Possible Values:

- ▶ Integer number of threads to use. Defaults to 0, which uses 1 thread.

Example:

```
CUDA_BINARY_LOADER_THREAD_COUNT=4
```

5.2.5. CUDA Error Log Management

5.2.5.1 CUDA_LOG_FILE

The environment variable specifies a location where descriptive error log messages will be printed as they occur for supported CUDA API calls that returned an error.

For example, if one attempts to launch a kernel with an invalid grid configuration, such as `kernel<<<1, dim3(1,1,128)>>>(...)`, that kernel will fail to launch and `cudaGetLastError()` will return a generic invalid configuration argument error. If the `CUDA_LOG_FILE` environment variable is set, the user can see the following descriptive error message in the log: `[CUDA][E] Block Dimensions (1,1,128) include one or more values that exceed the device limit of`

(1024, 1024, 64) and easily determine that the specified z-dimension of the block was invalid. See [Error Log Management](#) for more details.

Possible Values: stdout, stderr, or a valid file path (with appropriate access permissions)

Examples:

```
CUDA_LOG_FILE=stdout
CUDA_LOG_FILE=/tmp/dbg_cuda_log
```

5.3. C++ Language Support

nvcc processes CUDA and device code according to the following specifications:

- ▶ **C++03** (ISO/IEC 14882:2003), --std=c++03 flag.
- ▶ **C++11** (ISO/IEC 14882:2011), --std=c++11 flag.
- ▶ **C++14** (ISO/IEC 14882:2014), --std=c++14 flag.
- ▶ **C++17** (ISO/IEC 14882:2017), --std=c++17 flag.
- ▶ **C++20** (ISO/IEC 14882:2020), --std=c++20 flag.
- ▶ **C++23** (ISO/IEC 14882:2024), --std=c++23 flag.

Passing nvcc -std=c++<version> flag turns on all C++ features related to the specified version and also invokes the host preprocessor, compiler and linker with the corresponding C++ dialect option.

The compiler supports all language features of the supported standards, subject to the restrictions reported in the following sections.

5.3.1. C++11 Language Features

Table 34: C++11 Language Features Supported by NVCC for device code

Language Feature	C++11 Proposal	NVCC/CUDA 7.x	Toolkit
<i>Rvalue references</i>	N2118	✓	
Rvalue references for *this	N2439	✓	
Initialization of class objects by rvalues	N1610	✓	
Non-static data member initializers	N2756	✓	
Variadic templates	N2242	✓	
Extending variadic template template parameters	N2555	✓	
<i>Initializer lists</i>	N2672	✓	
Static assertions	N1720	✓	
auto-typed variables	N1984	✓	

continues on next page

Table 34 – continued from previous page

Language Feature	C++11 Proposal	NVCC/CUDA 7.x	Toolkit
Multi-declarator auto	N1737	✓	
Removal of auto as a storage-class specifier	N2546	✓	
New function declarator syntax	N2541	✓	
<i>Lambda expressions</i>	N2927	✓	
Declared type of an expression	N2343	✓	
Incomplete return types	N3276	✓	
Right angle brackets	N1757	✓	
Default template arguments for function templates	DR226	✓	
Solving the SFINAE problem for expressions	DR339	✓	
Alias templates	N2258	✓	
Extern templates	N1987	✓	
Null pointer constant	N2431	✓	
Strongly-typed enums	N2347	✓	
Forward declarations for enums	N2764 DR1206	✓	
Standardized attribute syntax	N2761	✓	
<i>Generalized constant expressions</i>	N2235	✓	
Alignment support	N2341	✓	
Conditionally-supported behavior	N1627	✓	
Changing undefined behavior into diagnosable errors	N1727	✓	
Delegating constructors	N1986	✓	
Inheriting constructors	N2540	✓	
Explicit conversion operators	N2437	✓	
New character types	N2249	✓	
Unicode string literals	N2442	✓	
Raw string literals	N2442	✓	
Universal character names in literals	N2170	✓	
User-defined literals	N2765	✓	
Standard Layout Types	N2342	✓	
<i>Defaulted functions</i>	N2346	✓	
Deleted functions	N2346	✓	
Extended friend declarations	N1791	✓	

continues on next page

Table 34 – continued from previous page

Language Feature	C++11 Proposal	NVCC/CUDA 7.x	Toolkit
Extending sizeof	N2253 DR850	✓	
<i>Inline namespaces</i>	N2535	✓	
Unrestricted unions	N2544	✓	
<i>Local and unnamed types as template arguments</i>	N2657	✓	
Range-based for	N2930	✓	
Explicit virtual overrides	N2928 N3206 N3272	✓	
Minimal support for garbage collection and reachability-based leak detection	N2670	×	
Allowing move constructors to throw [noexcept]	N3050	✓	
Defining move special member functions	N3053	✓	
Concurrency			
Sequence points	N2239	×	
Atomic operations	N2427	×	
Strong Compare and Exchange	N2748	×	
Bidirectional Fences	N2752	×	
Memory model	N2429	×	
Data-dependency ordering: atomics and memory model	N2664	×	
Propagating exceptions	N2179	×	
Allow atomics use in signal handlers	N2547	×	
Thread-local storage	N2659	×	
Dynamic initialization and destruction with concurrency	N2660	×	
C99 Features in C++11			
__func__ predefined identifier	N2340	✓	
C99 preprocessor	N1653	✓	
long long	N1811	✓	
Extended integral types	N1988	×	

5.3.2. C++14 Language Features

Table 35: C++14 Language Features Supported by NVCC for device code

Language Feature	C++14 Proposal	NVCC/CUDA Toolkit 9.x
Tweak to certain C++ contextual conversions	N3323	✓
Binary literals	N3472	✓
<i>Functions with deduced return type</i>	N3638	✓
Generalized lambda capture (init-capture)	N3648	✓
Generic (polymorphic) lambda expressions	N3649	✓
<i>Variable templates</i>	N3651	✓
Relaxing requirements on constexpr functions	N3652	✓
Member initializers and aggregates	N3653	✓
Clarifying memory allocation	N3664	×
Sized deallocation	N3778	×
[[deprecated]] attribute	N3760	✓
Single-quotation-mark as a digit separator	N3781	✓

5.3.3. C++17 Language Features

Table 36: C++17 Language Features Supported by NVCC for device code

Language Feature	C++17 Proposal	NVCC/CUDA Toolkit 11.x
Removing trigraphs	N4086	✓
u8 character literals	N4267	✓
Folding expressions	N4295	✓
Attributes for namespaces and enumerators	N4266	✓
Nested namespace definitions	N4230	✓
Allow constant evaluation for all non-type template arguments	N4268	✓
Extending static_assert	N3928	✓
New Rules for auto deduction from braced-init-list	N3922	✓
Allow typename in a template template parameter	N4051	✓

continues on next page